

Introduction to First-Order Evasion Attacks

Machine learning models appear robust when tested on normal data, often achieving 98-99% accuracy on held-out test sets. Yet these same models can be fooled by adversarial examples: inputs carefully modified to cause misclassification while remaining nearly identical to the original (that's the hope anyway). A digit image that clearly shows a "7" to human eyes can be tweaked by less than 1% and suddenly the model confidently predicts "2."

What makes these attacks possible? Neural networks learn by following gradients during training, adjusting weights to minimize loss. The same gradient information that trains the model also reveals its vulnerabilities. First-order attacks exploit this by computing gradients with respect to inputs rather than parameters, using calculus to find directions that maximally increase prediction error.

What Are First-Order Attacks?

First-order attacks use gradient information to craft adversarial examples. During normal training, gradients tell us how to adjust model weights to reduce error. During an attack, gradients tell us how to adjust inputs to increase error.

Think of it as asking the model: "If I change this pixel slightly, how much does your prediction change?" The gradient provides that answer for every pixel simultaneously. An attacker uses those answers to make coordinated changes across all pixels, pushing the input across a decision boundary while keeping the total modification small.

These attacks work in different scenarios. In white-box attacks, the attacker has complete access to the model and can compute exact gradients. In black-box attacks, the attacker only queries the model's outputs, either estimating gradients numerically or crafting examples on a similar surrogate model and relying on transferability.

The key constraint is keeping perturbations small as possible. We measure this using norms (covered in detail in the next chapter).

Two Fundamental Approaches

This module explores two foundational gradient-based attacks that represent different attack philosophies.

FGSM (Fast Gradient Sign Method) takes the direct approach. You decide upfront how much you're willing to change the input, then compute the gradient and move each pixel in the direction that increases loss. The "sign" part means you only look at whether each gradient component is positive or negative, not how large it is. This makes the attack extremely fast: one gradient computation, one step, done. It's simple enough to implement in

a few lines of code, yet effective enough that it became the baseline for measuring adversarial robustness.

DeepFool asks a fundamentally different question: what's the smallest change that fools the model? Instead of choosing a budget upfront, DeepFool iteratively searches for the closest decision boundary. It approximates the boundary as a flat surface locally, takes the shortest step to that surface, then repeats with a fresh approximation. After a few iterations, it reaches the true boundary with minimal perturbation. This tells you exactly how vulnerable each input is, making it valuable for measuring model robustness quantitatively.

Why This Matters

Adversarial examples expose a clear gap between high accuracy and true robustness. A model can correctly classify 99% of normal test images while catastrophically failing under adversarial examples. This vulnerability matters in security-critical applications.

First-order attacks are also remarkably transferable. An adversarial example crafted against one model often fools other models, even those with different architectures or training procedures. This enables realistic attacks where the adversary doesn't need full access to the target model.

For defenders, understanding these attacks is essential. They provide baselines for evaluating defensive techniques like adversarial training, input preprocessing, and detection systems. They also help identify which inputs lie close to decision boundaries and might be vulnerable to natural perturbations.

Security Frameworks

Both of the major AI security frameworks (OWASP and SAIF) recognize evasion attacks as a major threat. OWASP's Machine Learning Security Top 10 lists input manipulation as ML01:2023, the highest-ranked risk for traditional ML systems. The framework distinguishes these inference-time attacks from training-time threats like data poisoning, noting that evasion requires only the ability to query a deployed model.

Google's Secure AI Framework (SAIF) addresses evasion through defense in depth: adversarial training during development, robustness evaluation before deployment, and input filtering during operation. SAIF recommends that security teams establish red teams to continuously test production models with adversarial examples, measuring how much perturbation is needed to achieve target attack success rates.

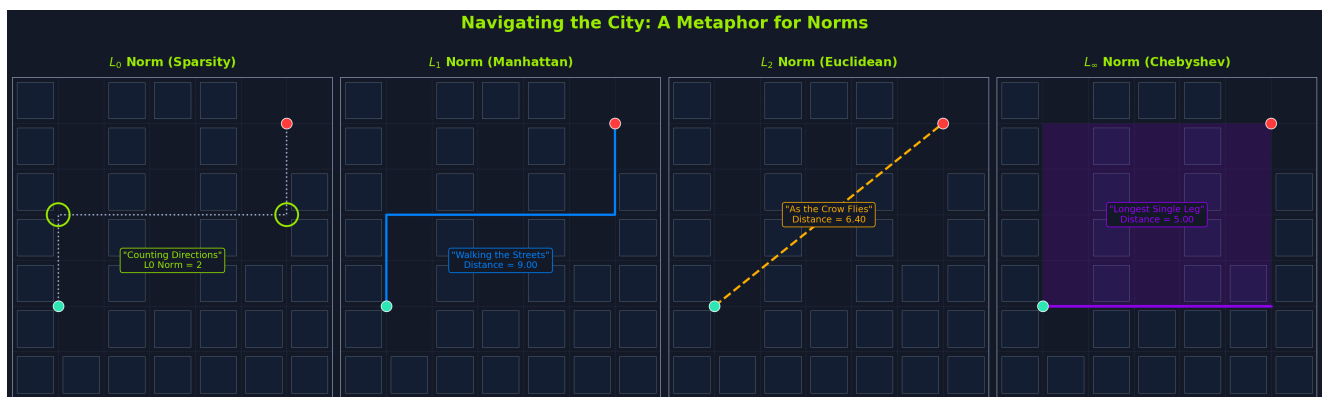
Both frameworks emphasize that defending against evasion requires technical understanding of how these attacks work. The techniques in this module provide that foundation, enabling you to implement OWASP's recommended defenses and execute SAIF's evaluation protocols.

Understanding Norms

When we talk about adversarial attacks on machine learning models, one of the most important questions is: "How much did we change the input?" This might seem simple - just look at the image and see if it looks different, but computers need precise mathematical ways to measure these changes, and that's where `norms` come in.

Think of a norm as a ruler for measuring changes. Just like you might measure distance in miles or kilometers, we use different norms to measure how much an adversarial attack has modified an input. The fascinating part is that different "rulers" (norms) measure changes in completely different ways, leading to very different types of attacks.

What Exactly Is a Norm?



Let's start with something familiar. Imagine you're in New York City and want to travel from Times Square to Central Park. How far is it? The answer depends on how you measure:

- If you count how many intersections you pass through (regardless of distance), that's one measure
- If you have to walk along the city streets (following the grid), that's a different distance
- If you can fly in a straight line (like a bird), that's another distance
- If you only care about the longest single stretch you have to travel, that's yet another way to measure

These different ways of measuring distance are exactly what norms do for adversarial perturbations. A `norm` is simply a mathematical tool that assigns a "size" or "length" to changes we make to an input.

The Three Rules Every Norm Must Follow

For something to qualify as a proper norm, it must follow three common-sense rules:

1. `Zero means zero`: The only thing with zero length is... nothing. If your measurement says something has zero size, it better actually be zero. This prevents our "ruler" from lying to us.
2. `Doubling means doubling`: If you make a change twice as big, the measurement should also be twice as big. This keeps our measurements consistent and predictable.

3. Shortcuts don't exist: The direct path between two points is never longer than going the roundabout way. In math terms, this is called the "triangle inequality" - imagine a triangle where one side can't be longer than the other two sides combined.

These rules ensure our measurement system makes sense and behaves predictably.

The p-Norm Family: Different Rulers for Different Jobs

The most common norms we use are called **p-norms**, and they're like a family of related measuring tools. The "p" is just a number that determines which family member we're using. Here's the formula:

$$\|x\|_p = (\sum_{i=1}^n |x_i|^p)^{1/p}$$

This formula is just saying: "Take each change, raise it to the power p, add them all up, then take the p-th root." Different values of p give us different measuring tools:

When

$$p=1$$

we get the

L_1

norm (like walking in Manhattan). When

$$p=2$$

we get the

L_2

norm (like flying in a straight line). When

$$p=\infty$$

we get the

L_∞

norm (just look at the biggest change).

As p increases, the norm increasingly focuses on larger values. With

$$p=1$$

all changes contribute equally to the sum. With

$$p=2$$

bigger changes matter more because of the squaring. And as p approaches infinity, only the single largest change matters at all.

But wait, what about

L_0 ?

Here's where things get interesting. The

L_0

"norm" is actually an imposter in the p-norm family. It doesn't follow the formula above at all. You can't set

$$p=0$$

in that equation and get something that counts non-zero elements.

Instead,
 L_0
 has its own completely different definition:

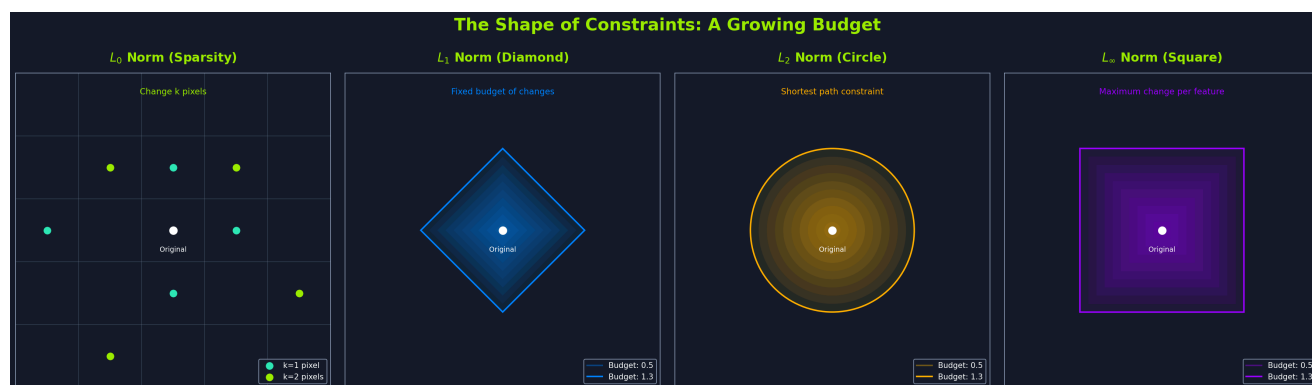
$$\|x\|_0 = \sum_i 1[x_i \neq 0] \quad \|x\|_0 = \sum_i \mathbb{1}[x_i \neq 0]$$

This just counts how many elements are non-zero, regardless of their magnitude. A pixel changed by 0.001 counts the same as one changed by 255. Despite not being a true p-norm (or even a proper norm, since it violates the scaling rule), we use

L_0
 constantly in adversarial ML because it captures something the p-norms can't: pure sparsity. It answers the question "how many things did we touch?" rather than "how much did we change them?"

The Shape of Constraints

Here's where it gets interesting. When we limit perturbations to a maximum norm value, we're essentially drawing a boundary that says "you can change the input, but only within this shape."



Imagine you're standing at a point and can move a fixed "distance" in any direction:

With the

L_0

norm, you can move in directions that change at most

k

coordinates (you can change

k

pixels to any value within the valid range). In continuous spaces, this

feasible set is a union of axis-aligned subspaces and is unbounded

unless magnitudes are also bounded. With the

L_1

norm, you can reach anywhere within a diamond shape. With the

L_2

norm, you can reach anywhere within a circle. With the

L_∞

norm, you can reach anywhere within a square.

These shapes matter because they determine which directions are "cheaper" to move in. With

L_0 ,

you get complete freedom in how much you change the selected pixels, but strict limits on how many you can touch; in practice, we often pair an

L_0

limit with a per-pixel magnitude bound (e.g., the valid image range) to make the set well-behaved. With

L_1 ,

moving diagonally costs more than moving along the axes. With

L_∞ ,

you can move equally far in any direction as long as no single coordinate changes too much.

How Different Norms Relate to Each Other

Different norms aren't completely independent - they're connected by mathematical relationships. Think of it like converting between miles and kilometers. For any set of changes in an n -dimensional space:

$$\|x\|_\infty \leq \|x\|_2 \leq \|x\|_1 \leq n \cdot \|x\|_2 \leq n \cdot \|x\|_\infty \quad \|x\|_2 \leq \|x\|_1 \leq \sqrt{n} \cdot \|x\|_2 \leq n \cdot \|x\|_\infty$$

And for the

L_0

norm, which counts non-zero elements:

$$\|x\|_0 \leq n \text{ and } \|x\|_2 \leq \|x\|_0 \cdot \|x\|_\infty \quad \|x\|_0 \leq n \quad \text{and} \quad \|x\|_2 \leq \sqrt{\|x\|_0} \cdot \|x\|_\infty$$

This means if you limit changes using one of the p -norms, you automatically get some limits on the others too. It's like saying "if you can only walk 10 blocks in Manhattan distance, there's also a limit to how far you could have flown in a straight line."

For

L_0 ,

there's an extra wrinkle:

counting how many coordinates you change does not, by itself, bound how large those changes can be.

To translate an

L_0

limit into bounds on

L_1

or

L_2 ,

you also need a per-coordinate magnitude bound (e.g., an

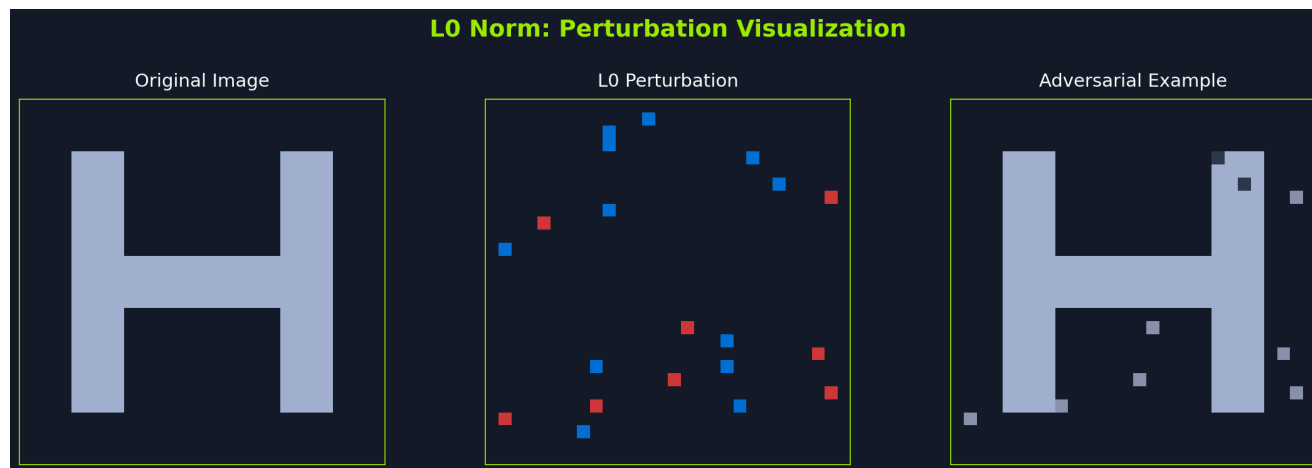
L_∞

cap or the valid pixel range). The inequality above,

$\|x\|_2 \leq \|x\|_0^{1/2} \|x\|_\infty^{1/2} \leq \|x\|_0^{1/2} \sqrt{\text{valid pixel range}}$,

is exactly this combination: sparsity plus a size limit.

The L0 Norm: Counting What Changed



Now let's dive into the specific norms used in adversarial attacks.

The

L_0

norm is the simplest conceptually - it just counts how many pixels (or features) you changed. Period. It doesn't care if you changed a pixel by a tiny amount or completely flipped it from black to white. Changed is changed.

$$\|\delta\|_0 = \sum_i \mathbb{1}[\delta_i \neq 0] \quad \|\delta\|_0 = \sum_i \mathbb{1}[\delta_i \neq 0]$$

This formula uses an "indicator function" (the

$\mathbb{1}$

symbol) that returns 1 if something changed and 0 if it didn't. Then it adds up all the 1s to count the total changes.

Imagine you're editing a photo and you can only use the paintbrush tool on 10 pixels. You could make those 10 pixels any color you want, but you can only touch 10 of them. That's an

L_0

constraint.

Advantages:

- Very interpretable - you know exactly which parts of the input matter

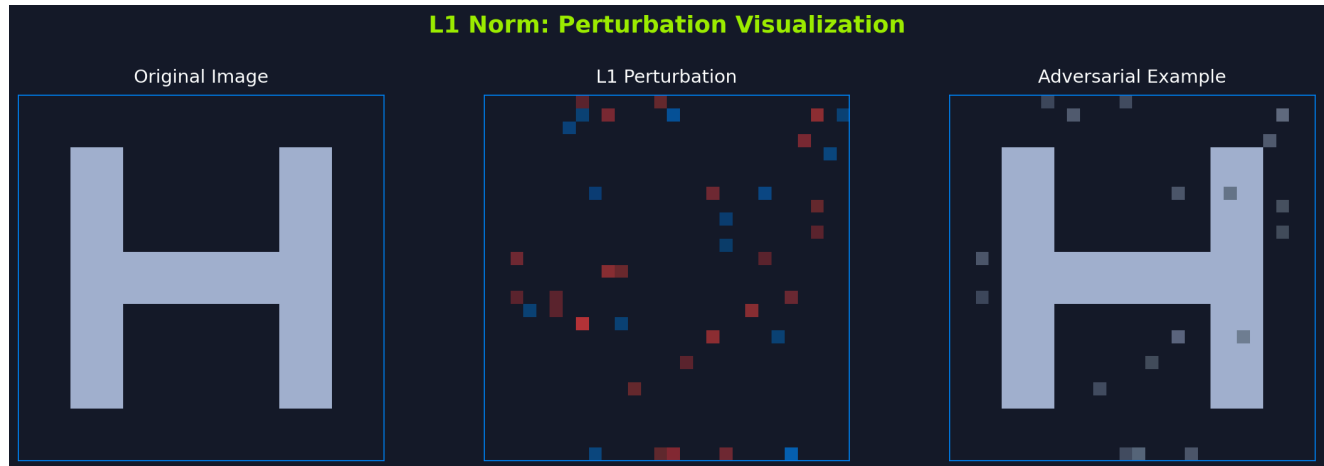
- Can make large changes to a few pixels while leaving the rest untouched

Disadvantages:

- Those few changed pixels might stand out like sore thumbs
- Mathematically difficult to optimize (it's not smooth or convex)

Fun fact: The L0 "norm" technically isn't a true norm because it violates rule #2 (doubling the changes doesn't double the count). But everyone calls it a norm anyway because it's useful.

The L1 Norm: Total Change Budget



The

L1

norm adds up the absolute value of all changes. It's like having a budget for how much total change you can make, and you can spread it around however you want.

$$\|\delta\|_1 = \sum_i |\delta_i|$$

Think of it like having 100 pixels to distribute among modifications. You could:

- Make 100 pixels each 1% different
- Make 10 pixels each 10% different
- Make 1 pixel 100% different
- Any combination that adds up to 100%

The

L1

norm tends to create sparse perturbations, it naturally concentrates the budget on a subset of coordinates rather than spreading tiny changes everywhere. This happens because of its diamond-shaped constraint boundary, which has "corners" that favor solutions with many zeros; the selected coordinates may change more, while many others stay exactly the same.

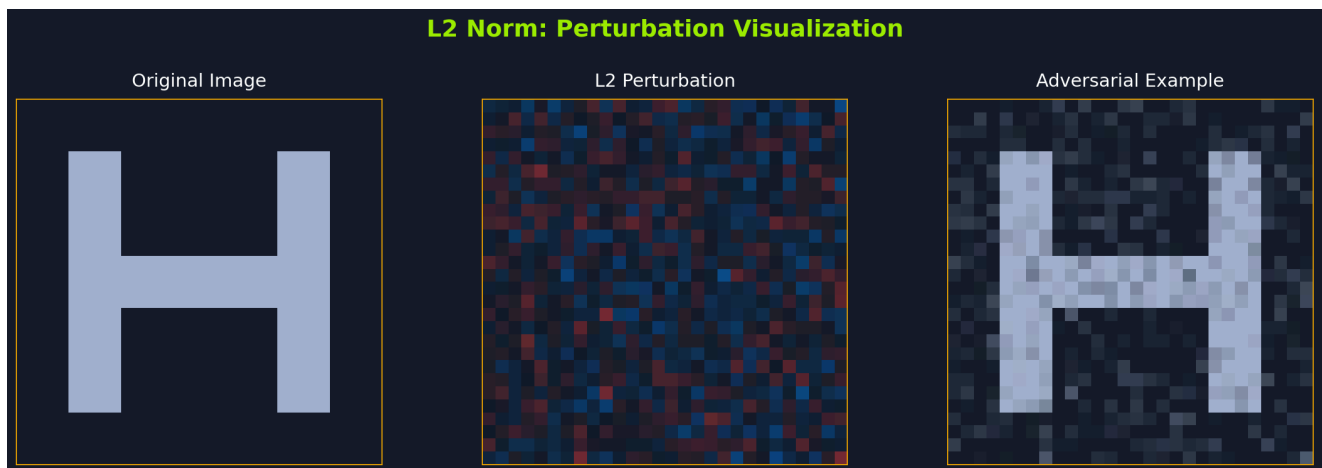
Advantages:

- Creates a nice balance between L0 and L2 behaviors
- Mathematically easier to work with than L0 (it's convex)
- Perturbations often look like scattered specks of noise

Disadvantages:

- Not as smooth to optimize as L2
- Can create visible "salt and pepper" noise patterns

The L2 Norm: Smooth, Even Changes



The

L2L₂

norm is the famous Euclidean distance - straight-line distance in space.

It measures perturbations by taking the square root of the sum of squared changes:

$$\|\delta\|_2 = \sqrt{\sum_i \delta_i^2} \quad \|\delta\|_2 = \sqrt{\sum_i \delta_i^2}$$

This is the same distance formula you learned in geometry class! The squaring operation makes this norm heavily penalize large individual changes. If you try to change one pixel by a lot, the square of that change becomes huge. So L2 perturbations naturally spread changes evenly across all pixels.

Imagine you're adding a thin layer of fog to an image. The fog affects everything equally, making the whole image slightly different rather than noticeably changing specific parts.

Advantages:

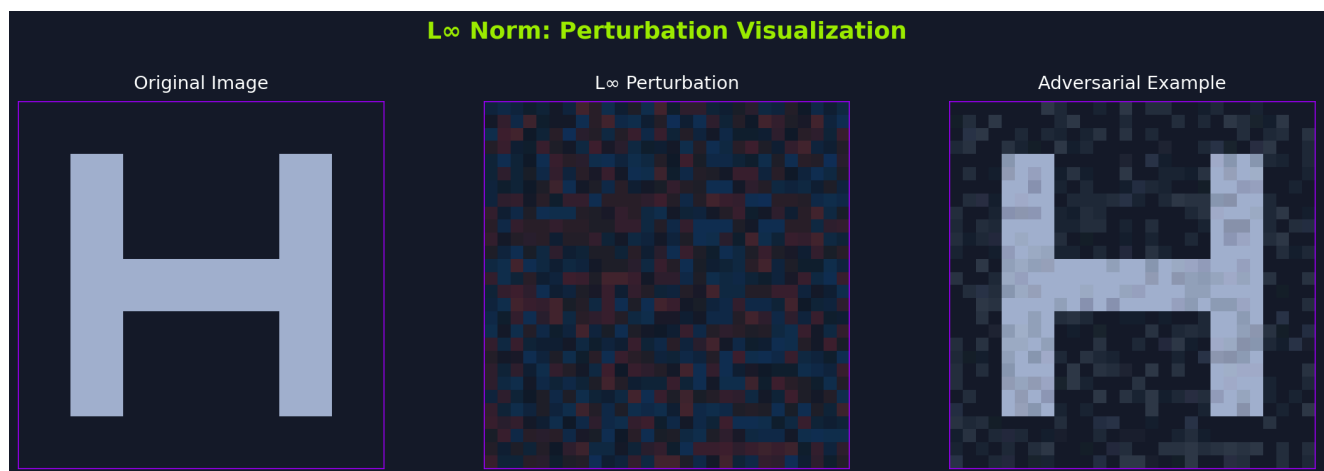
- Creates the smoothest, most imperceptible perturbations
- Mathematically well-behaved (convex and smooth away from 0). In practice, many methods use the squared L2 norm, which is differentiable everywhere and especially convenient for optimization.

- Well-understood optimization properties

Disadvantages:

- Changes everything at least a little bit
- Less interpretable - harder to see which features matter most
- May create a visible "haze" over the entire image

The L^∞ Norm: Maximum Change Limit



The

L^∞ norm

(L -infinity) only cares about the biggest single change you made. It's like a speed limit - you can go any speed you want as long as you never exceed the limit.

$$\|\delta\|_\infty = \max_i |\delta_i| \quad \|\delta\|_\infty = \max_i |\delta_i|$$

With an

L^∞ norm

constraint of 0.1, every pixel can change by up to 10%, but none can change by more than that. This creates perturbations where many pixels might be changed by similar amounts, up to the maximum allowed.

Advantages:

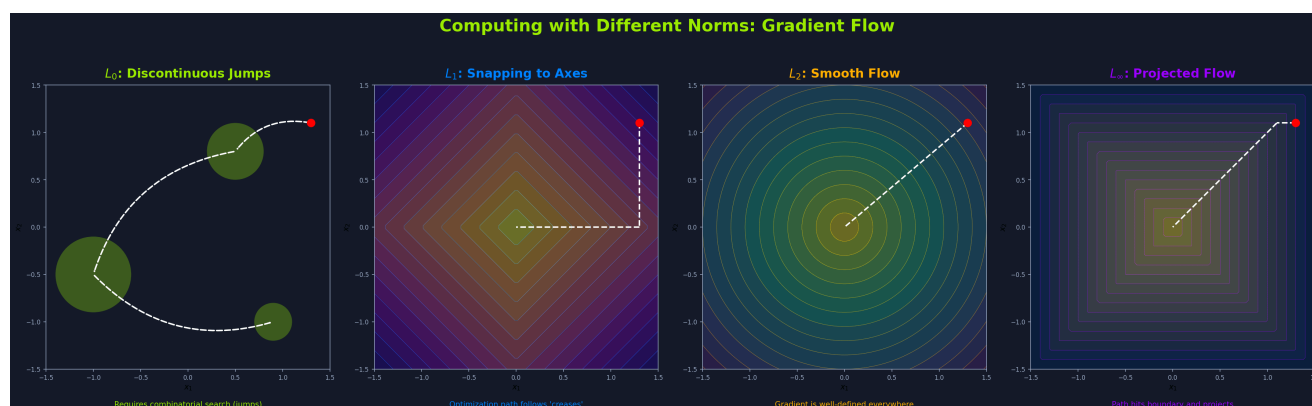
- Very intuitive constraint - "no pixel changes by more than X"
- Common in adversarial training defenses
- Creates uniform-looking perturbations

Disadvantages:

- Might change many pixels unnecessarily
- The hard maximum can be restrictive for optimization

Computing with

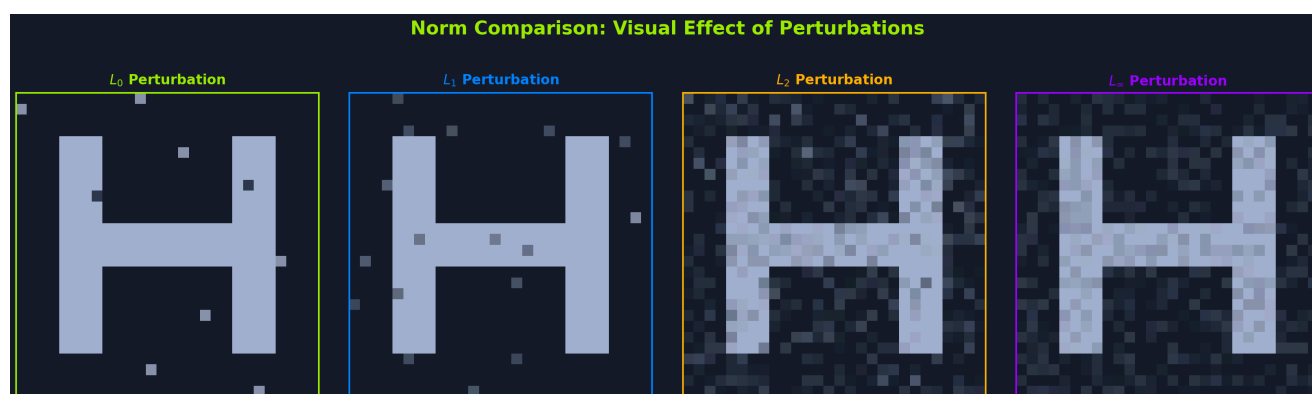
Different Norms



From a practical standpoint, different norms have different computational properties:

The L_0 norm is discontinuous and non-convex, often requiring greedy algorithms or combinatorial search. It's the "expert mode" of norm constraints. The L_1 norm has a "kink" at zero (the absolute value function isn't smooth there) and requires special optimization techniques like soft-thresholding or proximal gradient methods. The L_2 norm is smooth and differentiable everywhere, with gradients flowing nicely, making optimization straightforward. It's the "vanilla ice cream" of optimization - reliable and well-behaved. The L_∞ norm is efficiently computed but has sparse gradients (only the maximum element has a non-zero gradient) and is often handled with projected gradient descent.

The Bottom Line



Norms are the primary language we use to measure and constrain adversarial perturbations. Each norm -

L_0 ,

L_1 ,

L_2 ,

and

L_∞

- offers different trade-offs between imperceptibility, computational efficiency, and attack effectiveness. Understanding these trade-offs is crucial for both creating strong attacks and building robust defenses.

As the field evolves, we're seeing more sophisticated perturbation metrics that better capture human perception. But the classical norms remain central to adversarial ML because they provide a precise, mathematical framework for reasoning about attacks and defenses.

FGSM

The Fast Gradient Sign Method (FGSM) arrived as a bit of a wake-up call for the deep learning field.

In 2014, Goodfellow et al. introduced FGSM in "[Explaining and Harnessing Adversarial Examples](#)," showing that a classifier that looks reliable on clean data can be flipped by tweaks so small that humans hardly notice. Start with an input the model labels correctly. Add a calculated nudge that pushes the network where it is vulnerable. The prediction changes. That simple recipe seeded modern adversarial machine learning.

The method turns the model's own learning signal against it. Instead of changing parameters to reduce error, the attack changes the input to increase it. The gradient computation reveals how the loss reacts to each pixel, keeping only the direction of that reaction to take a tiny, coordinated step. The mathematics below formalizes that idea, but the approach is straightforward: use gradients to find a direction that the network dislikes and move there, just a little.

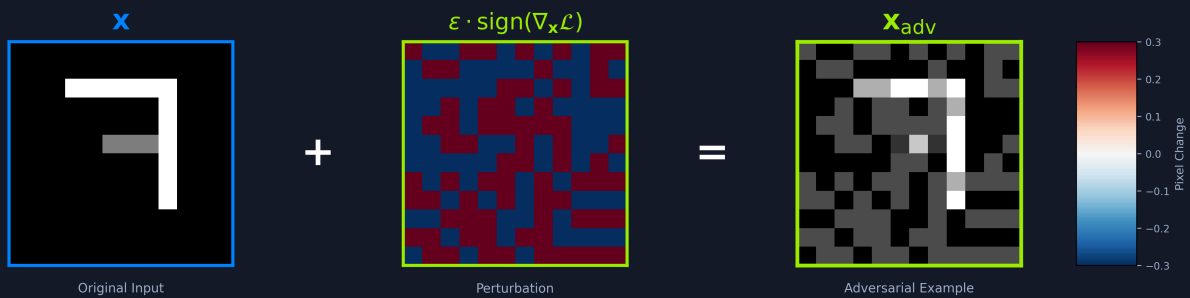
This matters because it exposes fragility that accuracy alone hides. High-performing models in high-dimensional spaces can be sensitive to small, structured changes that human vision ignores. In settings that demand reliability, that sensitivity is a security concern and a design constraint.

We'll first see the core formula and its intuition, then understand why the attack succeeds through local linearity and high dimensionality, and finally explore the mathematical foundations that explain its optimality as a solution to a constrained optimization problem.

FGSM Core Idea

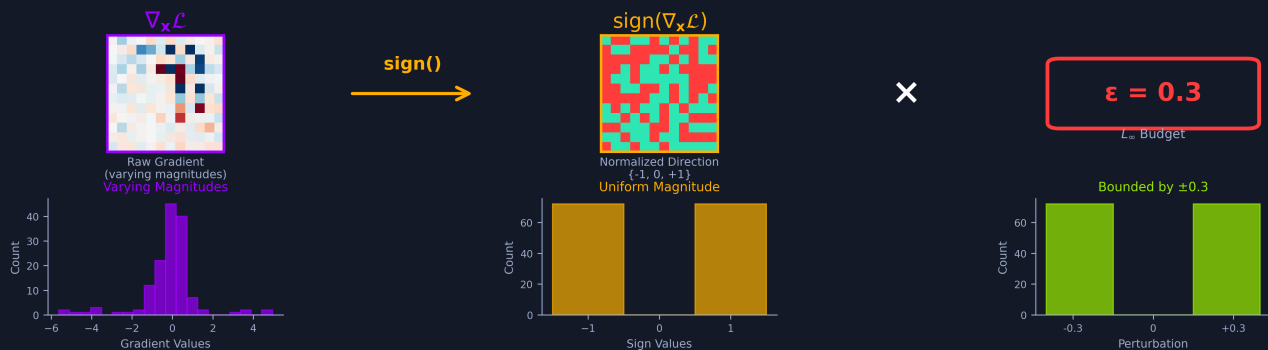
FGSM: Fast Gradient Sign Method

$$\mathbf{x}_{\text{adv}} = \mathbf{x} + \epsilon \cdot \text{sign}(\nabla_{\mathbf{x}} \mathcal{L})$$



Why Take the Sign?

The sign operation ensures L_∞ constraint: every pixel changes by exactly $\pm\epsilon$. This maximizes the loss increase while keeping perturbations imperceptible.



In plain terms, the gradient is a map of sensitivities: it indicates which way to nudge each pixel to make the loss increase the fastest. The $\text{sign}()$ operation extracts only the direction of that nudge for each pixel and discards how large the change would be, so the algorithm changes every pixel by the same small amount ϵ but in the direction that increases the loss. The L_∞ budget limits the maximum absolute change to any single pixel, not the average change across all pixels. (See the Introduction's Understanding Norms section for the explanations of L_∞ , L_2 , L_1 , and their geometric interpretations if you need a reminder.)

Formally, FGSM produces adversarial examples by taking a single step in input space that maximally increases model loss while keeping every pixel change bounded by an

L_∞

budget. With input

x ,

label

y ,

parameters

θ ,

and loss

$\mathcal{L}(\theta, x, y)$,

the adversarial input is

$$x_{\text{adv}} = x + \epsilon \text{sign}(\nabla_x \mathcal{L}(\theta, x, y)).$$

This expression leverages local linearity around

x .

A first-order Taylor view suggests that the steepest ascent in loss

arises along the gradient direction. Taking only the

sign

of the gradient equalizes step size across pixels, so the per-pixel

change is exactly

ϵ

in magnitude, which fits the

L_∞

constraint. The parameter

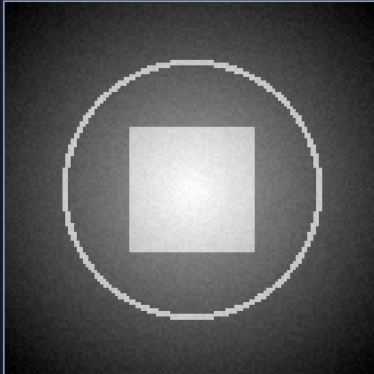
ϵ

is the perturbation budget that trades off imperceptibility and attack strength.

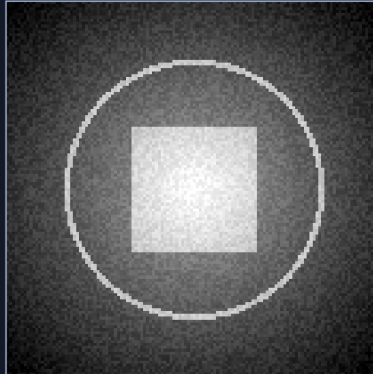
L_∞ : Maximum Change Limit

Every pixel bounded by $|\delta_i| \leq \epsilon$.

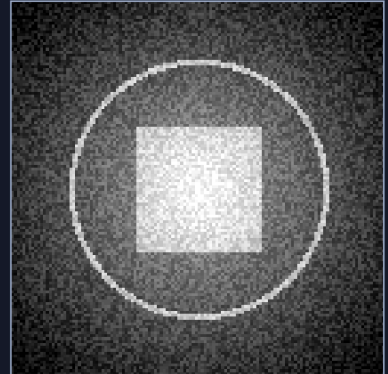
$\epsilon = 0.02$ (max per-pixel)



$\epsilon = 0.06$ (max per-pixel)



$\epsilon = 0.12$ (max per-pixel)



Uniform bound yields many pixels changed by similar amounts, none exceeding the cap.

Why FGSM Works

FGSM works for two simple reasons: local linearity and high dimensionality. Near a clean image, deep nets behave almost linearly (small input changes produce proportional score changes), so one gradient-based step already pushes the loss in the steepest direction. And because images have many pixels, tiny, correctly aligned tweaks at each one add up to a large change in the model's score. The

L_∞

constraint caps every pixel's change at

ϵ ,

giving a practical per-pixel visibility limit while still letting those many small tweaks accumulate.

To make this precise, consider a simple linear model where the accumulation effect becomes mathematically explicit.

For a linear score function

$f(x) = w^T x$,

the logit change caused by any perturbation with

$\|\delta\|_\infty \leq \epsilon$

satisfies the bound

$$|f(x+\delta) - f(x)| = |w^T \delta| \leq \epsilon \|w\|_1. |f(x+\delta) - f(x)| = \|w\|_1 \|\delta\|_\infty \leq \epsilon \|w\|_1.$$

<https://academy.hackthebox.com/path/preview/ai-red-teamer>

In high dimensions,

$$\|w\|_1 \|w\|_\infty$$

can be large even when individual components are modest, which illustrates how small per-pixel changes aligned with the gradient can accumulate into a substantial output shift. For example, if an image has 784 pixels (28×28 MNIST) and each weight component has magnitude 0.01, then

$$\|w\|_1 = 784 \times 0.01 = 7.84 \|w\|_\infty = 784 \times 0.01 = 7.84.$$

With

$$\epsilon = 0.1 \epsilon = 0.1,$$

the maximum logit change is

$$0.1 \times 7.84 = 0.784 0.1 \times 7.84 = 0.784,$$

which can easily flip a decision.

This inequality matters because it turns the vague idea of high-dimensional sensitivity into a concrete statement. Even though each pixel changes by at most ϵ , the aggregate effect grows with how many dimensions contribute and how strongly they align with the gradient. That is why small, structured perturbations can move a classifier's decision.

Constrained Linearization

Why does FGSM work so well? One view comes from optimization theory. The algorithm solves a specific problem exactly: finding the worst perturbation within our budget. The attack essentially asks "which direction hurts the model most?" and the sign-based step provides a precise answer. To see this, we approximate the curved loss surface with a flat plane that touches it at our starting point, turning a hard problem into one with a known solution.

The FGSM update arises as the exact optimizer of a first-order approximation to the inner maximization problem. Consider the adversarial objective that seeks a perturbation within an

$$L_\infty$$

ball that maximizes the loss, written as

$$\max_{\delta \in \mathbb{R}^n} \mathcal{L}(\theta, x + \delta, y) \text{ s.t. } \|\delta\|_\infty \leq \epsilon. \max_{\|\delta\|_\infty \leq \epsilon} \mathcal{L}(\theta, x + \delta, y) \quad \text{s.t.} \quad \|\delta\|_\infty \leq \epsilon.$$

Using a first-order Taylor expansion around

x

yields the approximation

$$\mathcal{L}(\theta, x+\delta, y) \approx \mathcal{L}(\theta, x, y) + \nabla_x \mathcal{L}(\theta, x, y)^\top \delta. \\ \mathcal{L}(\theta, x, y) + \|\nabla_x \mathcal{L}(\theta, x, y)\| \|\delta\| \leq \mathcal{L}(\theta, x+\delta, y) \leq \mathcal{L}(\theta, x, y) + \|\nabla_x \mathcal{L}(\theta, x, y)\| \|\delta\|.$$

The constant term does not affect the maximizer, so the approximate problem reduces to

$$\max_{\|\delta\| \leq \epsilon} \nabla_x \mathcal{L}(\theta, x, y)^\top \delta. \quad \max_{\|\delta\| \leq \epsilon} \nabla_x \mathcal{L}(\theta, x, y)^\top \delta \leq \|\nabla_x \mathcal{L}(\theta, x, y)\| \epsilon.$$

To find this maximum, we need a fundamental result from analysis.

Hölder's inequality states that for vectors

u

and

v

and conjugate exponents

p

and

q

satisfying

$$\frac{1}{p} + \frac{1}{q} = 1,$$

the dot product is bounded by

$$|u^\top v| \leq \|u\|_p \|v\|_q.$$

The specific case relevant here pairs

$$L^\infty - L^1$$

with

$$L^1 - L^\infty$$

as dual norms (since

$$\frac{1}{\infty} + \frac{1}{1} = 0 + 1 = 1),$$

giving the bound

$$|\nabla_x \mathcal{L}(\theta, x, y)^\top \delta| \leq \|\nabla_x \mathcal{L}(\theta, x, y)\|_1 \|\delta\|_\infty \leq \|\nabla_x \mathcal{L}(\theta, x, y)\|_1 \epsilon.$$

When

$$\|\delta\|_\infty = \epsilon,$$

this becomes

$$|\nabla_x \mathcal{L}(\theta, x, y)^\top \delta| \leq \|\nabla_x \mathcal{L}(\theta, x, y)\|_1 \epsilon,$$

and equality holds exactly when

$$\delta_i$$

is aligned component-wise with the sign of

$$g_i.$$

By Hölder's inequality and

$$L^\infty - L^1$$

duality, the optimal value equals

$$\epsilon \|\nabla_x \mathcal{L}(\theta, x, y)\|_1$$

and is attained at the vertex that aligns each component with the sign of the gradient. Specifically,

$$\delta^* = \epsilon \text{sign}(g), \max \|\delta\|_\infty \leq \epsilon \top \delta = \epsilon \|g\|_1. \delta^* = \epsilon \text{sign}(g), \quad \max_{\|\delta\|_\infty \leq \epsilon} g^\top \delta = \epsilon \|g\|_1.$$

Substituting

$$g = \nabla_x \mathcal{L}(\theta, x, y)$$

gives the FGSM update

$$x_{\text{adv}} = x + \delta^*,$$

which exactly solves the linearized constrained problem. The targeted variant follows by minimizing the loss towards a chosen target

y_t ,

equivalently maximizing the negative loss, which flips the sign of the step.

This linearization replaces the curved loss surface near

x

with a flat plane that just touches it at

x .

Maximizing a flat plane over a box-shaped region (the

L_∞ ball)

pushes each coordinate of the perturbation to its boundary with

the same sign as the gradient at that coordinate. The duality language

is a compact way of stating this alignment result.

Targeted and Untargeted Formulations

The original paper presents the untargeted objective that increases the loss for the true label:

$$x_{\text{adv}} = x + \epsilon \text{sign}(\nabla_x \mathcal{L}(\theta, x, y)). \quad \max_{\|\delta\|_\infty \leq \epsilon} \mathcal{L}(\theta, x + \delta, y).$$

The targeted variant, which became standard practice after the

original work, decreases the loss for a desired target class

y_t :

$$x_{\text{adv}}^{\text{target}} = x - \epsilon \text{sign}(\nabla_x \mathcal{L}(\theta, x, y_t)). \quad \min_{\|\delta\|_\infty \leq \epsilon} \mathcal{L}(\theta, x + \delta, y_t).$$

This distinction only affects the step direction and which label is

used in the gradient computation. The

L_∞

budget and clipping remain unchanged.

Dual Norm Perspective and Alternative Budgets

One way to remember these norms is to view

L_2

as overall energy across pixels,

L_1

as the total absolute change summed across pixels, and

L_∞

as the single largest change over all pixels. Duality means that if you constrain one kind of size for the perturbation, the worst-case dot product is controlled by the complementary kind of size for the gradient.

The FGSM sign step is specific to the

L_∞

constraint. For an

L_2 -bounded

perturbation of radius

ϵ ,

the linearized maximizer takes the direction of the gradient normalized in

L_2 ,

written as

$$\delta^* = \epsilon \frac{g}{\|g\|_2}, x_{adv} = x + \delta^*, \quad \delta^*_{L_2} = \epsilon \frac{g}{\|g\|_2}, \quad x_{adv} = x + \delta^*_{L_2}.$$

For an

L_1 -bounded

perturbation, the linearized maximizer concentrates the budget on coordinates with largest absolute gradient components. In practice, a sparse update that assigns

$\pm \epsilon$ to

along the top coordinates by magnitude approximates this solution. These alternatives follow from the duality between

L_p

and

L_q

norms with

$$\frac{1}{p} + \frac{1}{q} = 1.$$

Backpropagation to Inputs

The reason for computing this input gradient in an attack is straightforward. During standard training, gradients flow with respect to the parameters to tell the optimizer how to change the model. In an adversarial attack, the parameters are frozen and the object being

changed is the input itself. The attack therefore asks a different question: if the pixels were nudged a tiny amount, in which direction would the loss increase fastest. The answer to that question is exactly the vector

$$\nabla_x \mathcal{L}(\theta, x, y)$$

This input gradient is the quantity that appears in the linearized inner maximization. The first-order objective becomes

$$g^T \delta g^{\text{top}} \delta$$

with

$$g = \nabla_x \mathcal{L}(\theta, x, y)$$

so the algorithm must compute

gg

to know how to choose

δ .

Backpropagation to inputs provides

gg

efficiently in a single backward pass through the network with

xx

treated as a variable and

θ

kept fixed. In practical terms, the implementation sets

`requires_grad=True` on x , runs a forward pass

to compute the loss, calls `loss.backward()`, and reads

`x.grad` as the sensitivity map.

Let

$$z_i(x)$$

denote the pre-softmax logit for class

i

and let

$$p_i(x) = \text{softmax}(z(x))_i$$

With cross-entropy loss

$$\mathcal{L}(\theta, x, y) = -\log p_y(x)$$

the gradient with respect to input can be expressed via the chain rule

as

$$\nabla_x \mathcal{L}(\theta, x, y) = \sum_i (p_i(x) - \mathbb{1}[i=y]) \nabla_x z_i(x)$$

For a piecewise-linear network composed of affine layers and ReLU

activations, the mapping from inputs to logits is affine within each

activation region. Consequently, the Jacobians

$$\nabla_x z_i(x)$$

are constant within a region, which explains why a first-order step aligned with the gradient can be so effective locally.

The coefficients

$$p_i(x) - \mathbb{1}_{[i=y]} p_y(x) - \mathbb{1}_{[i=y]}$$

weight how each class's logit Jacobian contributes. Note that for the true class

y ,

the coefficient is

$$p_y - 1 = -(1 - p_y) p_y - 1 = -(1 - p_y):$$

as confidence grows

$$(p_y \rightarrow 1 \text{ as } p_y \rightarrow 1),$$

its magnitude shrinks, so this term does not dominate. The remaining terms (for

$$i \neq y)$$

sum to

$$1 - p_y - p_y$$

and push to increase competing logits. Altogether

$$\nabla_x \mathcal{L} = \left(\sum_i p_i \nabla_x z_i \right) - \nabla_x z_y \quad \nabla_x \mathcal{L} = \left(\sum_i p_i \nabla_x z_i \right) - \nabla_x z_y$$

points in the direction that most increases the loss by both decreasing the true-class logit and increasing others, within the local linear region.

For example, if

$$p_y(x) = 0.95$$

for the true class, the coefficient for

$$i = y$$

is

$$0.95 - 1 = -0.05$$

which is small in magnitude. The non-true coefficients collectively sum to

$$1 - p_y = 0.05$$

and distribute across

$$i \neq y$$

The resulting gradient gently lowers

$$z_y$$

and gently raises competing logits. By contrast, if

$$p_y(x) = 0.20$$

then

$$p_y - 1 = -0.80$$

and the non-true coefficients sum to

$$0.80$$

yielding a much larger gradient that more forcefully reduces the true-class logit while boosting others.

FGSM Setup

Library Installation

This module uses the HTB Evasion Library which provides common utilities. Install it first:

```
# Install the AI Library (or update it)
pip install --upgrade git+https://github.com/PandaSt0rm/htb-ai-library
```

Environment Setup

Reproducibility matters. Without it, gradient computations can vary between runs even on identical inputs, making debugging impossible and comparisons meaningless. The Library provides shared utilities to eliminate these sources of randomness while keeping the setup minimal:

```
import os
import random
import numpy as np
import torch
from torch import nn, Tensor
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

# Import common utilities from HTB Evasion Library
from htb_ai_library import (
    set_reproducibility,
    SimpleCNN,
    get_mnist_loaders,
    mnist_denormalize,
    train_model,
    evaluate_accuracy
)

# Configure reproducibility
set_reproducibility(1337)
```

How does `set_reproducibility` enforce determinism across multiple libraries? It controls three independent randomness sources. First, the `PYTHONHASHSEED` environment variable locks Python's built-in hash randomization, ensuring dictionary iteration order remains stable.

Second, both Python's `random` module and NumPy's generator receive the seed `1337`, synchronizing their outputs. Third, PyTorch's cuDNN backend (which normally chooses algorithms based on runtime heuristics) switches to deterministic mode, trading a small speed penalty for perfect reproducibility across CPU and GPU.

Data and Model Architecture

Why MNIST? Speed and clarity. Training a ResNet-50 on ImageNet takes hours and produces results hard to visualize. MNIST digits fit in a terminal, train in seconds, and still reveal vulnerabilities. We'll use a compact convolutional classifier that reaches 98% accuracy in one epoch, establishing a strong baseline while FGSM materially reduces robustness.

Device Configuration

PyTorch operations need a target device (CPU or GPU). Checking availability first ensures code runs everywhere, from laptops to cloud instances with CUDA:

```
# Configure computation device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

The conditional `torch.cuda.is_available()` returns `True` when NVIDIA drivers and CUDA toolkit are properly installed. GPU acceleration typically speeds training by 10-50x for convolutional networks, but our small MNIST model trains fast enough on CPU that device choice won't significantly affect the demonstration.

Data Loading

The library's data loader handles MNIST dataset preparation, conversion, and batching:

```
# Prepare data loaders using library function (normalized space)
train_loader, test_loader = get_mnist_loaders(batch_size=128,
normalize=True)
```

What does `get_mnist_loaders` do? It applies `transforms.ToTensor()` to convert each PIL image into a PyTorch tensor while rescaling from `[0, 255]` to `[0, 1]`. When `normalize=True` is specified, it applies MNIST-specific normalization that transforms pixels to a normalized space. The function downloads MNIST to `./data` if not already present. Crucially, it instantiates the training data loader's shuffling generator with seed `1337`, guaranteeing that batch 0 always contains the same 128 samples. The function sets `num_workers=0` to avoid multiprocessing race conditions and disables `pin_memory` for simplicity.

Model Definition

The architecture needs enough capacity to learn MNIST reliably but not so much that training becomes slow. Two convolutional layers extract spatial features, followed by two fully connected layers for classification:

```
# Initialize model using library's SimpleCNN
model = SimpleCNN().to(device)
```

What happens inside `SimpleCNN`? The first convolutional layer (`conv1`) processes the single-channel 28×28 input with 32 filters of size 3×3 . Setting `padding=1` maintains spatial dimensions at 28×28 after convolution. ReLU activation introduces nonlinearity. The second convolutional layer (`conv2`) applies 64 filters with the same kernel size, doubling the channel count to capture richer feature representations.

Two max-pooling operations (each with stride 2) reduce spatial resolution from 28×28 to 14×14 , then from 14×14 to 7×7 . This gives a final feature map of shape $64 \times 7 \times 7$. The `view` operation reshapes this into a flat 3136-dimensional vector (since $64 \times 7 \times 7 = 3136$).

Two fully connected layers process this vector, with the final layer outputting 10 raw logits corresponding to the digit classes 0-9.

The `.to(device)` call moves all model parameters to the configured device (GPU if available, CPU otherwise). This ensures consistency between where the model lives and where input data will be sent during training.

Training and Evaluation

Training and evaluation follow standard supervised learning patterns. The library encapsulates these in `train_model` and `evaluate_accuracy` to keep code focused on attacks rather than infrastructure:

```
# Train the model using library function
trained_model = train_model(model, train_loader, test_loader, epochs=1,
                             device=device)

# Evaluate baseline accuracy using library function
baseline_acc = evaluate_accuracy(trained_model, test_loader, device)
print(f"Baseline test accuracy: {baseline_acc:.2f}%")
```


The `train_model` function orchestrates the complete training loop. Each epoch begins with `model.train()`, enabling dropout (if present) and switching batch normalization to training mode where it updates running statistics. The optimizer (Adam with default learning rate 0.001) receives `set_to_none=True` during gradient clearing, which deallocates gradient tensors entirely rather than zeroing them in place, saving memory and time. Loss accumulates across batches weighted by batch size, producing an accurate epoch-level average. After all epochs finish, the function returns the updated model.

How does `evaluate_accuracy` differ from training? It switches the model to evaluation mode via `model.eval()`, which freezes batch normalization statistics and disables dropout. The `torch.no_grad()` context manager prevents gradient computation, reducing memory usage and accelerating inference. For each batch, `argmax(dim=1)` extracts the highest-scoring class index from the 10-dimensional logit vector. Accuracy is the fraction of correct predictions across the entire test set.

Does one epoch suffice? For MNIST, yes. The dataset is small (60,000 training images) and the patterns are simple (isolated digits on blank backgrounds). A single pass through the data typically converges to 98-99% accuracy. Training longer yields diminishing returns, maybe reaching 99.2%, but the added accuracy doesn't change the main point: even a well-trained model remains vulnerable to adversarial perturbations.

Expected output:

```
Epoch 1/1: Avg Loss = 0.1566, Test Accuracy = 98.41%  
Baseline test accuracy: 98.41%
```

The high accuracy (98.41%) on clean test data establishes a strong baseline for us to work with.

Normalization

Before proceeding with FGSM implementation, we need to understand what normalization does and why it matters for both training and adversarial attacks. Normalization transforms input data to have zero mean and unit variance, which fundamentally changes how we interpret perturbation budgets.

The Operation

Normalization applies a simple linear transformation to each pixel.

Given a dataset with mean

μ

and standard deviation

σ ,

the normalized value

x_{norm}

is computed from the original pixel value

x

as:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma}$$

The subtraction

$$x - \mu$$

centers the data around zero, removing the baseline brightness level.

The division by

$$\sigma$$

scales the data to unit variance, ensuring similar ranges across different datasets. For MNIST, the library uses

$$\mu = 0.1307$$

and

$$\sigma = 0.3081$$

which are computed from the entire training set of 60,000 images.

How are these statistics calculated? The mean

$$\mu = 0.1307$$

represents the average pixel intensity across all training images in

$$[0, 1]$$

range. Since MNIST digits are mostly black background (value 0) with white foreground (value 1), this low mean makes sense. The standard deviation

$$\sigma = 0.3081$$

measures how much pixel values vary around that mean. These specific values are standard for MNIST and should be used consistently across all experiments for reproducibility.

Why Normalize?

Here's the confusing part: normalized and unnormalized images look

identical to us. A digit 7 in pixel values

$$[0, 1]$$

looks exactly like the same digit after normalization to

$$[-0.42, 2.82]$$

Visually, nothing changed. So why bother?

The answer lies in the neural network's gradient mathematics, not human perception. This matters critically for adversarial attacks because FGSM relies entirely on computing gradients with respect to inputs. If those gradients behave poorly during training, they behave poorly during attacks too. Understanding how normalization affects gradient flow reveals why attacks work differently on normalized versus unnormalized models.

Training Without Normalization

Consider training a simple neural network on unnormalized MNIST pixels in $[0, 1][0, 1]$.

During backpropagation, the gradient magnitude depends on both the input values and the weight values. When inputs are all positive (pixel values between 0 and 1), gradients for weights in the first layer all point in similar directions because

$$\frac{\partial \text{loss}}{\partial w} = x \cdot \delta \quad \frac{\partial \text{loss}}{\partial w} = x \cdot \delta$$

where

$$x > 0, \delta > 0$$

always.

This creates three problems. First, the optimizer takes tiny, inefficient steps because all gradients point in correlated directions rather than exploring the full parameter space. A network that could converge in 10 epochs might take 30 epochs instead. Second, different layers receive vastly different gradient magnitudes. Early layers might get gradients around 0.001 while later layers get gradients around 10.0, forcing you to use layer-specific learning rates or adaptive optimizers. Third, using a single learning rate becomes a balancing act. Set it too high and late layers explode. Set it too low and early layers barely move.

What does this look like in practice? Training on unnormalized MNIST with a simple CNN and learning rate 0.001 produces slow convergence. At epoch 1, the loss starts at 2.3 with accuracy around 10% (random guessing). By epoch 5, loss drops to 1.8 with 35% accuracy, showing slow progress. At epoch 10, loss reaches 1.2 with 65% accuracy, still climbing slowly. Only at epoch 20 does the loss hit 0.6 with 85% accuracy. The model eventually learns, but painfully slowly, and gradients fluctuate wildly during training.

Training With Normalization

Now normalize those same images using

$$(x - 0.1307) / 0.3081$$

A raw MNIST pixel with value

$$0.8$$

(bright white on the digit) becomes

$$(0.8 - 0.1307) / 0.3081 \approx 2.17$$

A background pixel with value

$$0.0$$

(black) becomes

$$(0.0 - 0.1307) / 0.3081 \approx -0.42$$

The transformed values span roughly

$[-0.42, 2.82]$ $[-0.42, 2.82]$

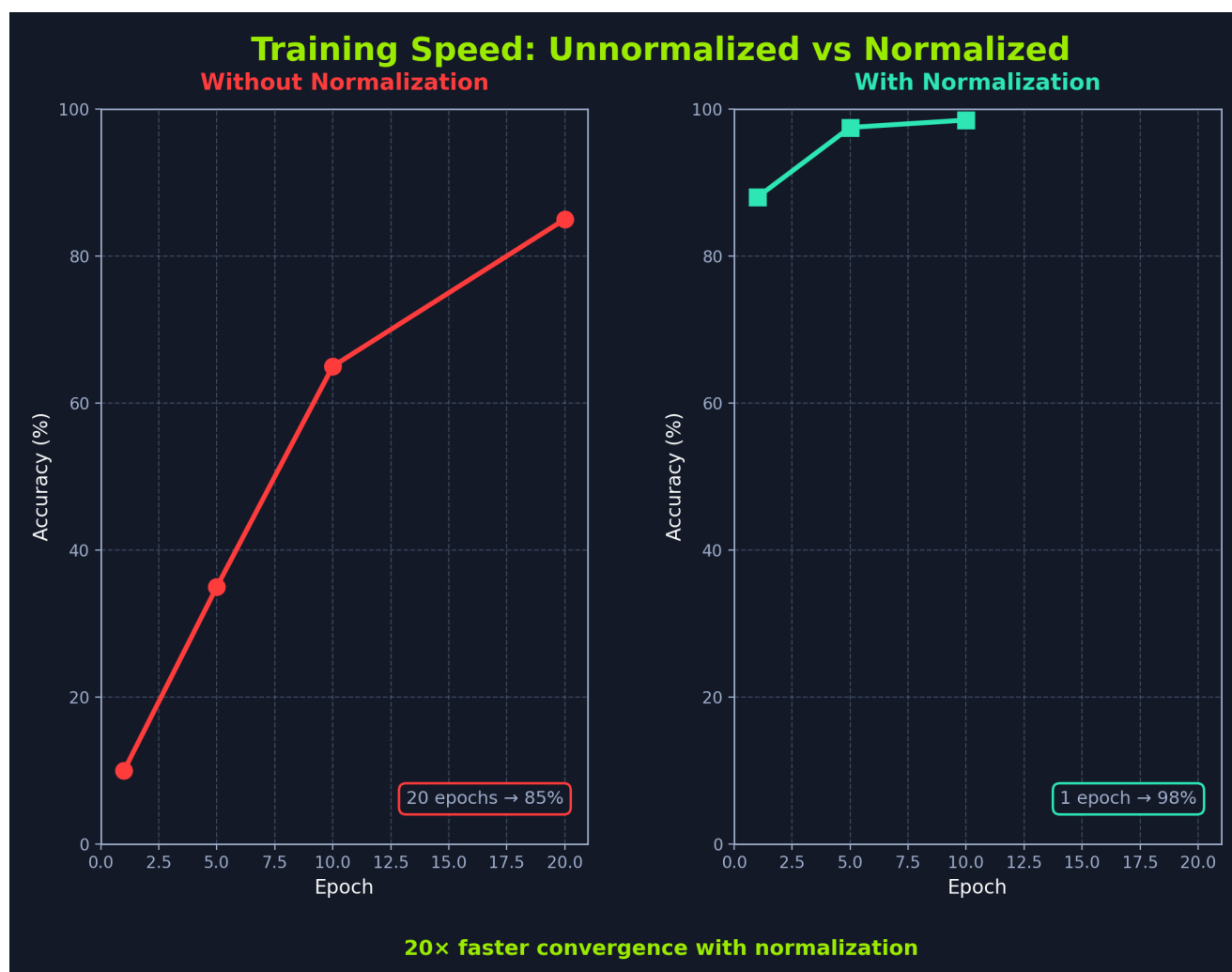
instead of

$[0, 1]$ $[0, 1]$,

with the new distribution centered near zero.

What changed mathematically? The inputs now have zero mean and unit variance. This means gradients in early layers receive both positive and negative signals, breaking the correlation. Weight updates explore the parameter space more efficiently. All layers receive gradient magnitudes in similar ranges because the input statistics are standardized. The optimizer can use a single learning rate across all layers.

The difference in training speed is substantial. Training the same CNN on normalized MNIST with the same learning rate 0.001 shows rapid convergence. At epoch 1, loss drops immediately to 0.4 with 88% accuracy, a large jump compared to the unnormalized baseline. By epoch 5, loss reaches 0.08 with 97.5% accuracy, showing rapid convergence. At epoch 10, loss hits 0.04 with 98.5% accuracy, strong performance. The model reaches 98% accuracy in one epoch versus 20 epochs without normalization. Training time drops from minutes to seconds. Gradient magnitudes stay stable throughout training instead of spiking and crashing.



Training Speed and Attack Vulnerability

This rapid convergence has a surprising consequence for adversarial robustness. The faster training creates sharper, more confident decision boundaries. Sharp boundaries mean small input perturbations can push samples across them more easily. An underfit model (like the unnormalized one at epoch 5 with 35% accuracy) has fuzzy, uncertain boundaries. Attacking it with FGSM often fails because the gradients point nowhere useful. The model hasn't learned crisp features yet.

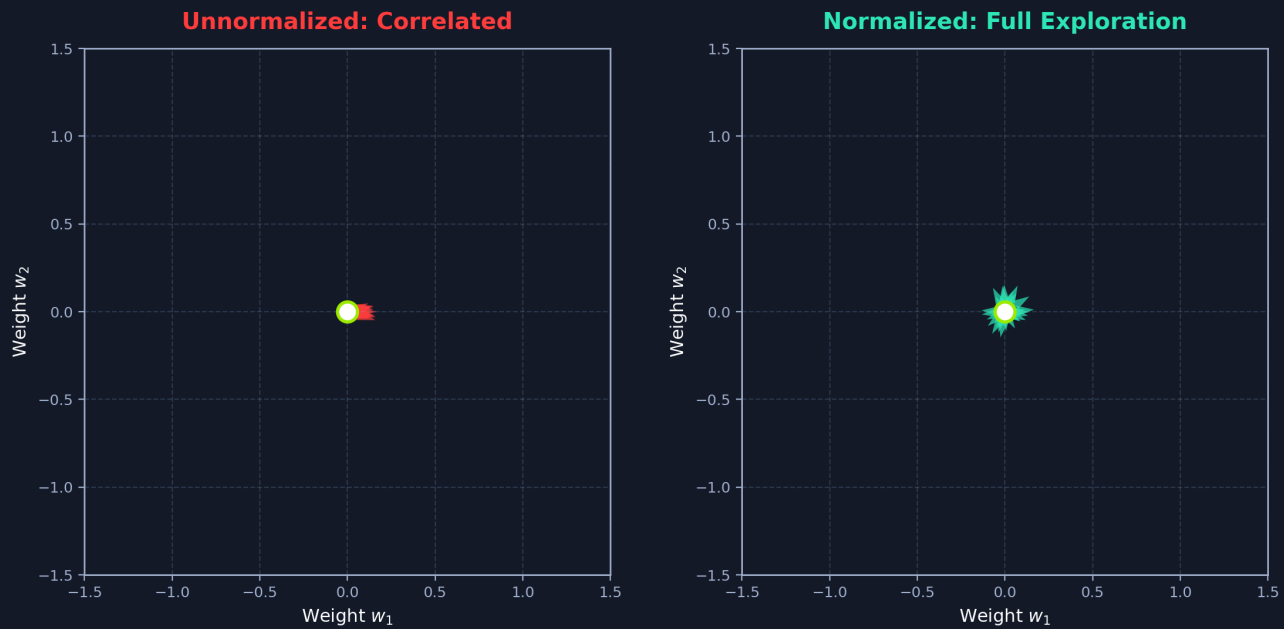
A well-trained normalized model has confident, sharp boundaries, making it simultaneously more accurate on clean data and more vulnerable to adversarial examples. The better the model, the easier it is to attack with gradient methods. This paradox drives modern adversarial robustness research: we want accurate models, but accuracy and vulnerability often go hand in hand.

The Underlying Mechanism

Activation functions like ReLU and optimization algorithms like SGD are designed assuming inputs are roughly zero-centered with similar scales. When you feed ReLU an input of 0.7, it outputs 0.7. When you feed it 2.17, it still outputs 2.17, but the `derivatives` behave better because the distribution of activations across the layer has mean closer to zero. This prevents the `dead ReLU problem` where neurons get stuck outputting zero because all inputs are positive and push weights into negative territory.

For optimization, zero-centered inputs mean weight gradients don't all point in the same direction. Instead of all gradients being positive (causing slow, correlated updates), you get a mix of positive and negative gradients that explore the loss surface more efficiently. The standardized variance means the optimizer doesn't need to guess whether a gradient of 0.01 is large or small because it's calibrated to the input distribution.

Gradient Behavior in Parameter Space



Normalized inputs enable efficient parameter space exploration

Gradient Quality and Attack Success

Consider two identical models, one trained on unnormalized $[0,1][0,1]$ inputs and one on normalized inputs. When computing input gradients for FGSM, the normalized model yields gradients that explore the full parameter space because training saw both positive and negative input-space values. The unnormalized model's gradients remain correlated in direction because all training inputs were positive, limiting how effectively FGSM can probe decision boundaries.

This translates to real attack differences. On a normalized model, $\epsilon=0.8$ in normalized space (about 0.25 in pixel space) might achieve 95% attack success. The same pixel-space perturbation on an unnormalized model achieves only 60% success because the gradient directions are less informative. The attack isn't inherently weaker, the model's gradient landscape is just less structured.

This improvement compounds with network depth. A 2-layer network might train reasonably well without normalization. A 10-layer network will likely fail to converge at all without normalized inputs, because gradient magnitudes either explode (growing exponentially through layers) or vanish (shrinking to zero). Normalization keeps gradient flow stable even through dozens of layers, which also means adversarial gradients remain informative throughout the network.

Range Transformation and Valid Bounds

When we normalize MNIST, the valid input range changes from

$[0, 1]$

to approximately

$[-0.424, 2.821]$.

These bounds come from applying the normalization formula to the original limits:

$$x_{\min} = 0.0 - 0.1307 \cdot 0.3081 \approx -0.424 \quad x_{\text{min}} = \frac{0.0 - 0.1307}{0.3081} \approx -0.424$$

$$x_{\max} = 1.0 - 0.1307 \cdot 0.3081 \approx 2.821 \quad x_{\text{max}} = \frac{1.0 - 0.1307}{0.3081} \approx 2.821$$

These constants appear throughout the attack implementations as

`MNIST_NORM_MIN` and `MNIST_NORM_MAX`. When we

clamp adversarial images to these bounds, we ensure they remain valid

MNIST inputs that can be converted back to

$[0, 1]$

pixel space for visualization without clipping artifacts.

Impact on Epsilon Budgets

Normalization fundamentally changes how we interpret

ϵ

budgets in FGSM attacks. When inputs are normalized,

ϵ

values are in normalized units unless explicitly stated. The

relationship between normalized-space and pixel-space budgets is:

$$\epsilon_{\text{pixel}} = \sigma \cdot \epsilon_{\text{norm}} \quad \epsilon_{\text{pixel}} = \sigma \cdot \epsilon_{\text{norm}}$$

For MNIST with

$$\sigma = 0.3081 \quad \sigma = 0.3081,$$

an

$$\epsilon_{\text{norm}} = 0.8 \quad \epsilon_{\text{norm}} = 0.8$$

in normalized space corresponds to

$$\epsilon_{\text{pixel}} = 0.8 \times 0.3081 \approx 0.25 \quad \epsilon_{\text{pixel}} = 0.8 \times 0.3081 \approx 0.25$$

in

$[0, 1]$

pixel space. This equals roughly

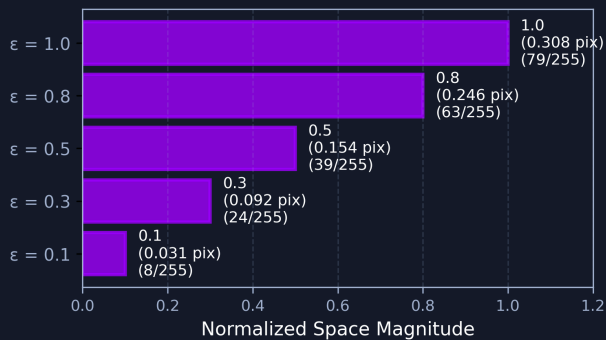
$$0.25 \times 255 \approx 64 \quad 0.25 \times 255 \approx 64$$

intensity levels at 8-bit precision.

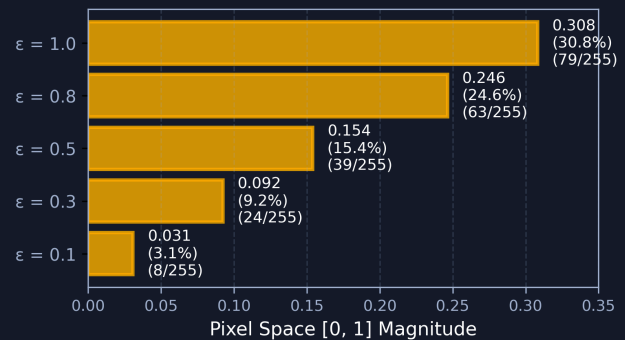
Epsilon Budget: Normalized vs Pixel Space Impact

$$\epsilon_{\text{pixel}} = \sigma \cdot \epsilon_{\text{norm}} = 0.3081 \cdot \epsilon_{\text{norm}}$$

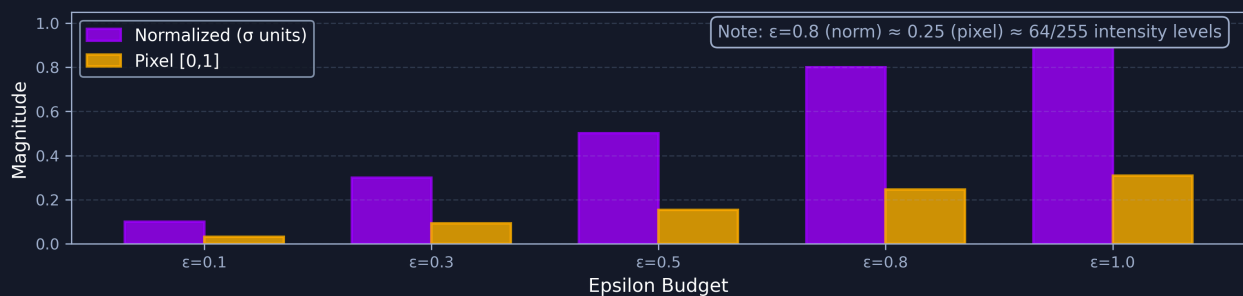
Epsilon in Normalized Space



Epsilon in Pixel Space



Direct Comparison: Same Budget, Different Scales



Why This Matters for FGSM

The FGSM update rule

$$x_{\text{adv}} = x + \epsilon \cdot \text{sign}(\nabla_x \mathcal{L})$$

relies on computing

$$\nabla_x \mathcal{L}$$

the gradient of loss with respect to the input. When the model was

trained on normalized inputs, those gradients are computed in normalized

space. The

ϵ (epsilon)

budget must be specified in the same space to have consistent meaning.

An

$$\epsilon = 0.3$$

in

$$[0, 1]$$

pixel space translates to roughly

$$\epsilon = 0.97$$

in MNIST's normalized space because

$$\epsilon_{\text{norm}} = \epsilon_{\text{pixel}} / \sigma = 0.3 / 0.3081 \approx 0.97$$

When we specify `epsilon=0.8` for an attack on normalized inputs, we're allowing each pixel to change by up to 0.8 standard deviations from its original value. This translates to different absolute changes depending on the pixel's location in the distribution. A change of 0.8 in normalized space represents about 25% of the full `[0,1][0,1]` range, which is visually subtle but mathematically significant for the model's decision boundary.

For adversarial attacks, this conversion matters when visualizing results or comparing across different normalization schemes. Attack code typically works in normalized space (where the model operates), but visualization converts back to `[0,1][0,1]` pixel space (where humans perceive images). An adversarial example with `epsilon=0.8` in normalized space looks subtly perturbed when denormalized for display, even though the numerical change is nearly 3 standard deviations.

Core Implementation

Before building the attack, we need a clean foundation. We assume you have the required libraries imported, a trained `SimpleCNN`, a `test_loader`, and a configured `device` from the setup section. The implementation uses PyTorch for tensor operations and automatic differentiation, along with the visualization tools and HTB styling constants already available from previous sections.

Computing Loss Without Side Effects

Separating forward computation and loss calculation into a small helper keeps gradient flow scoped to the input tensor and makes debugging simpler. This helper assumes the model is already in evaluation mode to avoid state updates from batch normalization or dropout. Keeping the function free of graph-modifying side effects lets us reuse it wherever a clean loss value is needed:

```
def _forward_and_loss(model: nn.Module, x: Tensor, y: Tensor) ->
tuple[Tensor, Tensor]:
    """Forward pass and cross-entropy loss without side effects.

    Args:
        model: Neural network classifier
        x: Input images tensor
        y: Target labels tensor
```

```

Returns:
    tuple[Tensor, Tensor]: Model logits and scalar loss value
"""
if getattr(model, "training", False):
    raise RuntimeError("Expected model.eval() for attack computations
to avoid BN/Dropout state updates")
logits = model(x)
loss = F.cross_entropy(logits, y)
return logits, loss

```

What do the shapes look like? For a batch of 128 MNIST images, `x` arrives with shape `[128, 1, 28, 28]` (batch size, channels, height, width). The model outputs `logits` of shape `[128, 10]`, one row per image with 10 class scores. Cross-entropy reduces these to a single scalar `loss` value. Returning both logits and loss gives callers flexibility: sometimes you need predictions, sometimes just the loss, sometimes both.

Gradient Computation

As established in the FGSM theory section, adversarial attacks require computing gradients with respect to inputs rather than parameters. Here's the PyTorch implementation:

```

def _input_gradient(model: nn.Module, x: Tensor, y: Tensor) -> Tensor:
    """Return gradient of loss with respect to input tensor x.

    Args:
        model: Neural network in evaluation mode
        x: Input images to compute gradients for
        y: True labels for loss computation

    Returns:
        Tensor: Gradient tensor with same shape as x
    """
    x_req = x.clone().detach().requires_grad_(True)
    _, loss = _forward_and_loss(model, x_req, y)
    model.zero_grad(set_to_none=True)
    loss.backward()
    return x_req.grad.detach()

```

The implementation details: `x.clone().detach()` creates a new tensor disconnected from the computational history. The `.requires_grad_(True)` enables gradient tracking on the

input. When `loss.backward()` executes, PyTorch computes $\nabla_x \mathcal{L}$ and stores it in `x_req.grad`. The final `.detach()` returns a standalone gradient tensor. Shape is preserved automatically: a `[128, 1, 28, 28]` input produces a `[128, 1, 28, 28]` gradient.

Core FGSM Attack Implementation

With gradients in hand, the attack becomes almost trivial. Extract the sign, scale by ϵ , add to the image, and clip. Four operations implement the entire attack:

```
def fgsm_attack(model: nn.Module,
                 images: Tensor,
                 labels: Tensor,
                 epsilon: float,
                 targeted: bool = False) -> Tensor:

    # Valid normalized range for MNIST
    MNIST_NORM_MIN = (0.0 - 0.1307) / 0.3081
    MNIST_NORM_MAX = (1.0 - 0.1307) / 0.3081

    if epsilon < 0:
        raise ValueError("epsilon must be non-negative")
    if not images.is_floating_point():
        raise ValueError("images must be floating point tensors")

    grad = _input_gradient(model, images, labels)
    step_dir = -1.0 if targeted else 1.0
    x_adv = images + step_dir * epsilon * grad.sign()
    x_adv = torch.clamp(x_adv, MNIST_NORM_MIN, MNIST_NORM_MAX)
    return x_adv.detach()
```

PyTorch's `.sign()` method operates element-wise on tensors, preserving shape and device placement. For a gradient tensor of shape `[128, 1, 28, 28]` on GPU, `grad.sign()` returns another `[128, 1, 28, 28]` tensor on the same GPU with identical memory layout. The operation is computationally cheap (single pass, no sorting or reduction) and memory-efficient (no

intermediate allocations beyond the output tensor). The multiplication

`epsilon * grad.sign()` broadcasts the scalar `epsilon` across all tensor elements, creating the perturbation

δ

where each component has magnitude exactly `epsilon` (or zero if the gradient was exactly zero, which rarely occurs with floating-point arithmetic).

The scalar `step_dir` controls attack direction through multiplication. Setting `step_dir=1.0` for untargeted attacks means the perturbation `step_dir * epsilon * grad.sign()` adds to the image, while `step_dir=-1.0` for targeted attacks subtracts. This avoids duplicating the gradient computation logic. The `torch.clamp()` operation clips adversarial images to the valid normalized range for MNIST (approximately -0.424 to 2.821), preventing grey washout when denormalized for visualization. This range corresponds to $[0, 1]$ in pixel space after applying the inverse normalization transform. Clamping happens element-wise, and the final `.detach()` extracts the adversarial images without gradient tracking.

Testing the Attack

Does FGSM actually work? Let's grab a batch, attack it, and count how many predictions flip. The test focuses on samples the model originally classified correctly, since flipping an already-wrong prediction tells us nothing:

```
images, labels = next(iter(test_loader))
images, labels = images.to(device), labels.to(device)

model.eval()
# Epsilon in normalized space (≈0.25 in pixel space)
epsilon = 0.8
with torch.no_grad():
    clean_pred = model(images).argmax(dim=1)

x_adv = fgsm_attack(model, images, labels, epsilon)
with torch.no_grad():
    adv_pred = model(x_adv).argmax(dim=1)

originally_correct = (clean_pred == labels)
flipped = (adv_pred != labels) & originally_correct
success = flipped.sum().item() / max(int(originally_correct.sum().item()),
1)
print(f"FGSM flips (first batch): {success:.2%}")
```

Expected output:

```
FGSM flips (first batch): 71.09%
```

How should we interpret a 71.09% flip rate? Out of 128 samples in the batch, if 128 were originally correct and 91 flipped, the success rate would be

$91/128 \approx 0.7109$ 91/128 \approx 0.7109.

This run uses

$\epsilon=0.8$ \epsilon=0.8

in normalized MNIST space. With mean

0.1307 0.1307

and std

0.3081 0.3081,

this maps to approximately

$0.8 \times 0.3081 \approx 0.25$ 0.8 \times 0.3081 \approx 0.25

in

$[0,1]$ [0,1]

pixel space, about

$0.25 \times 255 \approx 64$ 0.25 \times 255 \approx 64

intensity levels at 8-bit precision. The exact flip rate varies with the trained weights and the specific batch.

Pixel-Space FGSM Variant

The core FGSM implementation works entirely in normalized space: it expects images already normalized, uses epsilon in normalized units, and returns adversarials in normalized space. This matches our setup where `get_mnist_loaders(normalize=True)` provides pre-normalized batches. However, some workflows start with raw pixel-space images in $[0,1]$ and need epsilon specified as pixel-space perturbations (like "8/255 intensity levels").

For these pixel-space workflows attacking normalized models, we need a variant that accepts $[0,1]$ images, interprets

ϵ \epsilon

as a pixel-space budget, but still normalizes internally for the model

and converts gradients correctly. The key is dividing normalized-space gradients by

σ \sigma

to map them back to pixel space before applying the sign step with the pixel-space

ϵ \epsilon.

Why the conversion? When gradients are computed with respect to normalized inputs

$x_{\text{norm}} = (x - \mu) / \sigma$,

those gradients live in normalized space. To apply a pixel-space epsilon budget, we must convert gradients back using the chain rule:

$\nabla_x \mathcal{L} = \nabla_{x_{\text{norm}}} \mathcal{L} / \sigma$.

This ensures $\epsilon = 0.031$ (approximately $8/255$) means

exactly 8 intensity levels in original $[0, 1]$ space, not in

the stretched normalized space where magnitudes differ by a factor of σ .

Creating Normalization Parameters

Before implementing the pixel-space FGSM variant, we need a helper function to prepare normalization parameters as broadcastable tensors matching image batch dimensions. The normalization transform requires mean and standard deviation values, but we receive them as Python lists. Converting them to properly shaped tensors ensures they broadcast correctly across batch, height, and width dimensions when normalizing images:

```
def _norm_params(images: Tensor, mean: list, std: list) -> tuple[Tensor,
Tensor]:
    """Convert normalization parameters to broadcastable tensors.

    Args:
        images: Input images tensor with shape (N, C, H, W)
        mean: Normalization mean per channel as list
        std: Normalization std per channel as list

    Returns:
        tuple[Tensor, Tensor]: Mean and std tensors with shape (1, C, 1,
1)
    """
    device, dtype, C = images.device, images.dtype, images.shape[1]
    mean_t = torch.tensor(mean, device=device, dtype=dtype).view(1, -1, 1,
1)
    std_t = torch.tensor(std, device=device, dtype=dtype).view(1, -1, 1,
1)
    if mean_t.shape[1] != C or std_t.shape[1] != C:
        raise ValueError("mean/std channels must match images")
    return mean_t, std_t
```

The `.view(1, -1, 1, 1)` reshapes the mean/std vectors for broadcasting. For RGB images with `mean=[0.485, 0.456, 0.406]`, the tensor shape becomes `[1, 3, 1, 1]`, broadcasting across batch, height, and width dimensions.

Pixel-Space FGSM Implementation

How do we bridge pixel-space inputs with normalized models while keeping epsilon interpretable? The implementation needs to handle a tricky dance: accept `[0,1]` images, transform them to normalized space for the model's gradient computation, then transform those gradients back to pixel space so our pixel-space epsilon has correct meaning. The key challenge is the gradient conversion step, where we must divide by σ to undo the scaling that normalization introduced. Without this step, applying a pixel-space epsilon to normalized-space gradients produces incorrect perturbation magnitudes.

```
def fgsm_pixel_space(model: nn.Module,
                    images: Tensor,
                    labels: Tensor,
                    epsilon: float,
                    mean: list,
                    std: list,
                    targeted: bool = False) -> Tensor:
    """FGSM for pixel-space inputs attacking normalized models.

    This variant accepts images in [0,1] pixel space rather than
    normalized
    space. It normalizes inputs internally for the model, converts
    gradients
    back to pixel space, and returns adversarials in [0,1] pixel space.

    Args:
        model: Model expecting normalized inputs
        images: Clean images in [0,1] pixel space (unnormalized)
        labels: Target labels
        epsilon: Max perturbation in pixel space (e.g., 8/255)
        mean: Normalization mean per channel
        std: Normalization std per channel
        targeted: If True, minimize loss towards labels

    Returns:
        Tensor: Adversarial images in [0,1] pixel space (unnormalized)
```

```

"""
mean_t, std_t = _norm_params(images, mean, std)
x = images.clone().detach()
x_norm = (x - mean_t) / std_t
x_norm.requires_grad_(True)

_, loss = _forward_and_loss(model, x_norm, labels)
model.zero_grad(set_to_none=True)
loss.backward()

# Convert gradient from normalized space to image space
grad_img = x_norm.grad / std_t
step_dir = -1.0 if targeted else 1.0
x_adv = torch.clamp(x + step_dir * epsilon * grad_img.sign(), 0.0,
1.0)
return x_adv.detach()

```

Why divide by `std_t`? The normalization

$(x-\mu)/\sigma$ ($(x - \mu)/\sigma$)

scales pixel values. Gradients computed with respect to the normalized input

x_{norm}

live in that scaled space. To interpret `epsilon` as a bound in the original $[0,1]$ pixel space, we must undo the scaling. If `std=[0.229]` and the normalized-space gradient is 2.5, the image-space gradient becomes $2.5/0.229 \approx 10.925 / 0.229 \approx 10.9$.

The sign step then uses this rescaled gradient, ensuring

$\epsilon=8/255 \approx 0.031$ ($\epsilon=8/255 \approx 0.031$)

means exactly 8 intensity levels in the original image, not 8 levels in the normalized space where magnitudes differ.

When to Use This Variant

This pixel-space variant is useful when working with pipelines that provide raw unnormalized images but you need to attack a model expecting normalized inputs. For example, some datasets or external APIs return images in $[0,1]$ without normalization applied.

However, if your data is already normalized (as with our

`get_mnist_loaders(normalize=True)` setup), you should use the core `fgsm_attack` directly, which operates entirely in normalized space. Converting normalized $\rightarrow$ pixel $\rightarrow$ attack $\rightarrow$ normalized is unnecessarily complex.

For workflows requiring this variant: if images arrive in $[0,1]$, specify ϵ in pixel units (like $8/255 \approx 0.0318/255 \approx 0.031$), and the function handles the internal normalization and gradient conversion automatically:

```
# Example: Starting with pixel-space images
epsilon_px = 8 / 255 # pixel-space epsilon ( $\approx 0.031$ )
mean, std = [0.1307], [0.3081]

# Denormalize existing normalized images to get pixel-space images
mean_t, std_t = _norm_params(images, mean, std)
pixel_images = images * std_t + mean_t
pixel_images = torch.clamp(pixel_images, 0.0, 1.0)

# Attack in pixel space
x_adv_pixel = fgsm_pixel_space(model, pixel_images, labels, epsilon_px,
                                mean, std)

# x_adv_pixel is in  $[0,1]$  and can be displayed or saved directly
# If you need to pass to the model again, normalize it first:
x_adv_norm = (x_adv_pixel - mean_t) / std_t
```

The function ensures ϵ retains its pixel-space interpretation throughout the attack by converting gradients via division by σ before applying the sign step. This keeps perturbation budgets meaningful when comparing across different normalization schemes.

Evaluation Metrics

With the core FGSM attack implemented, we now build metrics to evaluate its effectiveness. Flip rate alone doesn't tell the full story. What happens to model confidence? How much did perturbations actually change the image in L_2 terms? This section implements an evaluation function that tracks

accuracy, success rate, confidence drop, and perturbation norms, giving a complete picture of attack impact.

What Metrics Do We Need?

Evaluating an adversarial attack requires looking beyond whether predictions flip. We need to understand the full impact on model behavior. Accuracy metrics tell us how many samples the model classifies correctly before and after perturbation, establishing the baseline and measuring degradation. Success rate specifically measures what percentage of originally correct predictions the attack manages to flip, isolating attack effectiveness from model quality. Confidence metrics reveal how certain the model is in its predictions, showing whether the attack creates genuine confusion or just barely pushes samples across decision boundaries. Perturbation norms quantify the distortion introduced, measuring both average

L_2

distance per sample and maximum

L_∞

change to verify we respect the epsilon budget.

Building the Evaluation Function

First, we need the model's predictions and confidence scores for both clean and adversarial inputs. Running both batches through the model yields logits, which we convert to probabilities using softmax and extract predicted classes using argmax:

```
from typing import Dict

def evaluate_attack(model: nn.Module,
                   clean_images: Tensor,
                   adversarial_images: Tensor,
                   true_labels: Tensor) -> Dict[str, float]:
    """Compute accuracy, success rate, confidence shift, and norms.

    Args:
        model: Evaluated classifier in evaluation mode
        clean_images: Clean inputs in the model's expected domain (e.g.,
normalized MNIST)
        adversarial_images: Adversarial counterparts in the same domain as
`clean_images`
        true_labels: Ground-truth labels

    Returns:
```

```

Dict[str, float]: Aggregated metrics summarizing attack impact
"""
model.eval()
with torch.no_grad():
    clean_logits = model(clean_images)
    adv_logits = model(adversarial_images)

    clean_probs = F.softmax(clean_logits, dim=1)
    adv_probs = F.softmax(adv_logits, dim=1)

    clean_pred = clean_logits.argmax(dim=1)
    adv_pred = adv_logits.argmax(dim=1)

```

The `torch.no_grad()` context disables gradient tracking since we're only evaluating, not training. For a batch of 128 images with 10 classes, `clean_logits` has shape `[128, 10]`, `clean_probs` has the same shape with rows summing to 1.0, and `clean_pred` has shape `[128]` containing class indices.

Measuring Accuracy and Success Rate

Next, we compute correctness masks and derive the attack success rate. Comparing predictions to true labels yields boolean tensors indicating which samples are classified correctly. The attack success rate focuses specifically on samples that were originally correct but became incorrect after perturbation:

```

clean_correct = (clean_pred == true_labels)
adv_correct = (adv_pred == true_labels)

originally_correct = clean_correct
flipped = (~adv_correct) & originally_correct

```

The `flipped` mask uses logical operations to identify samples where `clean_correct` is `True` but `adv_correct` is `False`. If 125 out of 128 samples were originally correct and 80 flipped, the success rate is $80/125 = 0.64$.

Extracting Confidence Metrics

Confidence metrics reveal how certain the model is in its predictions. We extract the probability the model assigns to the true class for each sample, both before and after perturbation. The

gather operation selects specific probability values based on the true label indices:

```
conf_clean = clean_probs.gather(1, true_labels.view(-1, 1)).squeeze(1)
conf_adv = adv_probs.gather(1, true_labels.view(-1, 1)).squeeze(1)
```

How does gather work? If `true_labels=[3, 7]` and `clean_probs` has shape `[2, 10]`, then `true_labels.view(-1, 1)` reshapes to `[[3], [7]]`. The `gather(1, ...)` operation selects column 3 from row 0 and column 7 from row 1, extracting the probabilities for the true classes. The `squeeze(1)` removes the extra dimension, yielding a 1D tensor. A confidence drop from 0.92 to 0.31 indicates the attack eroded the model's certainty by 61 percentage points, even if the prediction didn't flip.

Computing Perturbation Norms

Finally, we measure the magnitude of perturbations introduced by the attack. The

L_2

norm quantifies the Euclidean distance between clean and adversarial images (measuring overall distortion), while the

L_∞

norm captures the maximum change to any single pixel (verifying we respect the epsilon budget):

```
l2 = (adversarial_images -
clean_images).view(clean_images.size(0), -1).norm(p=2, dim=1)
linf = (adversarial_images - clean_images).abs().amax()

return {
    "clean_accuracy": clean_correct.float().mean().item(),
    "adversarial_accuracy": adv_correct.float().mean().item(),
    # Success rate among originally correct samples only
    "attack_success_rate": (
        flipped.float().sum() /
originally_correct.float().sum().clamp_min(1.0)
    ).item(),
    "avg_clean_confidence": conf_clean.mean().item(),
    "avg_adv_confidence": conf_adv.mean().item(),
    "avg_confidence_drop": (conf_clean - conf_adv).mean().item(),
    "avg_l2_perturbation": l2.mean().item(),
    "max_linf_perturbation": linf.item(),
}
```

```
}
```

The `.view(clean_images.size(0), -1)` flattens each image to a 1D vector while preserving the batch dimension, enabling per-sample

L_2

norm computation. For MNIST images with shape

`[128, 1, 28, 28]`, this becomes `[128, 784]`. The

`norm(p=2, dim=1)` computes the

L_2

norm along dimension 1, yielding a tensor of shape `[128]`

with one norm value per image. The

L_∞

norm uses `.amax()` without specifying dimensions, returning the single largest absolute perturbation across the entire batch.

Applying the Metrics

Applying these metrics to the batch from Core Implementation:

```
# Assume images, labels, x_adv from the Core Implementation section
metrics = evaluate_attack(model, images, x_adv, labels)
for k, v in metrics.items():
    print(f"{k}: {v:.4f}")
```

Expected output:

```
clean_accuracy: 0.9766
adversarial_accuracy: 0.3203
attack_success_rate: 0.6797
avg_clean_confidence: 0.9824
avg_adv_confidence: 0.3891
avg_confidence_drop: 0.5933
avg_l2_perturbation: 10.8451
max_linf_perturbation: 0.8000
```

The metrics show that FGSM with `epsilon=0.8` (in normalized space, approximately `epsilon=0.25` in pixel space) successfully flips 67.97% of originally correct predictions, dropping average confidence from 98.24% to 38.91%. The `max_linf_perturbation` confirms the attack respects the

epsilon bound. The

L2L₂

perturbation of approximately 10.8 reflects the normalized space

magnitude, which corresponds to subtle visual changes when denormalized for display.

Visualization

Visual analysis reveals how adversarial perturbations affect model predictions beyond numerical metrics. This section builds visualization infrastructure to display clean images, adversarial examples, perturbations, and probability shifts side by side. The visualizations help understand attack mechanisms and validate implementation correctness.

Visualization Setup

To maintain consistent visual styling across all plots, we need a helper function that applies the theme to matplotlib axes:

```
import matplotlib.pyplot as plt
import numpy as np

# Colors imported from library
from htb_ai_library import (
    HTB_GREEN, NODE_BLACK, HACKER_GREY, WHITE,
    AZURE, NUGGET_YELLOW, MALWARE_RED, VIVID_PURPLE, AQUAMARINE
)

def _style_axes(ax: plt.Axes) -> None:
    """Apply Hack The Box dark theme to an axes instance.

    Args:
        ax: Matplotlib axes to style
    """
    ax.set_facecolor(NODE_BLACK)
    ax.tick_params(colors=HACKER_GREY)
    for spine in ax.spines.values():
        spine.set_color(HACKER_GREY)
    ax.grid(True, color=HACKER_GREY, linestyle="--", alpha=0.25)
```

The function sets background color, tick colors, spine colors, and adds a subtle grid. The `alpha=0.25` creates a faint grid that doesn't dominate the visualization.

Main Visualization Function

The visualization displays original, adversarial, and perturbation images alongside class probabilities. Let's build this step by step.

First, the function prepares the attack and computes predictions:

```
def visualize_attack(model: nn.Module,
                    image: Tensor,
                    label: Tensor,
                    make_adv,
                    title: str,
                    num_classes: int = 10,
                    targeted: bool = False,
                    target_class: int | None = None) -> None:
    """HTB-styled visualization for adversarial examples.

    Args:
        model: Classifier in evaluation mode
        image: Single image in normalized space, shape (C,H,W)
        label: Scalar true label tensor
        make_adv: Callable (model, image_batch, label_batch) -> adv_batch
in normalized space
        title: Figure title
        num_classes: Number of classes to show in probability bars
        targeted: Whether the attack is targeted
        target_class: Optional target class to annotate
    """
    model.eval()
    dev = next(model.parameters()).device
    image_dev = image.to(dev)
    label_dev = label.to(dev)

    # Compute clean predictions
    with torch.no_grad():
        clean_probs = F.softmax(model(image_dev.unsqueeze(0)),
dim=1).squeeze(0)
        clean_pred = int(clean_probs.argmax().item())

    # Generate adversarial example
    x_adv_dev = make_adv(model, image_dev.unsqueeze(0),
label_dev.unsqueeze(0)).squeeze(0)
    perturbation_dev = x_adv_dev - image_dev

    # Compute adversarial predictions
```

```

with torch.no_grad():
    adv_probs = F.softmax(model(x_adv_dev.unsqueeze(0)),
dim=1).squeeze(0)
    adv_pred = int(adv_probs.argmax().item())

# Denormalize for visualization
image_vis =
mnist_denormalize(image_dev.unsqueeze(0)).squeeze(0).detach().cpu()
x_adv_vis =
mnist_denormalize(x_adv_dev.unsqueeze(0)).squeeze(0).detach().cpu()
perturbation_vis = (x_adv_vis - image_vis)

```

The `unsqueeze(0)` adds a batch dimension since models expect batched inputs. If `clean_probs[3] = 0.95`, the model is 95% confident in class 3. The denormalization step is necessary: `mnist_denormalize` converts normalized-space images back to `[0,1]` pixel space for visualization. Without this step, normalized values like -0.3 or 2.5 would be incorrectly displayed, producing unrecognizable images. The perturbation is computed in pixel space after denormalization to show the actual visual change.

Next, the figure layout is created with three image panels and one probability bar chart:

```

# Create figure with grid layout
fig = plt.figure(figsize=(16, 10), facecolor=NODE_BLACK)
gs = fig.add_gridspec(2, 3, hspace=0.35, wspace=0.35)

```

The original image panel shows the clean input with its prediction:

```

# Original image panel
ax1 = fig.add_subplot(gs[0, 0])
_style_axes(ax1)
if image_vis.shape[0] == 1:
    ax1.imshow(image_vis.squeeze(0), cmap='gray', vmin=0, vmax=1)
else:
    ax1.imshow(image_vis.permute(1, 2, 0))
ax1.set_title(f"Original | class={clean_pred} | p=
{clean_probs[clean_pred]:.2%}",
              color=HTB_GREEN, fontweight="bold")
ax1.set_xticks([])
ax1.set_yticks([])

```


The adversarial image panel uses red title if misclassified:

```
# Adversarial image panel
ax2 = fig.add_subplot(gs[0, 1])
_style_axes(ax2)
if x_adv_vis.shape[0] == 1:
    ax2.imshow(x_adv_vis.squeeze(0), cmap='gray', vmin=0, vmax=1)
else:
    ax2.imshow(x_adv_vis.permute(1, 2, 0))
    title_color = MALWARE_RED if adv_pred != int(label.item()) else
    HTB_GREEN
    adv_title = f"Adversarial | class={adv_pred} | p=
    {adv_probs[adv_pred]:.2%}"
    if targeted and target_class is not None:
        adv_title += f" | target={target_class}"
    ax2.set_title(adv_title, color=title_color, fontweight="bold")
    ax2.set_xticks([])
    ax2.set_yticks([])
```

The perturbation panel scales differences for visibility:

```
# Perturbation panel (scaled for visibility)
ax3 = fig.add_subplot(gs[0, 2])
_style_axes(ax3)
pert_scaled = (perturbation_vis * 10 + 0.5).clamp(0, 1)
if pert_scaled.shape[0] == 1:
    ax3.imshow(pert_scaled.squeeze(0), cmap='gray', vmin=0, vmax=1)
else:
    ax3.imshow(pert_scaled.permute(1, 2, 0))
    ax3.set_title("Perturbation (x10)", color=NUGGET_YELLOW,
    fontweight="bold")
    ax3.set_xticks([])
    ax3.set_yticks([])
```

The scaling transform `(perturbation_vis * 10 + 0.5)` serves a specific purpose for visualizing signed perturbations. Perturbations can be both positive (brightening pixels) and negative (darkening pixels), typically in a small range like `[-0.05, +0.05]`. Without scaling, these tiny values would appear as uniform gray when displayed. The multiplication by `10` amplifies the perturbations to `[-0.5, +0.5]`, making them visible. The addition of `0.5` centers this range to `[0.0, 1.0]`, where `0.5` represents no change (medium gray), values above `0.5` show positive perturbations (lighter), and values below `0.5` show negative

perturbations (darker). For example, a perturbation of $+0.03$ becomes $0.03 * 10 + 0.5 = 0.8$ (light gray), while -0.03 becomes $-0.03 * 10 + 0.5 = 0.2$ (dark gray). The final `clamp(0, 1)` ensures extreme values stay within the valid display range. The scaling factor 10 is chosen empirically to make typical FGSM perturbations visible without saturating the display.

Finally, the probability comparison bar chart:

```
# Class probability comparison
ax4 = fig.add_subplot(gs[1, :])
_style_axes(ax4)
x = np.arange(num_classes)
width = 0.4
ax4.bar(x - width/2, clean_probs[:num_classes].cpu(), width,
        color=AZURE, label="clean")
ax4.bar(x + width/2, adv_probs[:num_classes].cpu(), width,
        color=MALWARE_RED, label="adv")
ax4.set_xlabel("Class", color=WHITE)
ax4.set_ylabel("Probability", color=WHITE)
legend = ax4.legend(facecolor=NODE_BLACK, edgecolor=HACKER_GREY)
for text in legend.get_texts():
    text.set_color(WHITE)
ax4.set_title("Class probabilities", color=HTB_GREEN,
fontweight="bold")
for text in ax4.get_xticklabels() + ax4.get_yticklabels():
    text.set_color(HACKER_GREY)

# Add main title and display
fig.suptitle(title, color=HTB_GREEN, fontweight="bold", fontsize=24,
y=0.98)
fig.tight_layout(rect=(0, 0, 1, 0.93))
plt.show()
```

FGSM-Specific Wrapper

A thin wrapper simplifies FGSM visualization:

```
def visualize_fgsm_attack(model: nn.Module,
                        image: Tensor,
                        label: Tensor,
                        epsilon: float,
                        num_classes: int = 10,
```

```

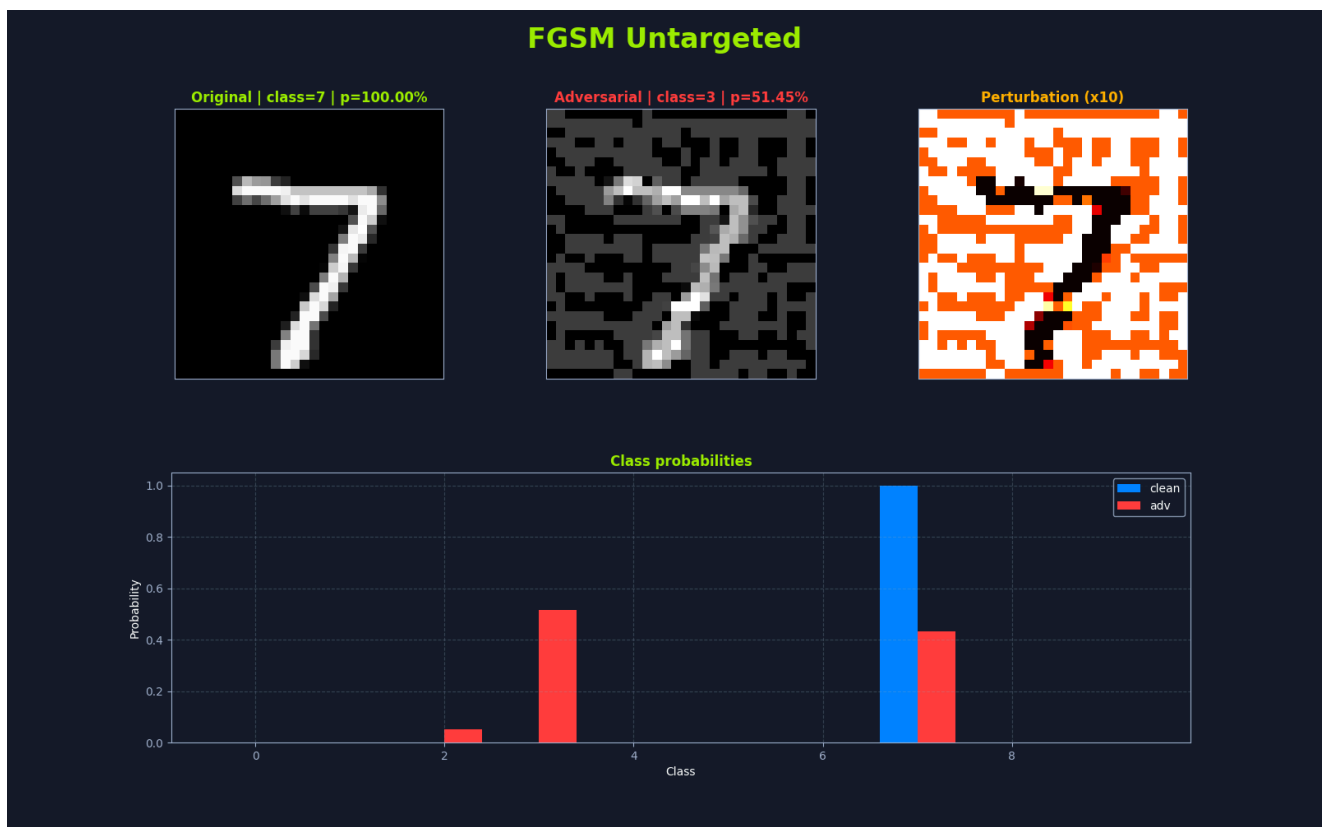
        targeted: bool = False,
        target_class: int | None = None) -> None:
    """Wrapper for visualize_attack using FGSM.

    Args:
        model: Classifier model
        image: Single image tensor
        label: True label
        epsilon: Perturbation budget
        num_classes: Classes to display
        targeted: If True, targeted attack
        target_class: Target class for targeted attack
    """
    def _make_adv(m, xb, yb):
        if targeted and target_class is None:
            raise ValueError("target_class must be provided when
targeted=True")
        y_used = yb if not targeted else torch.full_like(yb, target_class)
        return fgsm_attack(m, xb, y_used, epsilon, targeted=targeted)

    mode = "Targeted" if targeted else "Untargeted"
    visualize_attack(model, image, label, _make_adv,
                    title=f"FGSM {mode}",
                    num_classes=num_classes,
                    targeted=targeted,
                    target_class=target_class)

```

The panels show us how a small L_∞ -bounded change materially changes predicted probabilities. The perturbation view is scaled to enhance visibility and does not reflect true magnitude.



Rendering a complete view for a single sample:

```
# Assume images, labels from test_loader (from Setup)
# Assume epsilon from Core Implementation (epsilon=0.8)
_ = visualize_fgsm_attack(model, images[0].detach().cpu(),
                          labels[0].detach().cpu(), epsilon)
```

Targeted FGSM

Targeted FGSM changes the objective from reducing confidence in the true class to increasing confidence in a specific target class. With cross-entropy, untargeted FGSM ascends

$$\mathcal{L}(\theta, x, y)$$

targeted FGSM descends

$$\mathcal{L}(\theta, x, y_t)$$

equivalent to ascending

$$-\mathcal{L}(\theta, x, y_t)$$

The implementation has two precise differences from untargeted.

First, the gradient uses target label

y_t

instead of true label. Second, the update sign flips so the step reduces

loss for

y_t

rather than increasing loss for

yy.

The

L_{∞}

budget and clipping remain unchanged.

Because targeted FGSM must steer predictions toward a particular class (not just away from the current one), it often requires slightly larger

ϵ epsilon

or benefits from early stopping once the model predicts the target.

Targeted Attack Example

The implementation forces a

$1 \rightarrow 7$

flip by supplying the target label in the loss and reversing the update direction. We need to find a digit 1 in the test set, then search for the minimal epsilon that successfully forces the model to predict 7. The search uses progressively larger epsilon values and terminates early once any candidate succeeds:

Setup and Configuration

To find the minimal perturbation budget, we'll test epsilon values in ascending order and track the first successful configuration:

```
eps_candidates = [0.5, 0.8, 1.0]
success_image, success_label, success_eps = None, None, None
```

The `eps_candidates` list specifies epsilon values in normalized space, starting from a conservative 0.5 and increasing to 1.0. The three `None` values track results:

`success_image` stores the original digit 1, `success_label` holds its true label, and `success_eps` records which epsilon value succeeded. Starting all at `None` lets us detect failure by checking if they remain unset after the search completes.

Finding a Candidate Digit

We need to locate a digit 1 from the test set that the model correctly classifies, ensuring we have a valid baseline for the targeted attack:

```
model.eval()
candidate, candidate_label = None, None
```

```

for xb, yb in test_loader:
    xb, yb = xb.to(device), yb.to(device)
    match_indices = (yb == 1).nonzero(as_tuple=True)[0]
    if len(match_indices) == 0:
        continue

    # Check predictions for all digit 1s in this batch
    with torch.no_grad():
        preds = model(xb[match_indices]).argmax(dim=1)
        correct_mask = (preds == 1)
        if correct_mask.any():
            # Take first correctly classified digit 1
            local_idx = correct_mask.nonzero(as_tuple=True)[0][0].item()
            idx = match_indices[local_idx].item()
            candidate = xb[idx]
            candidate_label = yb[idx]
            break

if candidate is None:
    raise RuntimeError("Could not find a correctly classified digit 1 in
test set")

```

The loop iterates over batches from `test_loader`, yielding tensors `xb` and `yb` with shapes `[128, 1, 28, 28]` and `[128]` respectively. The expression `(yb == 1)` creates a boolean tensor where `True` marks positions containing label 1. If a batch has labels `[3, 1, 7, 1, 2]`, the comparison produces `[False, True, False, True, False]`. The `nonzero(as_tuple=True)[0]` operation extracts the indices of `True` values as a 1D tensor, yielding `[1, 3]` for this example.

Before selecting a candidate, we verify the model actually predicts it as class 1 by running inference on all digit 1s in the batch with `model(xb[match_indices])`. The `correct_mask = (preds == 1)` identifies which ones are correctly classified. We only proceed if at least one is correct, using `correct_mask.any()`. The double indexing `match_indices[local_idx]` maps from the filtered subset back to the original batch position. Setting `model.eval()` before the loop ensures batch normalization and dropout behave correctly during inference. The final check `if candidate is None` catches the edge case where no correctly classified digit 1 exists in the entire test set.

Testing Epsilon Values

Now we run targeted FGSM with each epsilon candidate and check if the attack forces prediction to class 7:

```

target_label = torch.tensor([7], device=device)

for eps_try in eps_candidates:
    x_adv = fgsm_attack(
        model,
        candidate.unsqueeze(0),
        target_label,
        epsilon=eps_try,
        targeted=True,
    )
    with torch.no_grad():
        pred = model(x_adv).argmax(dim=1).item()
    print(f"epsilon={eps_try:.2f} -> predicted {pred}")

    if pred == 7:
        success_image = candidate
        success_label = candidate_label
        success_eps = eps_try
        break

```

The loop tests epsilon values in ascending order. The `fgsm_attack` call requires careful tensor shaping. The `candidate` has shape `[1, 28, 28]` after indexing from the batch, so `unsqueeze(0)` adds a batch dimension to produce `[1, 1, 28, 28]`, matching the model's expected input format. The `target_label` is created once before the loop as `torch.tensor([7], device=device)`, producing a tensor of shape `[1]` already on the correct device. The `targeted=True` flag tells `fgsm_attack` to reverse the gradient direction, minimizing loss toward class 7 instead of maximizing loss for the true class.

After generating the adversarial, we check the model's prediction with `argmax(dim=1).item()`. If `pred == 7`, the attack succeeded at this epsilon, so we capture the configuration and break immediately. This early termination avoids testing larger epsilon values once we find the minimal budget that works.

Handling Failure and Visualization

If no epsilon succeeded, we raise an error. Otherwise, visualize the successful attack:

```

if success_image is None:
    raise RuntimeError("Targeted FGSM did not achieve 1 -> 7 within the tested epsilons.")

_ = visualize_fgsm_attack(

```

```

model,
success_image.detach().cpu(),
success_label.detach().cpu(),
success_eps,
targeted=True,
target_class=7,
)

```

The `success_image` is `None` check detects complete failure across all epsilon candidates. If all tested values failed to force the target prediction, the error message helps debug by indicating that larger epsilon values or different hyperparameters may be needed. On success, we move tensors back to CPU with `.detach().cpu()` before passing to the visualization function, which expects CPU tensors for matplotlib rendering.

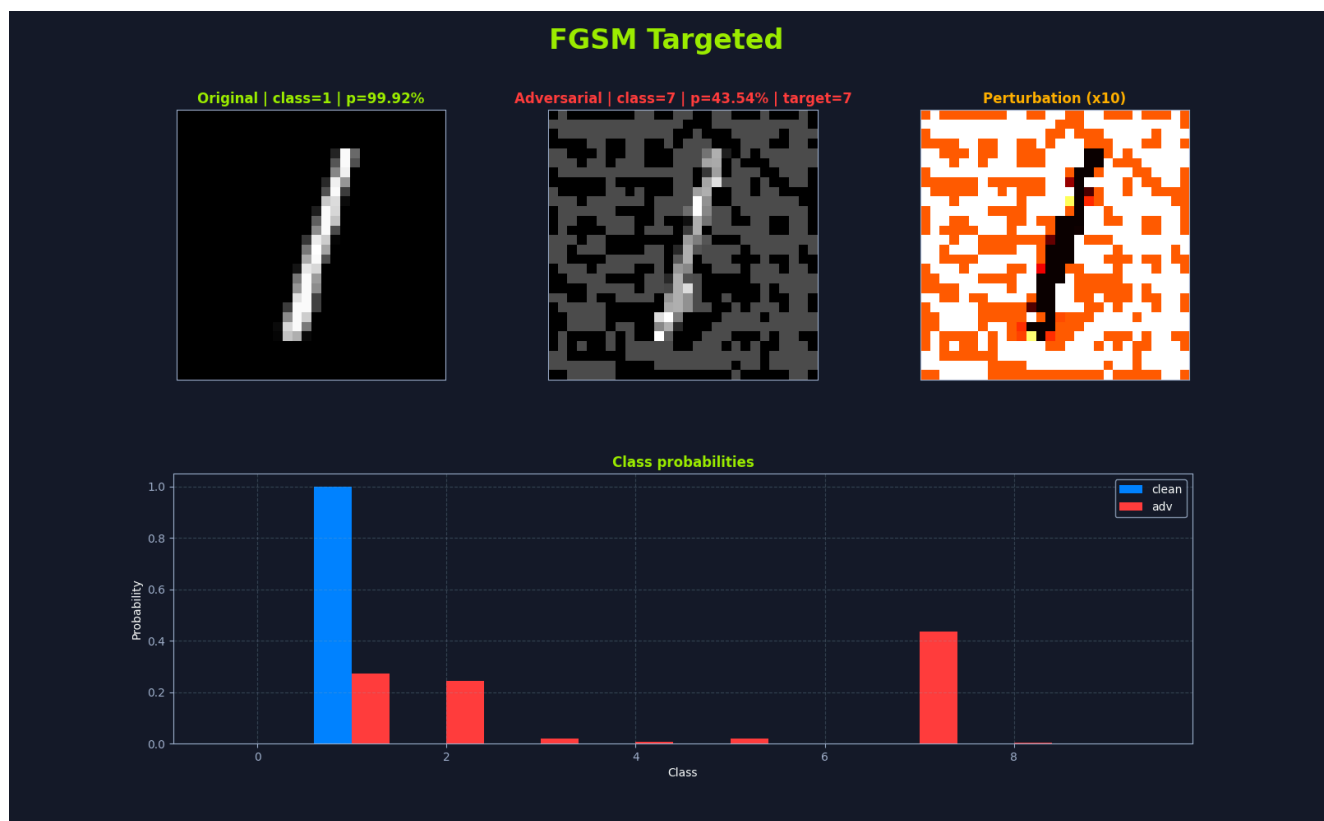
Expected output:

```

epsilon=0.50 -> predicted 1
epsilon=0.80 -> predicted 7

```

The output shows that `epsilon=0.5` in normalized space fails to flip the prediction, while increasing to `epsilon=0.8` succeeds and forces class 7. Targeted attacks often need larger budgets than untargeted ones because they must push toward a particular class, but the exact threshold depends on the specific sample and trained weights.



I-FGSM

The Iterative Fast Gradient Sign Method (I-FGSM), also known as the Basic Iterative Method (BIM), was introduced by Kurakin et al. in Adversarial Examples in the Physical World (2016) "linked below" as an extension of Goodfellow et al.'s original FGSM method. Instead of one large, well-aimed step, the algorithm takes several small, well-aimed steps. Each step follows the input gradient's sign, then projects back to the allowed L_∞ budget around the original image. The result is a refined adversarial that more reliably crosses decision boundaries with the same overall budget.

[*Adversarial Examples in the Physical World*](#)

The motivation is simple. A single first-order step is efficient but it approximates a curved landscape with a flat plane. By repeating small steps and re-evaluating the gradient at the updated point, the algorithm better tracks the local geometry and often finds stronger perturbations at the same ϵ .

The projection keeps the perturbation honest by enforcing the same per-pixel cap after every move, so improvements arise from better directions rather than larger budgets.

Core Update and Projection

Starting from

$$x^{(0)} = x,$$

the update for untargeted iterative FGSM is

$$x^{(t+1)} = \Pi_{\mathcal{B}^\infty(x, \epsilon)}(x^{(t)} + \alpha \text{sign}(\nabla_{x^{(t)}} \mathcal{L}(\theta, x^{(t)}, y))), x^{(t+1)} = \Pi_{\mathcal{B}^\infty(x, \epsilon)}(x^{(t)} + \alpha \text{sign}(\nabla_{x^{(t)}} \mathcal{L}(\theta, x^{(t)}, y))),$$

where

α

is the step size, typically

$$\alpha = \epsilon / T$$

for

T

iterations, and

$$\Pi_{\mathcal{B}^\infty(x, \epsilon)}(x)$$

projects to the

L_∞

ball of radius

ϵ

around

x

and then clips to the valid input domain. The projection is not an extra regularizer; it is the mathematical way to say "stay within the same per-pixel budget after each step." In coordinates, the projection is just per-pixel clipping:

$\Pi_{\mathcal{B}^\infty(x, \epsilon)}(x') = x + \text{clip}(x' - x, -\epsilon, \epsilon), x' \leftarrow \text{clip}(x', x_{\min}, x_{\max}). \Pi_{\mathcal{B}^\infty(x, \epsilon)}(x') = x + \text{clip}(x' - x, -\epsilon, \epsilon), \text{ where } x' \leftarrow \text{clip}(x', x_{\min}, x_{\max}).$

For targeted attacks, replace

y

by the target label

y_t

and reverse the step direction so that the update increases the target's score rather than the true label's score. This is the same idea as targeted FGSM, applied repeatedly with projection so the budget remains ϵ throughout.

Why Iteration Helps

The first step from

x

follows the gradient sign computed at

x ,

which is the exact optimizer of the linearized inner maximization for the

L_∞

constraint. After stepping, the loss surface is no longer

well-approximated by the original tangent plane. Recomputing the gradient at

$x^{(t)}$

and taking another projected step adjusts to the new local geometry.

Over several steps, this process tends to push examples closer to the true decision boundary while honoring the same budget

ϵ .

For quick illustration, consider a case where FGSM with

$\epsilon = 0.8$

moves the loss from 0.3 to 1.2 (a change of 0.9). With I-FGSM using 10

iterations and

$\alpha=0.08$, $\alpha=0.08$,

the loss might evolve as: $0.3 \rightarrow 0.45 \rightarrow 0.62 \rightarrow 0.81 \rightarrow 0.98 \rightarrow 1.15 \rightarrow 1.31 \rightarrow 1.44 \rightarrow 1.55 \rightarrow 1.63 \rightarrow 1.68$, achieving a final loss of 1.68 (a change of 1.38). The iterative refinement yields 53% more loss increase with the same

L_∞

budget.

Prerequisites

We'll build directly on the FGSM implementation from the previous sections. Ensure you have all the code from the FGSM Setup and Core Implementation sections, which provide the trained model, data loaders, and baseline attack functions.

For reference, you should have:

- A trained `SimpleCNN` model on MNIST with ~98% clean accuracy
- The `test_loader` and `device` configuration from the setup
- The `_input_gradient` and `fgsm_attack` functions from the FGSM implementation

I-FGSM Implementation

Building on Kurakin et al.'s I-FGSM algorithm (2016), the implementation of the projected iterative update uses

$\alpha = \frac{\epsilon}{T}$

by default with optional targeted behavior. Each iteration recomputes the gradient at the current adversarial and applies the per-pixel sign step followed by projection. The projection line enforces the

L_∞

cap relative to the original input rather than the previous iterate,

which keeps the budget interpretable as a maximum per-pixel change from

xx.

Core Iterative Algorithm

What makes iteration work? Small steps with fresh gradients. At each iteration, compute where you are, find the steepest direction, take a small sign step, then snap back to the allowed region:

```
def iterative_fgsm(model: nn.Module,
                  images: Tensor,
                  labels: Tensor,
                  epsilon: float,
```

```

        num_iter: int,
        alpha: float | None = None,
        targeted: bool = False,
        random_start: bool = False) -> Tensor:
    """Iterative FGSM (Basic Iterative Method) with projection.

    Args:
        model: Target classifier in evaluation mode
        images: Clean images (normalized)
        labels: Ground-truth or target labels
        epsilon: L_infinity budget (in normalized space)
        num_iter: Number of iterations
        alpha: Step size per iteration (defaults to epsilon/T)
        targeted: If True, targeted attack
        random_start: If True, initialize within the epsilon ball

    Returns:
        Tensor: Adversarial images (normalized)
    """
    # Valid normalized range for MNIST
    MNIST_NORM_MIN = (0.0 - 0.1307) / 0.3081
    MNIST_NORM_MAX = (1.0 - 0.1307) / 0.3081

    if alpha is None:
        alpha = epsilon / max(num_iter, 1)
    if random_start:
        torch.manual_seed(1337)
        delta = torch.empty_like(images).uniform_(-epsilon, epsilon)
        x_adv = torch.clamp(images + delta, MNIST_NORM_MIN,
MNIST_NORM_MAX)
    else:
        x_adv = images.clone()

    for _ in range(num_iter):
        x_adv = x_adv.detach().requires_grad_(True)
        logits = model(x_adv)
        loss = F.cross_entropy(logits, labels)
        model.zero_grad(set_to_none=True)
        loss.backward()
        step_dir = -1.0 if targeted else 1.0
        x_adv = x_adv + step_dir * alpha * x_adv.grad.sign()
        x_adv = torch.clamp(images + (x_adv - images).clamp(-epsilon,
epsilon), MNIST_NORM_MIN, MNIST_NORM_MAX)

```

```
return x_adv.detach()
```

The default step size $\alpha = \text{epsilon} / \max(\text{num_iter}, 1)$ divides the budget across iterations. If $\text{epsilon}=0.8$ and $\text{num_iter}=10$, then $\alpha=0.08$ per step. Even if all 10 steps point in the same direction (unlikely), the total movement is capped at $10 \times 0.08 = 0.8$.

The random start option adds uniform noise in $[-\text{epsilon}, \text{epsilon}]$ before iteration begins, helping escape local neighborhoods where the gradient points nowhere useful.

How does projection keep the perturbation honest? The line $(x_adv - \text{images}).\text{clamp}(-\text{epsilon}, \text{epsilon})$ measures how far each pixel has drifted from its original value and clips any drift exceeding $\pm \text{epsilon}$.

If a pixel has moved $+0.9$ but $\text{epsilon}=0.8$, clamp reduces it to $+0.8$. The outer

`torch.clamp(..., MNIST_NORM_MIN, MNIST_NORM_MAX)` then ensures all pixel values stay in the valid normalized input range.

Projection happens relative to the original `images`, not the previous iterate, so the budget always means

" ϵ away from the starting point," not " ϵ per step."

Testing Iterative FGSM

Does iteration really improve over single-step FGSM? Let's run I-FGSM on a batch and measure the flip rate. The test uses $\alpha = \frac{\epsilon}{T}$ and random starts to maximize effectiveness:

```
# Assume model, test_loader, device from FGSM Setup
images, labels = next(iter(test_loader))
images, labels = images.to(device), labels.to(device)

epsilon = 0.8
num_iter = 10
alpha = epsilon / num_iter # alpha = 0.08
```

```

with torch.no_grad():
    clean_pred = model(images).argmax(dim=1)

x_adv_ifgsm = iterative_fgsm(
    model, images, labels,
    epsilon=epsilon,
    num_iter=num_iter,
    alpha=alpha,
    targeted=False,
    random_start=True
)

with torch.no_grad():
    adv_pred_ifgsm = model(x_adv_ifgsm).argmax(dim=1)

originally_correct = clean_pred == labels
flipped_ifgsm = (adv_pred_ifgsm != labels) & originally_correct
print(
    f"I-FGSM flips (first batch): "
    f"{(flipped_ifgsm.float().sum() /
originally_correct.float().sum().clamp_min(1.0)).item():.2%}"
)

```

Expected output:

```
I-FGSM flips (first batch): 100.00%
```

The printed flip rate focuses on samples the model originally classified correctly to isolate genuine attack impact. In this run, every originally correct sample flipped, yielding 100.00%. Compare this to FGSM's 71.09% at the same epsilon. Iteration improves the success rate without increasing the perturbation budget.

Measuring Attack Impact

The flip rate is much higher, but what else changed? Confidence levels, perturbation norms, and the distribution of attack success across samples all provide insight into how α and

TT

affect outcomes:

```
# Reuse evaluate_attack function from the Evaluation Metrics section
metrics_ifgsm = evaluate_attack(model, images, x_adv_ifgsm, labels)
for k, v in metrics_ifgsm.items():
    print(f"{k}: {v:.4f}")
```

Expected output:

```
clean_accuracy: 1.0000
adversarial_accuracy: 0.0000
attack_success_rate: 1.0000
avg_clean_confidence: 0.9853
avg_adv_confidence: 0.0115
avg_confidence_drop: 0.9739
avg_l2_perturbation: 14.0519
max_linf_perturbation: 0.8000
```

These results show a complete collapse in accuracy under I-FGSM at the tested budget. With `clean_accuracy` of 1.0000, all samples were initially correct. After the attack, `adversarial_accuracy` drops to 0.0000 and `attack_success_rate` reaches 1.0000, meaning every originally correct sample was flipped. The `avg_confidence_drop` of 0.9739 indicates the model's probability assigned to the true class fell by about 97 percentage points on average, from 0.9853 to 0.0115. The `max_linf_perturbation` confirms the $\epsilon=0.8$ bound is respected; the higher `avg_l2_perturbation` reflects that the budget is used broadly across pixels rather than concentrated in a few locations.

Finally, the `max_linf_perturbation` of 0.8000 confirms the attack respects the epsilon bound in normalized space. Increasing

TT

with

$\alpha = \frac{\epsilon}{T}$

often raises success rate without increasing

$\max\|\delta\|_\infty$.

The

L2L₂

norm may drop slightly as the attack distributes the budget more efficiently across pixels rather than overshooting in some dimensions.

I-FGSM Analysis

Beyond basic implementation and testing, understanding how I-FGSM behaves across different configurations reveals optimization trade-offs. This section explores visualization, hyperparameter effects, the relationship to PGD, targeted variants, and direct comparisons with single-step FGSM.

Visualization

To see how I-FGSM modifies images visually, let's reuse the visualization infrastructure from the Visualization section. This displays clean and adversarial images side by side, along with the scaled perturbation and class probability shifts.

```
def visualize_ifgsm(model: nn.Module,
                   image: Tensor,
                   label: Tensor,
                   epsilon: float,
                   num_iter: int,
                   targeted: bool = False,
                   target_class: int | None = None) -> None:
    """Wrapper for visualize_attack using I-FGSM.

    Args:
        model: Classifier model
        image: Single image tensor [C,H,W]
        label: True label
        epsilon: Perturbation budget
        num_iter: Number of iterations
        targeted: If True, targeted attack
        target_class: Target class for targeted attacks
    """
    alpha = epsilon / max(num_iter, 1)

    def _make_adv(m, xb, yb):
        y_used = yb if not targeted else torch.full_like(yb, target_class)
        return iterative_fgsm(
            m, xb, y_used,
            epsilon, num_iter, alpha,
```



```

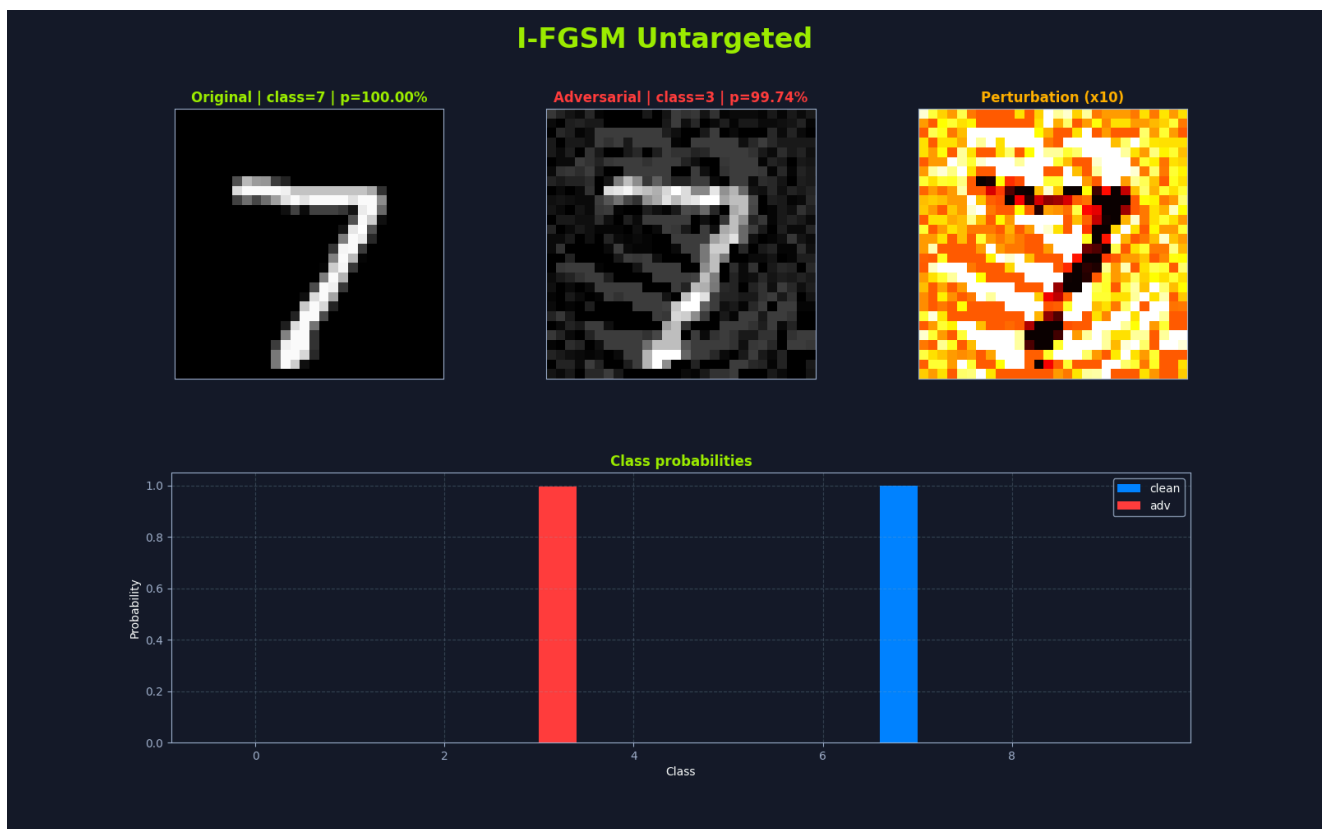
        targeted=targeted,
        random_start=True
    )

    mode = "Targeted" if targeted else "Untargeted"
    visualize_attack(
        model, image, label, _make_adv,
        title=f"I-FGSM {mode}",
        targeted=targeted,
        target_class=target_class
    )

# Visualize first sample from test batch
_ = visualize_ifgsm(
    model,
    images[0].detach().cpu(),
    labels[0].detach().cpu(),
    epsilon,
    num_iter,
    targeted=False
)

```

The visualization reveals how iterative refinement often achieves cleaner adversarial examples than single-step FGSM. The perturbation appears more structured, focusing on regions most sensitive to the model's decision.



Step Size, Iterations, and Random Starts

Choosing

α

and

T

balances speed and strength. The default

$$\alpha = \epsilon T \alpha = \frac{\epsilon}{T}$$

keeps the worst-case per-pixel change within the budget while allowing sufficient refinement.

Consider these trade-offs with specific numbers. With large α and few iterations (say

$$\alpha = 0.1$$

and

$$T = 2$$

the method moves quickly but may overshoot optimal adversarials. Loss changes might show rapid jumps: $0.3 \rightarrow 0.9 \rightarrow 1.1$. By contrast, small α with many iterations (such as

$$\alpha = 0.01$$

and

$$T = 20$$

refines gradually, producing smooth loss evolution: $0.3 \rightarrow 0.35 \rightarrow 0.41 \rightarrow \dots \rightarrow 1.45$. The gradual path often finds stronger adversarials because it tracks the curved loss surface more accurately.

Random starts add another dimension. Adding uniform noise $\delta \in [-\epsilon, \epsilon]$ before iterating explores different paths through the loss landscape. This exploration can increase success rate from 85% to 92% at the same budget, demonstrating how initialization affects final outcomes. In practice, random starts turn near-misses into successes by escaping local neighborhoods where the gradient points nowhere useful.

Relation to PGD

Projected Gradient Descent (PGD), formalized by Madry et al. in Towards Deep Learning Models Resistant to Adversarial Attacks (2017) "linked below", is the general form of iterative attacks under constraints. With the sign step, $\alpha = \epsilon T$, and optional random restarts, the implementation above coincides with the common BIM/PGD settings for the L_∞ threat model.

[Towards Deep Learning Models Resistant to Adversarial Attacks](#)

The relationship becomes clear when comparing update rules. I-FGSM uses the update

$$x^{(t+1)} = \Pi_{B_\infty}(x^{(t)} + \alpha \cdot \text{sign}(\nabla L))$$

to iteratively refine adversarial examples. PGD uses the same update but potentially includes multiple random restarts to explore different initialization points. BIM is essentially identical to I-FGSM but traditionally refers to the version without random initialization.

Random restarts further increase reliability by escaping poor local optima while keeping the same budget. Conceptually, PGD is I-FGSM plus a randomized initialization and, optionally, multiple attempts that keep the strongest adversarial found.

Targeted Iterative FGSM

Building directly on the targeted FGSM formulation, the iterative variant applies the same two changes (swap the label in the loss to the desired target

y_t

and reverse the step direction) but repeats them with projection after each step.

The per-iteration update becomes:

$$x^{(t+1)} = \Pi_{\mathcal{B}^\infty(x, \epsilon)}(x^{(t)} - \alpha \text{sign}(\nabla_x \mathcal{L}(\theta, x^{(t)}, y_t))), x^{(t+1)} = \Pi_{\mathcal{B}_\infty(x, \epsilon)}(x^{(t)} - \alpha \text{sign}(\nabla_{x^{(t)}} \mathcal{L}(\theta, x^{(t)}, y_t))),$$

This mirrors untargeted I-FGSM except for using y_t in the gradient and the minus sign. The targeted objective generally needs more iterations or slightly larger ϵ , and benefits from smaller α with more steps, random starts, and early stopping once the model predicts y_t .

Targeted Attack Example

Let's force a specific misclassification: turning a 1 into a 7. The attack supplies the target label 7 in the loss computation and reverses the update direction:

```
# Find one sample of '1'
one_img, one_lbl = None, None
for xb, yb in test_loader:
    m = (yb == 1)
    if m.any():
        j = m.nonzero(as_tuple=True)[0][0].item()
        one_img = xb[j].to(device)
        one_lbl = yb[j].to(device)
        break

# Try increasing epsilon values until successful
for eps_try in [0.5, 0.8, 1.0]:
    x_adv = iterative_fgsm(
        model,
        one_img.unsqueeze(0),
        torch.tensor(7, device=device).unsqueeze(0), # target label
        epsilon=eps_try,
        num_iter=num_iter,
        alpha=eps_try / max(num_iter, 1),
        targeted=True,
        random_start=True,
    )
    with torch.no_grad():
        pred = model(x_adv).argmax(dim=1).item()
```

```

print(f"epsilon={eps_try:.2f} -> predicted {pred}")

if pred == 7:
    _ = visualize_ifgsm(
        model,
        one_img.detach().cpu(),
        one_lbl.detach().cpu(),
        eps_try,
        num_iter,
        targeted=True,
        target_class=7,
    )
    break

```

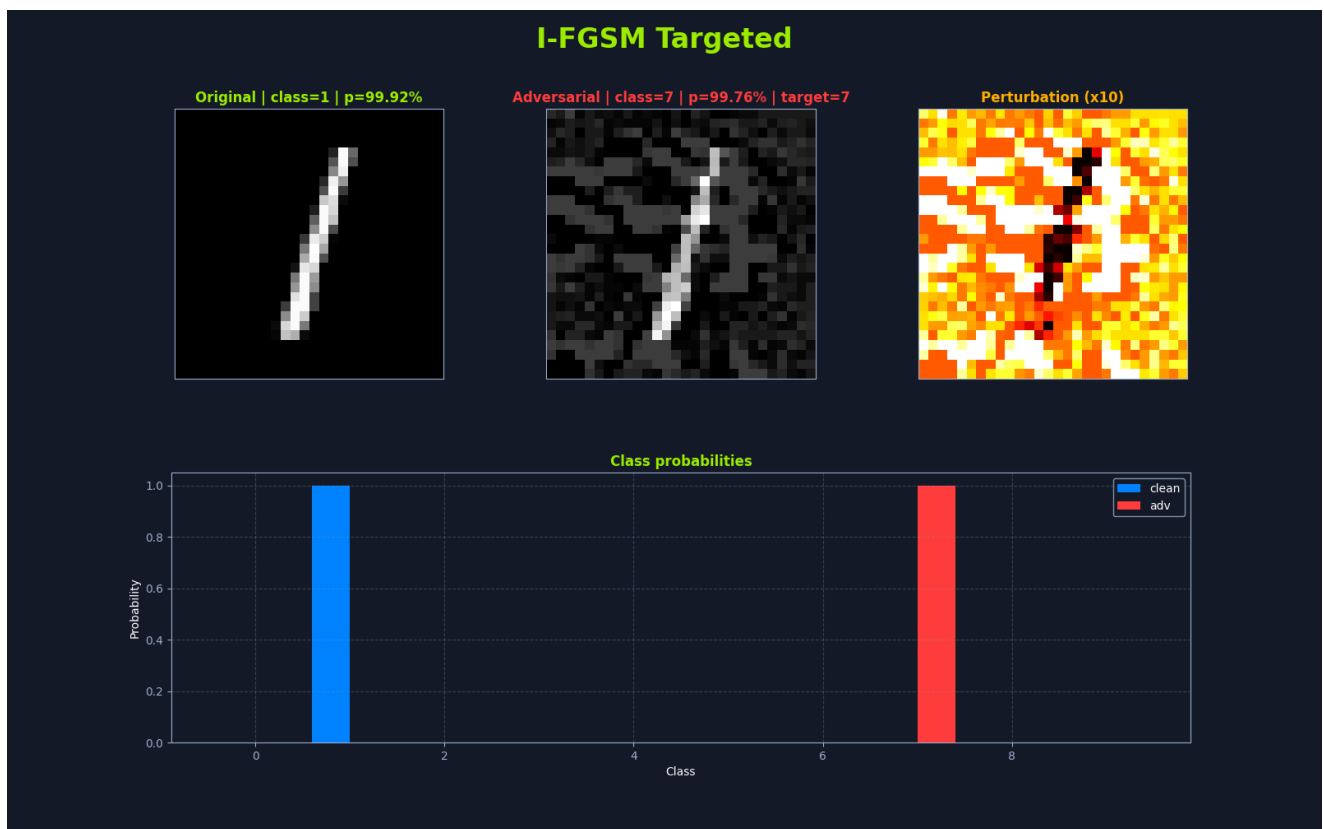
Expected output:

```

epsilon=0.50 -> predicted 1
epsilon=0.80 -> predicted 7

```

The loop progressively increases epsilon until the target is achieved. In this case, `epsilon=0.5` fails to achieve the target, with the prediction remaining at 1. Increasing to `epsilon=0.8` successfully achieves the target prediction 7. The visualization then shows the successful targeted attack. Note that I-FGSM achieves the targeted misclassification with a smaller epsilon (0.8 in normalized space) compared to FGSM which required `epsilon=1.0`, demonstrating the advantage of iterative refinement even for targeted attacks.



Comparison: FGSM vs I-FGSM

To demonstrate the improvement from iteration, both methods are compared on the same batch:

```
# Compare FGSM (one-step) and I-FGSM on the same batch
# Run both attacks with same epsilon
epsilon = 0.7
x_adv_fgsm = fgsm_attack(model, images, labels, epsilon)
x_adv_ifgsm = iterative_fgsm(
    model, images, labels,
    epsilon, num_iter=10,
    random_start=True
)

# Compare success rates
with torch.no_grad():
    fgsm_pred = model(x_adv_fgsm).argmax(dim=1)
    ifgsm_pred = model(x_adv_ifgsm).argmax(dim=1)

orig_correct = clean_pred == labels
fgsm_success = (
    ((fgsm_pred != labels) & orig_correct).float().sum()
    / orig_correct.float().sum().clamp_min(1.0)
)
```

```

ifgsm_success = (
    ((ifgsm_pred != labels) & orig_correct).float().sum()
    / orig_correct.float().sum().clamp_min(1.0)
)

print(f"FGSM success rate: {fgsm_success:.1%}")
print(f"I-FGSM success rate: {ifgsm_success:.1%}")
print(f"Improvement: {(ifgsm_success - fgsm_success) / fgsm_success:.1%}")

```

Expected output:

```

FGSM success rate: 57.8%
I-FGSM success rate: 95.3%
Improvement: 64.9%

```

These results show I-FGSM achieving 95.3% success where FGSM achieves 57.8%, a 64.9% relative improvement at the same

L_∞

budget of

$\epsilon=0.7$

in normalized space. Iterative refinement better exploits the loss

landscape within the same perturbation constraints, while both methods respect the identical per-pixel bound.

FGSM Challenge

Objective

Your task is to craft an adversarial example that fools an MNIST digit classifier using the Fast Gradient Sign Method. You will receive a baseline image that the classifier correctly predicts. Your job is to add a small, carefully crafted perturbation that causes misclassification while staying within a strict perturbation budget.

The constraint is

$\|x_{adv} - x\|_\infty \leq \epsilon$

where

x

is the baseline image and

x_{adv}

is your crafted adversarial example. Both conditions must hold: the

predicted class differs from the baseline label, and the maximum absolute pixel difference is at most `epsilon`.

Quick Start

All API endpoints expect images in `[0,1]` pixel space, not normalized tensors.

Start the instance from the platform. Check readiness:

```
export BASE_URL="http://instance_ip:port"
curl -s "$BASE_URL/health"
```

API

The API is minimal and deterministic. Each response is JSON. Images are base64-encoded PNG files of shape `28x28` and single channel in `[0,1]` after decoding.

GET /health

Returns service status and configuration parameters.

```
curl -s "$BASE_URL/health" | jq
```

Example response:

```
{
  "status": "ok",
  "epsilon": 0.25,
  "index": 2
}
```

GET /challenge

Returns the fixed sample and the constraint that must be satisfied.

```
curl -s "$BASE_URL/challenge" | jq
```

Example response shape:


```
{
  "sample_index": 2,
  "label": 1,
  "epsilon": 0.25,
  "image_b64": "<base64 PNG>"
}
```

The `sample_index` identifies the specific MNIST test sample used for this challenge. The `label` is the ground-truth class. The `epsilon` value defines the maximum L_{∞} distance allowed in `[0,1]` pixel space. The `image_b64` field contains the baseline image as a base64-encoded PNG.

POST /predict

Returns the model's prediction for an arbitrary image. Use this endpoint to validate the pipeline, encoding, and intermediate candidates. It does not return the flag.

```
curl -s -X POST "$BASE_URL/predict" \
  -H 'content-type: application/json' \
  -d '{"image_b64": "<base64 PNG of 28x28 grayscale>"}' | jq
```

Example response shape:

```
{
  "pred": 8,
  "confidence": 0.73
}
```

The `pred` field contains the predicted class (integer in `[0,9]`). The `confidence` field provides the softmax probability for the predicted class (float in `[0,1]`).

GET /weights

Returns the pretrained model `state_dict` as a binary file. Load it on CPU and use the provided classifier architecture to compute input gradients locally.

```
curl -s -o fgsm_weights.pth "$BASE_URL/weights"
```

The downloaded file is a PyTorch state dictionary compatible with the `SimpleClassifier` architecture detailed below. Load with `torch.load(path, map_location=torch.device('cpu'))`.

POST /submit

Validates that the candidate is a proper `28x28` single-channel image in `[0,1]`, that the L_∞ distance to the baseline image is at most `epsilon`, and that the classifier's predicted class differs from the baseline label. Returns the flag on success.

```
curl -s -X POST "$BASE_URL/submit" \
  -H 'content-type: application/json' \
  -d '{"image_b64": "<base64 PNG of 28x28 grayscale>"}' | jq
```

Example success response shape:

```
{
  "ok": true,
  "pred": 8,
  "linf": 0.24,
  "flag": "HTB{...}"
}
```

On validation failure, the endpoint returns HTTP 400 with a descriptive error message (e.g., `"L_inf too large: 0.26 > 0.25"` or `"Not misclassified; try a stronger perturbation"`).

Minimal Python Scaffolds

The following scaffolds provide API helpers, model architecture, and integration testing. They do not implement the attack.

API Helpers and Challenge Fetching

```
import os, io, base64, numpy as np, requests, torch, torch.nn as nn
from PIL import Image
```

```
BASE_URL = os.getenv("BASE_URL", "http://127.0.0.1:8000")
MNIST_MEAN, MNIST_STD = 0.1307, 0.3081
```

```
def x01_from_b64_png(b64: str) -> np.ndarray:
    """Convert base64 PNG to [0,1] numpy array.

    Args:
        b64: Base64 encoded PNG string

    Returns:
        np.ndarray: Image as (28, 28) array in [0,1] range
    """
    raw = base64.b64decode(b64)
    img = Image.open(io.BytesIO(raw)).convert("L")
    if img.size != (28, 28):
        raise ValueError("Expected 28x28 PNG")
    x = np.asarray(img, dtype=np.float32) / 255.0
    return np.clip(x, 0.0, 1.0)
```

```
def b64_png_from_x01(x2d: np.ndarray) -> str:
    """Convert [0,1] array to base64 PNG.

    Args:
        x2d: Image array in [0,1] range

    Returns:
        str: Base64 encoded PNG string
    """
    x255 = np.clip((x2d * 255.0).round(), 0, 255).astype(np.uint8)
    img = Image.fromarray(x255, mode="L")
    buf = io.BytesIO()
    img.save(buf, format="PNG", optimize=True)
    return base64.b64encode(buf.getvalue()).decode("ascii")
```

```
def linf(a: np.ndarray, b: np.ndarray) -> float:
    """Compute Linf distance between two arrays.

    Args:
        a, b: Arrays to compare
```

```

Returns:
    float: Maximum absolute difference
"""
return float(np.max(np.abs(a - b)))

# Fetch challenge and check clean prediction
ch = requests.get(f"{BASE_URL}/challenge", timeout=10).json()
x = x01_from_b64_png(ch["image_b64"])          # (28, 28)
lab = int(ch["label"])                          # baseline label
eps = float(ch["epsilon"])                      # numeric bound
res = requests.post(f"{BASE_URL}/predict", json={"image_b64":
b64_png_from_x01(x)}, timeout=10).json()
print({"baseline_label": lab, "server_pred": res["pred"], "epsilon": eps})

```

Model Architecture and Loading

The server uses this architecture. You need to replicate it locally to compute gradients.

```

class SimpleClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, 1)
        self.conv2 = nn.Conv2d(32, 64, 3, 1)
        self.dropout1 = nn.Dropout(0.25)
        self.dropout2 = nn.Dropout(0.5)
        self.fc1 = nn.Linear(9216, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x01: torch.Tensor) -> torch.Tensor:
        """Forward pass with internal normalization.

        Args:
            x01: Input tensor in [0,1] with shape (N, 1, 28, 28)

        Returns:
            Log-probabilities with shape (N, 10)
        """
        x = (x01 - MNIST_MEAN) / MNIST_STD
        x = torch.relu(self.conv1(x))
        x = torch.relu(self.conv2(x))
        x = torch.max_pool2d(x, 2)
        x = self.dropout1(x)

```

```

        x = torch.flatten(x, 1)
        x = torch.relu(self.fc1(x))
        x = self.dropout2(x)
        x = self.fc2(x)
        return torch.log_softmax(x, dim=1)

# Download and load weights
wt = requests.get(f"{BASE_URL}/weights", timeout=10).content
open("fgsm_weights.pth", "wb").write(wt)

model = SimpleClassifier().eval()
state = torch.load("fgsm_weights.pth", map_location=torch.device("cpu"))
model.load_state_dict(state)

# Verify model works locally
x_tensor = torch.from_numpy(x[None, None, ...]).float()
logits = model(x_tensor)
local_pred = int(torch.argmax(logits, dim=1).item())
print(f"Local prediction: {local_pred}, should match server:
{res['pred']}")

```

Testing Server Validation

Verify server-side checks by submitting the clean image and observing expected failures.

```

bad = requests.post(f"{BASE_URL}/submit", json={"image_b64":
b64_png_from_x01(x)}, timeout=10)
print(bad.status_code, bad.text) # expected: 400 with "Not misclassified"

```

Enable step-by-step solutions for all questions



Questions

Answer the question(s) below

to complete this Section and earn cubes!

Click here to spawn the target
system!

Target(s): Fetching status...

Life Left: 0 minute(s)

+ 5 After successfully completing the challenge, what is the flag you receive?

+10 Streak pts

Submit

DeepFool and the Quest for Minimality

The Fast Gradient Sign Method demonstrated efficient adversarial generation through a single gradient step. One backward pass. One step along the gradient sign. Adversarial example generated. Yet FGSM operates within a predetermined perturbation budget ϵ , treating all pixels equally regardless of how much each actually contributes to changing the decision. The perturbations FGSM produces, while effective, are not necessarily minimal.

What is the smallest perturbation needed to fool a neural network? This question cuts to the heart of adversarial robustness. The answer matters. Smaller perturbations evade detection more easily, transfer more reliably between models, and provide tighter bounds on a model's true robustness, moving from approximate measures to precise geometric measurements.

DeepFool emerged in 2016 as an answer to this challenge. The algorithm, introduced by Moosavi-Dezfooli, Fawzi, and Frossard at CVPR 2016, treats adversarial example generation as a geometric problem: find the shortest path from a data point to the closest decision boundary.

From Fixed Budgets to Minimal Perturbations

The transition from FGSM's fixed-budget approach to DeepFool's minimal-perturbation philosophy represents a clear shift in attack design. FGSM asks "given a hammer of size ϵ , where should I strike?" DeepFool asks something entirely different: "what is the smallest hammer that will break this?" The contrast exposes competing philosophies. FGSM maximizes damage within a fixed L_∞ budget ϵ , operating under predetermined constraints. DeepFool minimizes the budget needed to achieve misclassification, discovering constraints rather than imposing them.

Consider the `perturbation budget epsilon` in FGSM. Setting this value requires prior knowledge or assumptions about the model's vulnerability. Too small? The attack fails. Too large? The perturbation becomes detectable or destroys semantic meaning. The choice of `epsilon` becomes a hyperparameter that conflates the attack method with the model's actual robustness, mixing measurement with methodology. Two models might appear equally robust under one choice of `epsilon` but show markedly different vulnerabilities when `epsilon` changes, demonstrating that the robustness measure depends more on parameter selection than on true model characteristics.

DeepFool eliminates this arbitrary choice entirely. By finding the `minimal sufficient perturbation` for each input individually, the resulting perturbation magnitude becomes a direct measurement of that input's robustness, not a reflection of parameter choices. This shift transforms adversarial attacks from destruction tools into measurement instruments, providing a principled way to compare robustness across different models, architectures, and even different inputs within the same model.

Mathematical Foundations

Recall from the FGSM exploration that neural networks partition the input space into regions, with `decision boundaries` separating different classes. FGSM moves perpendicular to these boundaries in the L_∞ sense, taking the largest allowed step. A different question asks for the shortest path to any boundary, regardless of direction.

What does the geometry of these boundaries show about minimal perturbations? Decision boundaries in high-dimensional spaces have specific properties that DeepFool exploits. While we often visualize decision boundaries as curves or surfaces in 2D or 3D, in the high-dimensional spaces where real neural networks operate (thousands to millions of dimensions for image classifiers), these boundaries form complex `manifolds`. The distance from any point to the nearest boundary varies widely depending on direction. Some directions might require traversing large distances to reach a boundary, while others might encounter one almost immediately.

Linear Classifiers and Optimal Projections

For a binary linear classifier

$$f(x) = w^T x + b$$

finding the minimal

L_2

perturbation is straightforward. The classifier computes a score by taking a weighted sum of all input features

$(w^T x + b)$

means multiply each feature by its corresponding weight and sum them plus a bias term

b .

When this score is positive, the classifier predicts one class; when negative, it predicts the other. The decision boundary is the hyperplane where

$f(x) = 0$,

the exact surface where the classifier is perfectly undecided between the two classes.

To find the shortest distance from a point

x_0

to this hyperplane, we use a formula from geometry:

$$d = \frac{|f(x_0)|}{\|w\|_2}$$

Think of this formula as a fraction with two parts. The numerator

$|f(x_0)|$

is the absolute value of the classifier's score at the starting point.

If the classifier outputs a score of 10, the point is "10 units" away from the boundary in the classifier's internal measurement. The denominator

$\|w\|_2$

is the norm (the "length") of the weight vector. It

indicates how "steep" the classifier's decision landscape is. A larger weight norm means the output changes more rapidly as the input moves, so a shorter distance is required to reach the boundary.

The minimal perturbation that reaches the boundary is:

$$r^* = -\frac{f(x_0)}{\|w\|_2^2} w$$

This formula specifies how to change the input to reach the decision boundary with the smallest possible change. The vector

w

provides the direction to move (it points "uphill" in the classifier's landscape). The negative sign indicates movement "downhill" toward the boundary when in the positive region. The fraction

$\frac{f(x_0)}{\|w\|_2^2}$

determines the step size in that direction. The term

$$\|w\|_2 \|w\|_2$$

appears squared in the denominator because it accounts both for normalization of the direction and for how quickly the landscape changes.

This formula shows the geometric intuition: we project the point orthogonally onto the decision boundary. The perturbation scales with the distance to the boundary

$$(|f(x_0)| |f(x_0)|)$$

and inversely with the gradient magnitude

$$(\|w\|_2 \|w\|_2^2).$$

Unlike FGSM's sign operation that equalizes all pixel changes, this approach naturally weights each dimension by its importance. If one pixel has a weight of 10 and another has a weight of 1, the first pixel will change ten times as much in the perturbation, because it has ten times the influence on the classification.

The orthogonality of this projection is mathematically optimal. In the

$$L_2$$

norm, the shortest path between a point and a hyperplane is always perpendicular to that hyperplane. This perpendicular direction aligns with the gradient of the classifier function,

∇f ,

but unlike FGSM, only taking the sign is avoided. Relative magnitudes across dimensions are preserved, allowing pixels or features that strongly influence the classification to change more than those with weak influence.

The term

$$|f(x_0)| |f(x_0)|$$

in the numerator represents the classifier's confidence at the original point. Points classified with high confidence (large

$$|f(x_0)| |f(x_0)|)$$

require larger perturbations to flip, while points near the boundary

(small

$$|f(x_0)| |f(x_0)|)$$

need only tiny nudges. This relationship between confidence and robustness seems intuitive, but deep networks often violate this intuition, showing high confidence on points that are extremely close to decision boundaries.

Extending to Deep Networks through Iterative Linearization

Deep networks introduce complexity. Serious complexity. Their decision boundaries are not flat hyperplanes but curved surfaces that fold, twist, and create intricate patterns across thousands of dimensions. How can we find the minimal path to a boundary when we cannot see its global shape? The simple projection formula for linear classifiers breaks down. Why? Linear approximations only hold in tiny neighborhoods around each point, making global calculations impossible.

DeepFool solves this through `iterative linearization`. Think of navigating a curved mountain in dense fog. Only the immediate terrain is visible. At each position, compute the best local direction based on what you can see. Take a small step. Reassess from the new vantage point. Repeat until reaching the boundary. This strategy builds a path incrementally, with each step guided by fresh local information rather than a single global calculation that might miss the boundary's curvature entirely.

The process unfolds through careful local approximations. First, DeepFool linearizes the classifier around the current point using first-order Taylor expansion. Think of Taylor expansion as zooming in on a curved line until it looks straight. If you zoom in close enough on any smooth curve, it appears linear. For a neural network with output

$f(x)$

for the true class and

$g(x)$

for an alternative class, the linearization around point

x_i

gives us

$f(x) \approx f(x_i) + \nabla f(x_i)^T (x - x_i)$

and similarly for

$g(x)$.

Breaking this down:

$f(x_i)$

is the network's current output,

$\nabla f(x_i)$

is the gradient (which tells how the output changes as each input feature is modified), and

$(x - x_i)$

is the displacement from the current position. The formula essentially says: "the new output equals the current output plus the rate of change times the distance moved." This transforms the complex non-linear decision boundary into a simple hyperplane that approximates the true boundary near the current location.

What comes next? Compute the minimal perturbation for this linear approximation using the closed-form solution for linear classifiers. This gives the optimal direction and distance to

move if the classifier were exactly linear. The key insight is that while this perturbation might not reach the actual non-linear boundary, it moves closer to it in a principled way.

Step size matters critically. Too large, and the algorithm risks overshooting or trusting an inaccurate linear approximation far from the current point. Too small, and convergence crawls. DeepFool typically takes the full computed step, betting on sufficient local linearity near well-trained decision boundaries.

The algorithm re-evaluates the network at the new point and repeats. Each iteration provides a fresh linear approximation at the new location, gradually building a path that follows the boundary's curvature. The iterations continue until the classification changes, signaling successful crossing of the decision boundary.

DeepFool adaptively chooses both direction and magnitude to minimize the total perturbation. Each iteration refines the path based on the local geometry, progressively approaching the true non-linear decision boundary. The accumulated perturbations from all iterations sum to create the final adversarial perturbation, which approximates the minimal perturbation needed to cross the non-linear boundary.

This approach is adaptable. In regions where the decision boundary is nearly linear, DeepFool might reach it in a single iteration. In regions of high curvature, it automatically takes more iterations, each adjusting to follow the boundary's contours. This adaptive behavior ensures that the final perturbation remains close to minimal regardless of the local geometry's complexity.

DeepFool Theory and Formulation

The previous section established DeepFool's geometric foundations through binary classification and iterative linearization. The algorithm projects inputs orthogonally onto linearized decision boundaries, accumulating perturbations until the true boundary is crossed. Real-world classifiers, however, operate on many classes simultaneously. How does this geometric framework extend when multiple decision boundaries surround each point?

Multi-Class Formulation

The iterative linearization strategy works well for binary classification, where only one decision boundary separates the classes. Real-world classifiers typically handle many classes simultaneously. MNIST recognizes 10 digits. ImageNet classifies 1000 object categories. From any point in the input space, multiple decision boundaries exist, each separating the current predicted class from a different alternative. Which boundary should DeepFool target?

The answer: whichever is closest. Rather than committing to a predetermined target class, DeepFool dynamically identifies the nearest boundary at each iteration and takes the

minimal step toward it. This adaptive strategy ensures the final perturbation approximates the minimal perturbation needed to cross any decision boundary, not just a specific one.

For an input

x

classified as class

$$\hat{k}(x) = \arg\max_k f_k(x) \quad \hat{k}(x) = \arg\max_k f_k(x),$$

where

$$f_k(x)$$

denotes the score for class

k ,

DeepFool seeks the minimal perturbation

r

such that

$$\hat{k}(x+r) \neq \hat{k}(x) \quad \hat{k}(x+r) \neq \hat{k}(x).$$

At each iteration, for every alternative class

$$k \neq \hat{k}(x_i) \quad k \neq \hat{k}(x_i),$$

define the gradient difference

$$w_k = \nabla f_k(x_i) - \nabla f_{\hat{k}(x_i)}(x_i) \quad w_k = \nabla f_k(x_i) - \nabla f_{\hat{k}(x_i)}(x_i)$$

and the score gap

$$f_k' = f_k(x_i) - f_{\hat{k}(x_i)}(x_i) \quad f_k' = f_k(x_i) - f_{\hat{k}(x_i)}(x_i).$$

The vector

$$w_k$$

points in a direction that increases class

k 's

score while decreasing the current class's score. For instance, if the current class score is 8 and class

k 's

score is 3, then

$$f_k' = 3 - 8 = -5 \quad f_k' = 3 - 8 = -5,$$

indicating a five-point deficit to overcome.

The closest boundary is identified by

$$l = \arg\min_{k \neq \hat{k}(x_i)} |f_k'| / \|w_k\|_2 = \arg\min_{k \neq \hat{k}(x_i)} \frac{|f_k'|}{\|w_k\|_2},$$

where the numerator

$$|f_k'|$$

is the gap to close and the denominator

$$\|w_k\|_2$$

reflects how quickly that gap can be closed along direction

$$w_k.$$

The minimal step is then

$$r_l = |f_l'| / \|w_l\|_2^2 \quad r_l = \frac{|f_l'|}{\|w_l\|_2^2} w_l,$$

the orthogonal projection onto the closest linearized boundary, with

$$\|w_l\|_2^2$$

normalizing by sensitivity. The perturbed input updates as $x_{i+1} = x_i + r_i x_{\{i+1\}} = x_i + r_i$ before re-linearizing if the classification has not yet changed.

This formulation generalizes the binary case naturally. At each iteration, DeepFool evaluates all alternative classes, computes the distance to each boundary, and takes the minimal step toward the closest one. The geometric intuition remains identical: find the shortest path. The multi-class extension simply adds a selection step to identify which of many boundaries lies nearest.

The Overshoot Parameter

DeepFool includes an `overshoot` parameter (typically 0.02) that slightly overshoots the decision boundary: the actual step taken is

$$(1 + \text{overshoot}) \times r_i (1 + \text{overshoot}) \times r_i.$$

This serves two purposes:

The linearization at each point is only locally accurate. A small overshoot ensures we actually cross the non-linear boundary rather than merely touching it in the linear approximation. Without this, numerical precision issues could leave us infinitesimally close to but not across the boundary.

The overshoot also accelerates convergence by taking slightly larger steps, reducing the number of iterations needed. The trade-off is a marginally larger final perturbation, but the 2% typical value keeps this increase negligible while improving reliability.

Comparison with Iterative FGSM

Both methods iterate. Both compute gradients multiple times. Yet their algorithmic approaches differ fundamentally in how they use gradient information.

`Iterative FGSM` takes uniform steps in the direction of the gradient sign. Each iteration applies the same step size across all pixels, then projects the result back onto the constraint set. The sign operation discards magnitude information, treating a gradient component of 0.001 identically to one of 100.

DeepFool preserves gradient magnitudes to compute geometrically minimal steps. Each iteration identifies the closest decision boundary among all classes and takes exactly the step needed to reach that specific boundary in the linearized approximation. Features with large gradient components naturally receive larger perturbations, while features with small gradients change minimally. This adaptive weighting emerges automatically from the orthogonal projection formula rather than being imposed through hyperparameter selection.

Robustness Evaluation

DeepFool provides a quantitative measure of network robustness through the average minimal perturbation required to fool the classifier. This metric, often denoted as ρ_{adv} , is computed as:

$$\rho_{adv} = \frac{1}{|D|} \sum_{x \in D} \frac{\|r(x)\|_2}{\|x\|_2} = \frac{1}{|D|} \sum_{x \in D} \frac{\|r(x)\|_2}{\|x\|_2}$$

where

D

is a dataset and

$r(x)$

is the minimal perturbation found by DeepFool for input x .

This formula computes the average "relative perturbation size" across a dataset. For each image, the size of the perturbation

$$\|r(x)\|_2$$

is divided by the size of the original image

$$\|x\|_2$$

to obtain a percentage-like measure. If an image with norm 100 requires a perturbation with norm 2 to fool the classifier, that corresponds to a 2% relative perturbation. These percentages are then averaged across all images in the dataset

ρ_{adv} .

A model with

$$\rho_{adv} = 0.02$$

requires, on average, perturbations that are 2% the size of the original input to be fooled, while a model with

$$\rho_{adv} = 0.10$$

requires 10% perturbations, making it five times more robust.

This robustness measure has several advantages over other metrics. It directly quantifies the distance to decision boundaries, provides a normalized measure that accounts for input magnitude, and can be computed efficiently for large datasets. Networks with larger ρ_{adv} values are generally more robust to adversarial perturbations.

Extensions and Variations

The core DeepFool algorithm uses the

L_2

norm, but the methodology extends to other settings. The

L_∞

version finds the minimal perturbation in terms of maximum absolute change per pixel. For

L_∞

DeepFool, the distance computation changes:

$$\hat{l} = \operatorname{argmin}_{l \neq k} (x_i) \|f_k' - f_l'\|_1, r_i = \|f_l'\|_1 \operatorname{sign}(w_l) \cdot \hat{l}; \quad \operatorname{argmin}_{k \neq \hat{k}} (x_i) \frac{\|f_{k'} - f_{\hat{l}}'\|_1}{\|w_{\hat{l}}\|_1}, \quad r_i = \frac{\|f_{\hat{l}}'\|_1}{\|w_{\hat{l}}\|_1} \cdot \operatorname{sign}(w_{\hat{l}}).$$

The denominator switches from

L_2

norm to

L_1

norm, and the perturbation direction uses element-wise sign rather than normalized gradient. This distributes the perturbation uniformly across pixels, with magnitude

$$\|f_l'\|_1 \|w_l\|_1 \frac{\|f_l'\|_1}{\|w_l\|_1}$$

setting the amount needed to reach the boundary under the

L_∞

constraint.

Building on DeepFool's minimal perturbation principle, researchers developed **universal adversarial perturbations**: single perturbations that fool a network on most inputs from a dataset. The algorithm iteratively applies DeepFool to different training examples and accumulates the perturbations, with constraints on total magnitude. The existence of universal perturbations demonstrates that deep neural networks exhibit systematic vulnerabilities consistent across different inputs.

Building and Training the Target Model

DeepFool requires a trained neural network as the attack target. If you completed the FGSM module, the environment setup follows the same pattern using the HTB Evasion Library. The key difference lies in the model architecture: DeepFool benefits from dropout regularization to create more realistic decision boundaries for adversarial testing.

Environment Setup

The environment setup is identical to the FGSM Setup section, and we will reuse that entire scaffold. The only difference is importing `MNISTClassifierWithDropout` instead of `SimpleCNN`, plus caching utilities (`save_model`, `load_model`, `analyze_model_confidence`). For installation instructions, import details, and explanations of `set_reproducibility`, device selection, and data loading, refer to that section.

Key imports specific to DeepFool:

```

from htb_ai_library import (
    set_reproducibility,
    MNISTClassifierWithDropout,
    get_mnist_loaders,
    train_model,
    evaluate_accuracy,
    save_model,
    load_model,
    analyze_model_confidence,
    HTB_GREEN, NODE_BLACK, HACKER_GREY, WHITE,
    AZURE, NUGGET_YELLOW, MALWARE_RED, VIVID_PURPLE, AQUAMARINE
)

set_reproducibility(1337)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

```

Model Architecture: Why Dropout Matters

DeepFool requires a different architecture than FGSM. Why add dropout when FGSM used SimpleCNN without it? The answer lies in what we're measuring. An overfitted model might show extremely confident but fragile predictions near training examples, with decision boundaries tightly wrapped around memorized data points. Dropout forces the network to learn redundant representations where multiple feature combinations can indicate the same digit, creating boundaries that better reflect genuine uncertainty.

The architecture uses two convolutional blocks with a $3 \times 3 \times 3$ kernel. The first block outputs 32 channels with 25% dropout, the second outputs 64 channels. Max pooling reduces spatial dimensions from 28×28 to 7×7 . After flattening to 3136 dimensions ($64 \times 7 \times 7 \times 7$), a fully connected layer with 128 units and 50% dropout processes features before the final 10-class output layer.

The different dropout rates serve specific purposes. Convolutional features are spatially local and numerous (thousands of activations), so conservative 25% dropout prevents destroying spatial information. The fully connected layer has fewer units (128) but each connects to all previous features, creating overfitting risk. The aggressive 50% rate prevents reliance on specific neuron combinations.

During DeepFool's gradient computation, PyTorch automatically disables dropout through `model.eval()`. The attack sees the complete network without random feature zeroing, providing stable gradients for the iterative perturbation process.

Architecture Definition

The library provides this architecture:

```
# MNISTClassifierWithDropout is imported from htb_ai_library
# The architecture internally defines:
# - Conv1: 1->32 channels, 3x3 kernel, ReLU, 2x2 pooling, 25% dropout
# - Conv2: 32->64 channels, 3x3 kernel, ReLU, 2x2 pooling, 25% dropout
# - FC1: 3136->128, ReLU, 50% dropout
# - FC2: 128->10 (logits)

model = MNISTClassifierWithDropout().to(device)
print(f"Model parameters: {sum(p.numel() for p in model.parameters()):,}")
```

Training with Smart Caching

Training follows the same pattern as FGSM, but with intelligent caching to avoid retraining when a valid model already exists. The pipeline first checks for cached models, validates their accuracy, and only retrains when necessary. This saves significant time during iterative development.

File Path Configuration

The caching system requires persistent storage for model weights and metadata:

```
model_path = 'output/mnist_model.pth'
os.makedirs('output', exist_ok=True)
```

Why use `output/mnist_model.pth` instead of a temporary file? The `.pth` extension signals PyTorch serialization format to both humans and tools. Storing in a dedicated `output` directory separates generated artifacts from source code, making cleanup and version control exclusion straightforward. The same path serves dual purposes: checking for existing cached models via `os.path.exists(model_path)` and saving newly trained models via `save_model(..., model_path)`.

Loading and Validating Cached Models

Before training, we check if a cached model exists and whether it meets our accuracy requirements:

```

# Try loading cached model
if os.path.exists(model_path):
    print(f"Found cached model at {model_path}")
    model_data = load_model(model_path)
    model = model_data['model'].to(device)
    model.eval()

# Validate cached model
_, test_loader = get_mnist_loaders(batch_size=100, normalize=True)
accuracy = evaluate_accuracy(model, test_loader, device)
print(f"Cached model accuracy: {accuracy:.2f}%")

if accuracy < 90.0:
    print("Accuracy below threshold, retraining required")
    model = None
else:
    model = None

```

The validation step evaluates the cached model on test data using a batch size of 100, which balances evaluation speed with memory usage. The `evaluate_accuracy` function runs the model on all 10,000 MNIST test samples and computes the percentage of correct predictions. If accuracy falls below 90%, the model is rejected by setting `model = None`, triggering retraining. This threshold ensures we use only well-trained models for adversarial testing. Setting `model.eval()` disables dropout and switches batch normalization to inference mode, ensuring consistent evaluation behavior.

Training When Required

If no valid cached model exists or the cached model fails validation, training begins:

```

# Train if needed
if model is None:
    print("Training new model...")
    train_loader, test_loader = get_mnist_loaders(batch_size=64,
normalize=True)
    model = MNISTClassifierWithDropout().to(device)

model = train_model(
    model, train_loader, test_loader,
    epochs=5, device=device

```

)

Why 5 epochs instead of the single epoch used in FGSM? Dropout regularization slows convergence because each training iteration randomly disables different neurons, preventing the network from memorizing efficient feature paths. A single epoch with dropout typically reaches only 96-97% accuracy. Five epochs allow the network to discover multiple redundant feature combinations and stabilize its decision boundaries, consistently achieving 98-99% accuracy. The training mechanics (forward passes, loss computation, Adam optimization) remain identical to FGSM.

Persisting Model State

Training produces an ephemeral model in GPU or CPU memory. To enable reuse across sessions, we serialize both weights and metadata:

```
# Evaluate and cache
accuracy = evaluate_accuracy(model, test_loader, device)
print(f"Test Accuracy: {accuracy:.2f}%")

save_model({
    'model': model,
    'architecture': 'MNISTClassifierWithDropout',
    'accuracy': accuracy,
    'training_config': {
        'epochs': 5,
        'batch_size': 64,
        'device': str(device)
    }
}, model_path)
```

What gets serialized inside the dictionary? The `model` key stores the complete `nn.Module` object including architecture definition and trained parameters. PyTorch's `torch.save` recursively pickles this object, capturing layer structures, weight tensors, optimizer state, and internal buffers. The remaining keys store human-readable metadata: `architecture` identifies which model class to instantiate when loading, `accuracy` enables the validation step we saw earlier (rejecting models below 90%), and `training_config` preserves hyperparameters for reproducibility. This structure allows loading the model in a fresh Python session without recompiling or retraining. Typical accuracy falls between 98.5-99.0%, meaning approximately 9,850-9,900 correct classifications out of 10,000 test images.

Expected training output:

```
Training MNIST classifier on cuda...
Epoch 1/5 - Loss: 0.2134 - Accuracy: 93.54%
Epoch 2/5 - Loss: 0.0891 - Accuracy: 97.23%
Epoch 3/5 - Loss: 0.0623 - Accuracy: 98.12%
Epoch 4/5 - Loss: 0.0502 - Accuracy: 98.54%
Epoch 5/5 - Loss: 0.0421 - Accuracy: 98.76%
Test Accuracy: 98.95%
```

Model Verification

Verify the trained model exhibits appropriate confidence distributions:

```
# Analyze confidence distribution
_, test_loader = get_mnist_loaders(batch_size=100, normalize=True)
stats = analyze_model_confidence(model, test_loader, device=device,
num_samples=1000)
```

The `analyze_model_confidence` function returns a dictionary with confidence statistics. The trained model typically achieves approximately 99% accuracy on correctly predicted samples with high confidence (mean ~ 0.99), while incorrect predictions show lower confidence (mean ~ 0.6 - 0.7). This confidence distribution is ideal for DeepFool: high certainty means finding minimal perturbations requires navigating genuinely learned features rather than exploiting random noise.

DeepFool Implementation

The theoretical foundation established how DeepFool iteratively linearizes decision boundaries to find minimal perturbations. Now we translate this mathematical framework into executable code that performs multi-class attacks through adaptive boundary detection and refinement.

Function Definition and Initialization

The DeepFool function accepts an input image and a trained neural network, along with parameters controlling the algorithm's behavior. The `num_classes` parameter limits the search to the top-scoring classes, reducing computational cost. The `overshoot` factor ensures the algorithm crosses the true non-linear boundary rather than just touching the linearized approximation.

```
def deepfool(image: torch.Tensor,
             net: nn.Module,
```

```

        num_classes: int = 10,
        overshoot: float = 0.02,
        max_iter: int = 50,
        device: str = 'cuda') -> Tuple[torch.Tensor, int, int, int,
torch.Tensor]:
    """
    Generate minimal adversarial perturbation using DeepFool algorithm.

    Args:
        image (torch.Tensor): Input image tensor of shape (1, C, H, W)
        net (nn.Module): Target neural network in evaluation mode
        num_classes (int): Number of top-scoring classes to consider
        (default: 10)
        overshoot (float): Overshoot parameter for boundary crossing
        (default: 0.02)
        max_iter (int): Maximum iterations before terminating (default:
50)
        device (str): Computation device ('cuda' or 'cpu')

    Returns:
        Tuple containing:
        - r_tot (torch.Tensor): Total accumulated perturbation
        - loop_i (int): Number of iterations performed
        - label (int): Original predicted class
        - k_i (int): Final adversarial class
        - pert_image (torch.Tensor): Final perturbed image
    """
    image = image.to(device)
    net = net.to(device)

```

Device transfers ensure computations occur on the selected hardware for both inputs and model. The function signature defines the expected return values: the total perturbation accumulated across iterations, the number of iterations required, the original and adversarial class labels, and the final perturbed image.

Initial Classification and Class Ranking

The algorithm begins by establishing the original classification and ranking all classes by their scores. This ranking determines the order in which alternative classes are considered during the search for the minimal perturbation.

```
# Original prediction and class ordering (descending score)
f_image = net(image).data.cpu().numpy().flatten()
I = f_image.argsort()[::-1]
label = I[0]
```

The network produces logits for each class, which are converted to a NumPy array and flattened to 1D. The expression `argsort()[::-1]` sorts class indices in descending order of their scores. For example, if the logits are `[2.1, 5.3, 0.8, 3.7]` for classes 0-3, then `I` becomes `[1, 3, 0, 2]`, indicating class 1 has the highest score (5.3), followed by class 3 (3.7), class 0 (2.1), and class 2 (0.8). The variable `label = I[0]` selects the predicted class, which is the index with the highest score.

Initialization of Working Variables

The algorithm maintains several tensors throughout the iterative process: the current perturbed image, the accumulated perturbation, and iteration counters.

```
# Working tensors and accumulators
input_shape = image.shape
pert_image = image.clone()
r_tot = torch.zeros(input_shape).to(device)
loop_i = 0
```

The tensor `pert_image` holds the current perturbed image, initially equal to the original input. The accumulator `r_tot` tracks the total perturbation applied across all iterations, starting at zero. As the algorithm iterates, `r_tot` accumulates the incremental perturbations, ultimately representing the complete adversarial perturbation. The counter `loop_i` tracks iterations to enforce the maximum iteration limit.

Main Iteration Loop

The algorithm iteratively refines the perturbation until the classification changes or the maximum iteration limit is reached. Each iteration performs a local linearization and takes a step toward the nearest decision boundary.

```
# Iterate until a successful perturbation is found or the limit is
reached
while loop_i < max_iter:
    x = pert_image.clone().requires_grad_(True)
    fs = net(x)
    # Current top prediction at x
```

```

k_i = fs.data.cpu().numpy().flatten().argsort()[::-1][0]

# Stop when the prediction changes
if k_i != label:
    break

# Initialize the best candidate step for this iteration
pert = float('inf')
w = None

```

At each iteration, the classifier is re-evaluated at the current perturbed image x . The gradient computation requires `requires_grad_(True)` to enable automatic differentiation. The expression `fs.data.cpu().numpy().flatten().argsort()[::-1][0]` converts the logits to a NumPy array, sorts the classes by score in descending order, and selects the top class. If this predicted class `k_i` differs from the original `label`, the attack has succeeded and the loop terminates.

The variables `pert` and `w` track the smallest distance ratio and its corresponding gradient direction for the current iteration. Setting `pert` to infinity ensures any valid candidate will be smaller, guaranteeing the first valid boundary distance replaces this placeholder.

Finding the Minimal Step Direction

For each iteration, the algorithm examines the top-scoring alternative classes to find which has the closest decision boundary. This search involves computing gradients for each candidate, measuring distances, and tracking the minimum.

Setting Up the Candidate Class Loop

```

# Search minimal step among candidate classes
for k in range(1, num_classes):
    if I[k] == label:
        continue

```

The loop starts at index 1 (not 0) because `I[0]` is the current predicted class, which we're trying to move away from. The variable `I` contains class indices sorted by descending score from the initial prediction. If `I[k]` matches the current `label`, we skip it since moving toward the current class makes no sense. The `continue` statement immediately advances to the next iteration without executing the gradient computations below.

Computing Gradient for Candidate Class

```
# Compute gradient for candidate class
if x.grad is not None:
    x.grad.zero_()
fs[0, I[k]].backward(retain_graph=True)
grad_k = x.grad.data.clone()
```

Before computing a new gradient, we zero any existing gradient accumulated in `x.grad` from previous operations. The `.backward()` call computes gradients of the candidate class score `fs[0, I[k]]` with respect to the input `x`. The expression `fs[0, I[k]]` accesses the logit for class `I[k]` in the single-item batch: index `0` selects the batch dimension, `I[k]` selects the class dimension. The parameter `retain_graph=True` preserves the computation graph for subsequent backward passes. Without this, PyTorch would free the graph memory, causing errors when computing `grad_label` next. The `.clone()` creates an independent copy, preventing it from being overwritten when we zero gradients again.

Computing Gradient for Original Class

```
# Compute gradient for original class
if x.grad is not None:
    x.grad.zero_()
fs[0, label].backward(retain_graph=True)
grad_label = x.grad.data.clone()
```

We repeat the gradient computation process for the original class. Zeroing `x.grad` removes the candidate class gradient computed above. The backward pass computes how the original class score changes with input modifications. The `grad_label` tensor points in the direction that maximally increases the original class confidence. By computing both `grad_k` and `grad_label`, we can find the direction that simultaneously increases the candidate class while decreasing the original class.

Computing Direction and Distance

```
# Direction and distance under linearization
w_k = grad_k - grad_label
f_k = (fs[0, I[k]] - fs[0, label]).data.cpu().numpy()
pert_k = abs(f_k) / (torch.norm(w_k.flatten()) + 1e-10)
```

The difference `w_k = grad_k - grad_label` creates a combined gradient that simultaneously increases the candidate class score while decreasing the original class

score. Think of this as a tug-of-war: `grad_k` pulls toward the candidate, `grad_label` pulls toward the original, and `w_k` is the net direction.

The score gap `f_k` represents the deficit to overcome. If the original class has score 8.5 and the candidate has score 3.2, then $f_k = 3.2 - 8.5 = -5.3$, a 5.3-point deficit. The ratio $\text{pert}_k = |f_k| / ||w_k||_2$ estimates distance to the linearized boundary. Let's work through a complete example. Suppose $|f_k| = 5.3$ (the deficit from above) and $||w_k||_2 = 2.0$ (the gradient magnitude). Then $\text{pert}_k = 5.3 / 2.0 = 2.65$. This means moving 2.65 units in direction `w_k` would reach the linearized decision boundary between these two classes.

What does the gradient magnitude $||w_k||_2 = 2.0$ tell us? It indicates how rapidly the score gap changes as we move in direction `w_k`. A larger magnitude (say 4.0) means the gap closes faster, requiring a shorter distance: $\text{pert}_k = 5.3 / 4.0 = 1.325$ units. A smaller magnitude (say 1.0) means the gap closes slowly, requiring more distance: $\text{pert}_k = 5.3 / 1.0 = 5.3$ units. The small constant `1e-10` prevents division by zero if the gradient happens to be zero, though this rarely occurs in practice with well-trained networks.

Tracking the Closest Boundary

```
if pert_k < pert:
    pert = pert_k
    w = w_k
```

After computing the distance ratio `pert_k` for this candidate class, we check if it's smaller than the current minimum `pert` (initialized to infinity before the loop). If yes, we update both `pert` to the new minimum distance and `w` to the corresponding gradient direction. By the end of the loop, `pert` holds the smallest distance among all candidates, and `w` points toward the closest decision boundary. This greedy selection ensures each iteration takes the minimal step needed to approach any boundary, not just a predetermined target class.

Computing and Applying the Perturbation

Once the closest boundary is identified, the algorithm computes the minimal step to reach it and applies this perturbation with overshoot to ensure the true boundary is crossed.

```
# Minimal step for the selected direction
r_i = (pert + 1e-4) * w / (torch.norm(w.flatten()) + 1e-10)
r_tot = r_tot + r_i

# Apply with overshoot to ensure crossing
pert_image = image + (1 + overshoot) * r_tot
loop_i += 1
```

```
return r_tot, loop_i, label, k_i, pert_image
```

The perturbation `r_i` is computed by normalizing the gradient direction `w` to unit length and scaling it by the distance `pert`. The small constant `1e-4` added to `pert` prevents numerical issues when the distance is very small, while `1e-10` in the denominator prevents division by zero. The perturbation is accumulated in `r_tot`, tracking the total change from the original image.

The multiplication by `(1 + overshoot)` applies the overshoot factor discussed earlier, ensuring the algorithm reliably crosses the true non-linear boundary rather than merely approaching it. The function returns five values: the total perturbation `r_tot` representing the complete adversarial noise pattern, the number of iterations `loop_i` performed before success or timeout, the original class label, the final adversarial class label `k_i`, and the complete perturbed image ready for evaluation.

Adaptive Behavior and Optimization

The algorithm adapts to the local geometry. In regions where the decision boundary is nearly linear, it might converge in just one or two iterations. Where the boundary curves significantly, it automatically takes more iterations, each adjusting the path to follow the boundary's contours. The accumulated perturbation `r_tot` represents the complete path from the original point to the adversarial example, approximating the minimal trajectory through high-dimensional space.

An important optimization consideration is the `num_classes` parameter. While we could theoretically consider all classes in the dataset, in practice, considering only the top few classes (based on the initial prediction scores) is often sufficient and significantly reduces computation time. The original paper suggests using 10 classes for datasets like ImageNet. For MNIST with only 10 total classes, we consider all of them, but for ImageNet with 1000 classes, limiting to the top 10 reduces gradient computations by 99% while maintaining attack effectiveness.

Demonstrating DeepFool

With the DeepFool algorithm fully implemented, we can execute end-to-end attacks that quantify the minimal perturbations needed to fool the classifier. How small can these perturbations be? The following demonstration measures this and exposes DeepFool's adaptive geometric strategy through concrete examples.

Single-Image Attack Pipeline

Theory predicts DeepFool will find minimal perturbations through iterative boundary refinement. Does it actually work? We need to demonstrate the complete attack cycle on real

data, measuring not just success but efficiency: iteration count, perturbation magnitude, and spatial distribution patterns. The demonstration will validate theoretical predictions against concrete results.

Establishing the Attack Environment

Our first requirement is a trained target model and a clean sample to attack. Without these, we cannot generate adversarial perturbations or measure their characteristics. The model must be in evaluation mode to disable dropout, ensuring stable gradients during DeepFool's iterative refinement. We'll use a single-sample batch for detailed per-example analysis rather than batch aggregation.

```
# Load trained model
model_path = 'output/mnist_model.pth'
if os.path.exists(model_path):
    model_data = load_model(model_path)
    model = model_data['model'].to(device)
    model.eval()
else:
    raise FileNotFoundError("Model not found.")

# Get single test sample
_, test_loader = get_mnist_loaders(batch_size=1, normalize=True)
dataiter = iter(test_loader)
image, true_label = next(dataiter)
image = image.to(device)

print(f"True label: {true_label.item()}")
```

The `model.eval()` call switches batch normalization to inference mode and disables dropout, which is necessary for DeepFool since gradient computations must be consistent across iterations. Dropout's random zeroing would create noisy, unstable gradients that violate the algorithm's assumption of smooth local linearity. The single-sample batch enables tracking exact iteration counts and perturbation evolution for this specific input rather than averaged batch statistics.

Executing the Minimal Perturbation Attack

How far does this sample sit from the nearest decision boundary? We need to establish the baseline classification and confidence, then let DeepFool navigate toward the boundary. The baseline confidence indicates whether the sample lies deep within its class region (high confidence, many iterations expected) or near boundaries (low confidence, fast

convergence). The attack execution returns the complete perturbation trajectory accumulated across all iterations.

```
# Baseline classification
with torch.no_grad():
    original_output = model(image)
    original_pred = original_output.argmax(dim=1).item()
    original_confidence = F.softmax(original_output, dim=1).max().item()

print(f"Original: class {original_pred} (confidence:
{original_confidence:.3f})")

# Execute DeepFool attack
r_total, iterations, orig_label, pert_label, pert_image = deepfool(
    image, model, num_classes=10, overshoot=0.02, max_iter=50,
    device=device
)

print(f"Attack: {orig_label} → {pert_label} in {iterations} iterations")
```

The `no_grad()` context disables gradient tracking for the baseline prediction since we're only reading the model's decision, not optimizing anything. The DeepFool call returns five values: `r_total` contains the complete accumulated perturbation vector, `iterations` indicates how many refinement steps were needed, `orig_label` and `pert_label` show the classification flip, and `pert_image` is the final adversarial example. Well-separated MNIST classes typically converge in 1-3 iterations, while ambiguous classes near decision boundaries may require 4-6 iterations. The 2% overshoot parameter adds $0.02 * \text{perturbation}$ to ensure we actually cross the true non-linear boundary rather than just touching the linearized approximation, compensating for curvature the linear model cannot capture.

Quantifying Attack Efficiency and Perturbation Properties

DeepFool claims to find minimal perturbations, but what does "minimal" actually mean in practice? We need multiple metrics to validate this claim. The

L_2

norm quantifies total perturbation energy (what DeepFool optimizes),

while

L_∞

captures maximum pixel-wise change (what humans might notice). Comparing these exposes whether the perturbation spreads uniformly or concentrates

on key features. The relative perturbation enables fair comparison across different image magnitudes, and the adversarial confidence indicates whether we barely crossed the boundary (confirming minimality) or penetrated deep into another class region (suggesting excess perturbation).

```
# Compute perturbation norms
perturbation_norm_l2 = torch.norm(r_total).item()
perturbation_norm_linf = torch.abs(r_total).max().item()
relative_perturbation = perturbation_norm_l2 / torch.norm(image).item()

# Evaluate adversarial confidence
with torch.no_grad():
    adv_output = model(pert_image)
    adv_confidence = F.softmax(adv_output, dim=1).max().item()

# Display results
print(f"\n=== Attack Results ===")
print(f"L2 norm: {perturbation_norm_l2:.4f}")
print(f"L∞ norm: {perturbation_norm_linf:.4f}")
print(f"Relative perturbation: {relative_perturbation:.2%}")
print(f"Original confidence: {original_confidence:.3f}")
print(f"Adversarial confidence: {adv_confidence:.3f}")
```

The

L_2/L_∞ ratio demonstrates spatial concentration strategy. In our example,

$L_2=7.72$ and $L_\infty=1.58$.

The maximum single-pixel change is 1.58, while total perturbation energy is 7.72. If the perturbation were perfectly uniform across all 784 pixels, we'd expect

and

$L_\infty \approx L_2/784 \approx 7.72/784 \approx 0.0098$.

The maximum single-pixel change is 1.58, while total perturbation energy is 7.72. If the perturbation were perfectly uniform across all 784 pixels, we'd expect

$L_\infty \approx L_2/784 \approx 7.72/784 \approx 0.0098$.

The observed 1.58 is nearly 6x larger than this uniform baseline,

proving DeepFool concentrates modifications on strategically important pixels rather than distributing them uniformly. The relative perturbation formula

$$\frac{\|r\|_2 / \|x\|_2}{\|r\|_\infty / \|x\|_\infty}$$

normalizes by image magnitude: a bright image with

$\|x\|_2=20$ and $\|x\|_\infty=20$

and a dark image with

$\|x\|_2=20$ and $\|x\|_\infty=20$

and a dark image with

$\|x\|_2=20$ and $\|x\|_\infty=20$

and a dark image with

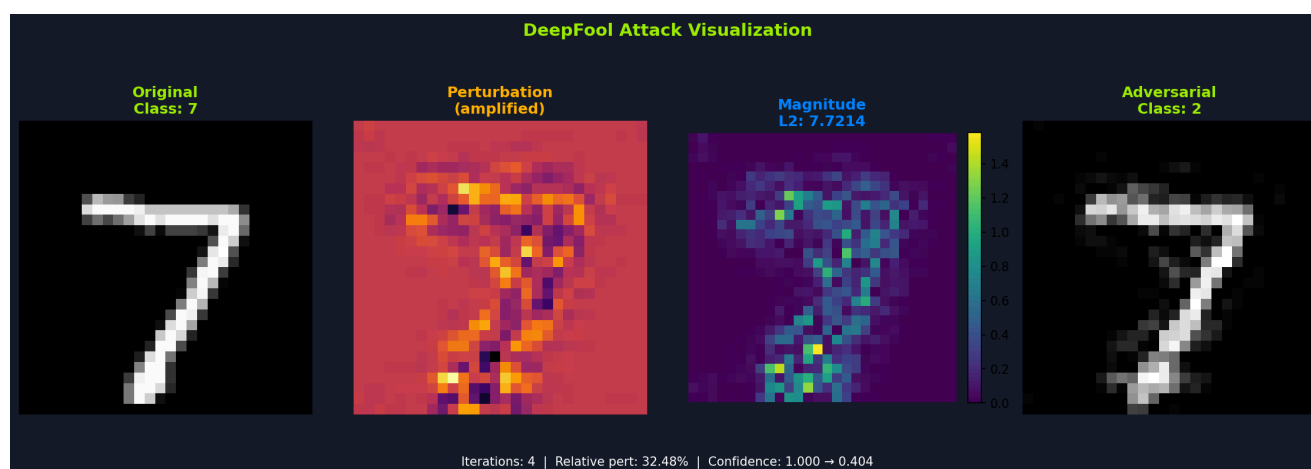
$$\|x\|_2 = 10 \setminus \|x\|_2 = 10$$

both showing 15% relative perturbation experienced proportionally equivalent modifications despite different absolute perturbation magnitudes. This digit '7' example shows 32.48% relative perturbation, which is higher than typical due to the significant structural change required to flip it to '2'.

The adversarial confidence drop from 1.000 to 0.404 confirms minimal boundary crossing with slight overshoot. The model went from absolute certainty to below the 0.5 decision threshold. The confidence settling at 0.404 indicates the attack crossed the boundary and penetrated slightly into the '2' class region, as expected with the 2% overshoot parameter. This validates that DeepFool's iterative refinement successfully placed the adversarial example just beyond the decision surface, ensuring reliable misclassification while maintaining near-minimal perturbation magnitude.

Exposing Spatial Attack Patterns

Metrics quantify perturbation magnitude, but where does DeepFool actually modify pixels? Theory suggests optimal geometric paths should target class-discriminative features along digit boundaries, not scatter noise uniformly. We need visual evidence. The challenge: minimal perturbations have such small magnitudes they're invisible when displayed at natural scale. We'll build a four-panel visualization showing the original image, an amplified perturbation pattern (normalized to full dynamic range for visibility), a magnitude heatmap showing intensity distribution, and the final adversarial result. This spatial analysis distinguishes DeepFool's targeted strategy from FGSM's uniform approach.



```
# Prepare images for visualization
original_img = mnist_denormalize(image.squeeze()).cpu().numpy()
adversarial_img = mnist_denormalize(pert_image.squeeze()).cpu().numpy()
perturbation = r_total.cpu().squeeze().numpy()

# Normalize perturbation for visibility (amplify minimal changes)
pert_display = perturbation - perturbation.min()
```

```

if pert_display.max() > 0:
    pert_display = pert_display / pert_display.max()

# Create four-panel visualization
fig, axes = plt.subplots(1, 4, figsize=(15, 5))
fig.patch.set_facecolor(NODE_BLACK)

for ax in axes:
    ax.set_facecolor(NODE_BLACK)
    for spine in ax.spines.values():
        spine.set_edgecolor(HACKER_GREY)

# Panel 1: Original clean image
axes[0].imshow(original_img, cmap='gray', vmin=0, vmax=1)
axes[0].set_title(f"Original\nClass: {original_pred}",
                  color=HTB_GREEN, fontweight='bold')
axes[0].axis('off')

# Panel 2: Amplified perturbation pattern
axes[1].imshow(pert_display, cmap='inferno')
axes[1].set_title("Perturbation\n(amplified)",
                  color=NUGGET_YELLOW, fontweight='bold')
axes[1].axis('off')

# Panel 3: Perturbation magnitude heatmap
im = axes[2].imshow(np.abs(perturbation), cmap='viridis')
axes[2].set_title(f"Magnitude\nL2: {perturbation_norm_l2:.4f}",
                  color=AZURE, fontweight='bold')
axes[2].axis('off')
plt.colorbar(im, ax=axes[2], fraction=0.046, pad=0.04)

# Panel 4: Adversarial result
title_color = HTB_GREEN if pert_label != original_pred else MALWARE_RED
axes[3].imshow(adversarial_img, cmap='gray', vmin=0, vmax=1)
axes[3].set_title(f"Adversarial\nClass: {pert_label}",
                  color=title_color, fontweight='bold')
axes[3].axis('off')

# Summary metrics
metrics_text = (
    f"Iterations: {iterations} | "
    f"Relative pert: {relative_perturbation:.2%} | "
    f"Confidence: {original_confidence:.3f} → {adv_confidence:.3f}"

```

```

)
fig.text(0.5, 0.02, metrics_text, ha='center', fontsize=10, color=WHITE)

plt.suptitle("DeepFool Attack Visualization", fontsize=14,
            color=HTB_GREEN, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

```

The normalization operation $(\text{perturbation} - \text{min}) / (\text{max} - \text{min})$ rescales the perturbation from its actual range (typically $[-0.5, 0.5]$ for MNIST) to $[0, 1]$ for visualization. Without this, the minimal perturbations would appear as uniform gray since matplotlib's colormap can't distinguish small variations. The `inferno` colormap maps 0 (minimal perturbation) to dark purple and 1 (maximum perturbation) to bright yellow, making spatial patterns visible. The `viridis` colormap for absolute magnitude uses the same principle but with green-yellow coloring better suited for intensity data.

Panel 2 demonstrates DeepFool's bidirectional strategy through color distribution. For our digit '7' to '2' attack, bright yellow clusters appear along the top and middle sections where the '7' stroke needs modification to resemble a '2'. Dark purple dominates background regions, showing minimal modification to irrelevant pixels. The asymmetry in color intensity (more brightening than darkening) reflects specific decision boundary geometry: transforming '7' into '2' requires adding features (the '2' curve) more than removing them. FGSM would show uniform yellow intensity across modified pixels regardless of importance. DeepFool's concentration pattern confirms the iterative gradient refinement successfully identified which pixels matter most for classification, allocating perturbation budget accordingly.

Batch Attack Generation

The single-image demonstration showed DeepFool's ability to find minimal perturbations. One sample. One perturbation. One data point. What patterns emerge when attacking multiple samples? Does perturbation size vary by digit class? Which samples resist attacks most strongly, and which crumble immediately? Batch analysis across diverse inputs exposes statistical patterns completely invisible in single examples, transforming anecdotal observations into quantitative measurements.

Systematic Sample Processing

Single-sample attacks provide proof of concept, but statistical patterns require population-level analysis. Does DeepFool consistently converge in similar iteration counts? Do all digit classes exhibit comparable robustness? Which decision boundaries require larger perturbations? We need to attack multiple diverse samples while tracking metrics for each:

perturbation norms, iteration counts, success indicators, and classification trajectories. The batch processing infrastructure will generate this dataset for statistical analysis.

Establishing the Batch Attack Pipeline

Our goal is processing many samples efficiently while preserving per-sample metrics. Why not use larger batches? DeepFool's iterative nature makes each sample independent: one may converge in 2 iterations while another requires 6. Batch processing would force all samples to wait for the slowest, wasting computation. We'll process 20 samples individually to balance statistical insight with runtime, accumulating results in a structured format for downstream analysis.

```
num_examples = 20
print(f"\nGenerating {num_examples} adversarial examples using
DeepFool...")

_, test_loader = get_mnist_loaders(batch_size=1, normalize=True)
model.eval()

results = []
success_count = 0

print(f"Test loader ready with {len(test_loader.dataset)} samples")
print(f"Will process first {num_examples} samples")
print("Starting batch attack generation...")
```

The single-sample batches enable independent processing where each attack terminates immediately upon convergence without waiting for others. The `results` list accumulates dictionaries containing both tensor data (images, perturbations) and scalar metrics (norms, iteration counts). The `success_count` tracks classification flips separately for quick summary statistics. Processing 20 samples takes roughly 1-2 seconds on GPU (50-100ms per sample), sufficient to expose patterns without extensive wait times.

Executing Attacks and Collecting Metrics

Each sample must be attacked independently, with metrics captured for statistical analysis. We need to store not just success/failure, but the complete trajectory: how many iterations were required, what perturbation magnitude was needed, whether the original prediction was correct, and what the adversarial label became. This rich dataset enables answering questions about convergence patterns, robustness variation across classes, and boundary geometry characteristics. The processing loop must also provide real-time feedback so we can monitor attack effectiveness as samples are processed.

```

for idx, (data, target) in enumerate(test_loader):
    if idx >= num_examples:
        break

    data = data.to(device)

    # Execute DeepFool attack
    r, iterations, orig_label, adv_label, pert_image = deepfool(
        data, model, num_classes=10, overshoot=0.02, max_iter=50,
device=device
    )

    # Track success and store metrics
    success = (orig_label != adv_label)
    if success:
        success_count += 1

    results.append({
        'original_image': data.cpu(),
        'perturbation': r.cpu(),
        'perturbed_image': pert_image.cpu(),
        'original_label': orig_label,
        'adversarial_label': adv_label,
        'iterations': iterations,
        'true_label': target.item(),
        'l2_norm': torch.norm(r.cpu()).item(),
        'success': success
    })

    # Progress feedback
    print(f" Example {idx+1}: True={target.item()}, Orig={orig_label}, "
          f"Adv={adv_label}, Iter={iterations}, L2=
{torch.norm(r.cpu()).item():.4f}")

```

The results dictionary captures nine distinct measurements per sample. Tensor data (`original_image` , `perturbation` , `perturbed_image`) gets transferred to CPU immediately via `.cpu()` to free GPU memory, preventing accumulation that would cause out-of-memory errors after processing dozens of samples. The `L2L_2` norm computation `torch.norm(r.cpu()).item()` measures total

perturbation energy, with `.item()` converting from a single-element tensor to a Python float for storage. The progress print exposes patterns as they emerge: some samples flip with iterations of 1-2 and

L2L_2

norms around 2-3 (well-separated classes), while others require 5-6 iterations with

L2L_2

norms of 8-10 (ambiguous boundaries).

Summarizing Attack Statistics

What does the population-level data show? We need aggregate statistics quantifying overall attack success, average perturbation magnitude, and typical convergence speed. These summary metrics enable comparing DeepFool's behavior across different models, architectures, or datasets. The success rate indicates reliability, average

L2L_2

norm quantifies typical robustness, and average iteration count measures convergence efficiency.

```
print(f"\nAttack Success Rate: {success_count}/{num_examples} "
      f"({100*success_count/num_examples:.1f}%)")
print(f"Average L2 norm: {np.mean([r['l2_norm'] for r in results]):.4f}")
print(f"Average iterations: {np.mean([r['iterations'] for r in
results]):.1f}")
```

Expected output:

```
Attack Success Rate: 20/20 (100.0%)
Average L2 norm: 4.8661
Average iterations: 3.0
```

The 100% success rate confirms DeepFool's reliability on well-separated MNIST classes. The average

L2L_2

norm of 4.87 represents typical perturbation magnitude across diverse digit samples. The 3.0 average iterations indicate most samples converge in 2-4 steps, with variation between fast convergers (1 iteration for well-separated classes like '3→5') and slower ones (5 iterations for structurally similar classes like '4→7'). Individual variation spans

from
L2L_2
= 0.60 (digit '3' to '5', structurally similar) to
L2L_2
= 7.72 (digit '7' to '2', requiring significant structural changes).
This 13x range demonstrates how decision boundary geometry varies widely across different regions of the input space, with some digit pairs separated by narrow gaps while others require substantial modifications.

Visualizing Population-Level Attack Patterns

Statistics quantify attack effectiveness, but visual inspection demonstrates the human-perceptibility of minimal perturbations. Do adversarial examples look obviously corrupted, or do the changes remain subtle enough to avoid casual detection? We need a grid comparing multiple original-adversarial pairs side-by-side, enabling visual assessment of perturbation subtlety across different digit classes. The visualization should highlight successful attacks (classification changed) versus failures (classification unchanged), using our normal themeing. This provides qualitative validation of how minimal DeepFool's perturbations actually are.

```
def visualize_attack_grid(results, save_dir='output'):
    """
    Create grid visualization showing original and adversarial images
    side-by-side.

    Args:
        results (list): Attack results from batch generation
        save_dir (str): Directory to save visualization
    """
    print("\nGenerating attack grid visualization...")

    num_examples = min(10, len(results))
    fig, axes = plt.subplots(4, 5, figsize=(15, 12))
    fig.patch.set_facecolor(NODE_BLACK)

    for ax in axes.flatten():
        ax.set_facecolor(NODE_BLACK)
        for spine in ax.spines.values():
            spine.set_edgecolor(HACKER_GREY)

    for idx in range(num_examples):
        row = idx // 5
```

```

col = idx % 5

# Original image (top row for this column)
ax_original = axes[row * 2, col]
img = mnist_denormalize(results[idx]
['original_image'].squeeze()).numpy()
ax_original.imshow(img, cmap='gray', vmin=0, vmax=1)
ax_original.set_title(f"Original: {results[idx]
['original_label']}",
                      color=HTB_GREEN, fontsize=10)
ax_original.axis('off')

# Adversarial image (bottom row for this column)
ax_adv = axes[row * 2 + 1, col]
adv_img = mnist_denormalize(results[idx]
['perturbed_image'].squeeze()).numpy()
ax_adv.imshow(adv_img, cmap='gray', vmin=0, vmax=1)

title_color = MALWARE_RED if results[idx]['success'] else
HACKER_GREY
ax_adv.set_title(f"Adversarial: {results[idx]
['adversarial_label']}",
                 color=title_color, fontsize=10)
ax_adv.axis('off')

plt.suptitle('DeepFool Attack: Original vs Adversarial Examples',
             color=HTB_GREEN, fontsize=16, y=0.98)
plt.tight_layout()
plt.savefig(os.path.join(save_dir, 'deepfool_examples.png'),
            facecolor=NODE_BLACK, dpi=150, bbox_inches='tight')
plt.close()

print(f"Grid visualization saved to {save_dir}/deepfool_examples.png")

# Generate the grid visualization
visualize_attack_grid(results, save_dir='output')

```



The grid layout uses integer division and modulo arithmetic for positioning: $\text{row} = \text{idx} // 5$ determines which pair of rows (0 or 1, since each pair shows original/adversarial), while $\text{col} = \text{idx} \% 5$ determines the column position (0-4). Each original image occupies `axes[row * 2, col]`, and its adversarial counterpart sits directly below at `axes[row * 2 + 1, col]`. This pairing enables direct visual comparison per sample. Red titles (`MALWARE_RED`) mark successful classification flips, while grey titles indicate failures (rare on MNIST). The paired images demonstrate DeepFool's minimality: on MNIST's simple grayscale digits, careful inspection may reveal subtle differences, but they are only particularly obvious given the high contrast nature of the MNIST dataset (white on black). Examples like '7' flipped to '2', '9' to '4', and '5' to '6' maintain recognizable digit structure while completely fooling the classifier. On complex natural images (like photographs), such minimal perturbations would be nearly impossible to detect with the human eye.

Perturbation Analysis and Metrics

Having generated batch adversarial examples, we now analyze their characteristics through spatial perturbation patterns and statistics. Where does DeepFool concentrate modifications? How do perturbation magnitudes distribute across samples? Which digit classes prove most vulnerable? These visualizations and statistics expose the geometric properties of minimal adversarial perturbations.

Spatial Perturbation Analysis

Theory predicts DeepFool will concentrate modifications along decision boundaries where class-discriminative features reside. Does empirical evidence support this? We need spatial heatmaps showing where DeepFool actually modifies pixels across multiple samples. The challenge: raw perturbation magnitudes are minimal (often $[-0.5, 0.5]$), making patterns invisible without amplification. We'll create two visualization layers: raw perturbation heatmaps using diverging colormaps (red for positive changes, blue for negative), and amplified difference visualizations (10x magnification) overlaid to expose subtle modifications. This dual approach validates whether DeepFool targets semantically meaningful regions or scatters noise uniformly.

```
def visualize_perturbation_analysis(results, save_dir='output'):
    """
    Analyze and visualize perturbation characteristics across samples.

    Creates two-row visualization: top shows raw perturbation heatmaps,
    bottom shows amplified differences overlaid on originals.

    Args:
        results (list): Attack results
        save_dir (str): Output directory
    """
    print("\nGenerating perturbation analysis...")

    fig, axes = plt.subplots(2, 3, figsize=(15, 10))
    fig.patch.set_facecolor(NODE_BLACK)

    for ax in axes.flatten():
        ax.set_facecolor(NODE_BLACK)
        for spine in ax.spines.values():
            spine.set_edgecolor(HACKER_GREY)

    # Select first 3 successful attacks
    successful_attacks = [r for r in results if r['success']][:3]

    for idx, result in enumerate(successful_attacks):
        # Top row: Raw perturbation heatmap
        ax_top = axes[0, idx]
        pert = result['perturbation'].squeeze().numpy()
        vmax = np.abs(pert).max() or 1e-6
        im_top = ax_top.imshow(pert, cmap='RdBu_r', vmin=-vmax, vmax=vmax)
        ax_top.set_title(f'Perturbation (L2={result["l2_norm"]:.3f})',
```

```

        color=HTB_GREEN, fontsize=10)
    ax_top.axis('off')

    cbar_top = plt.colorbar(im_top, ax=ax_top, fraction=0.046,
pad=0.04)
    cbar_top.outline.set_edgecolor(HACKER_GREY)
    cbar_top.ax.tick_params(colors=WHITE)

    # Bottom row: Amplified difference visualization
    ax_bottom = axes[1, idx]
    orig_img = result['original_image'].squeeze().numpy()
    adv_img = result['perturbed_image'].squeeze().detach().numpy()
    diff_amplified = (adv_img - orig_img) * 10 # 10x amplification
    for visibility

    im_bottom = ax_bottom.imshow(diff_amplified, cmap='RdBu_r',
vmin=-0.5, vmax=0.5)
    ax_bottom.set_title(f"{result['original_label']} →
{result['adversarial_label']} "
                        f"({result['iterations']} iters)",
                        color=NUGGET_YELLOW, fontsize=10)
    ax_bottom.axis('off')

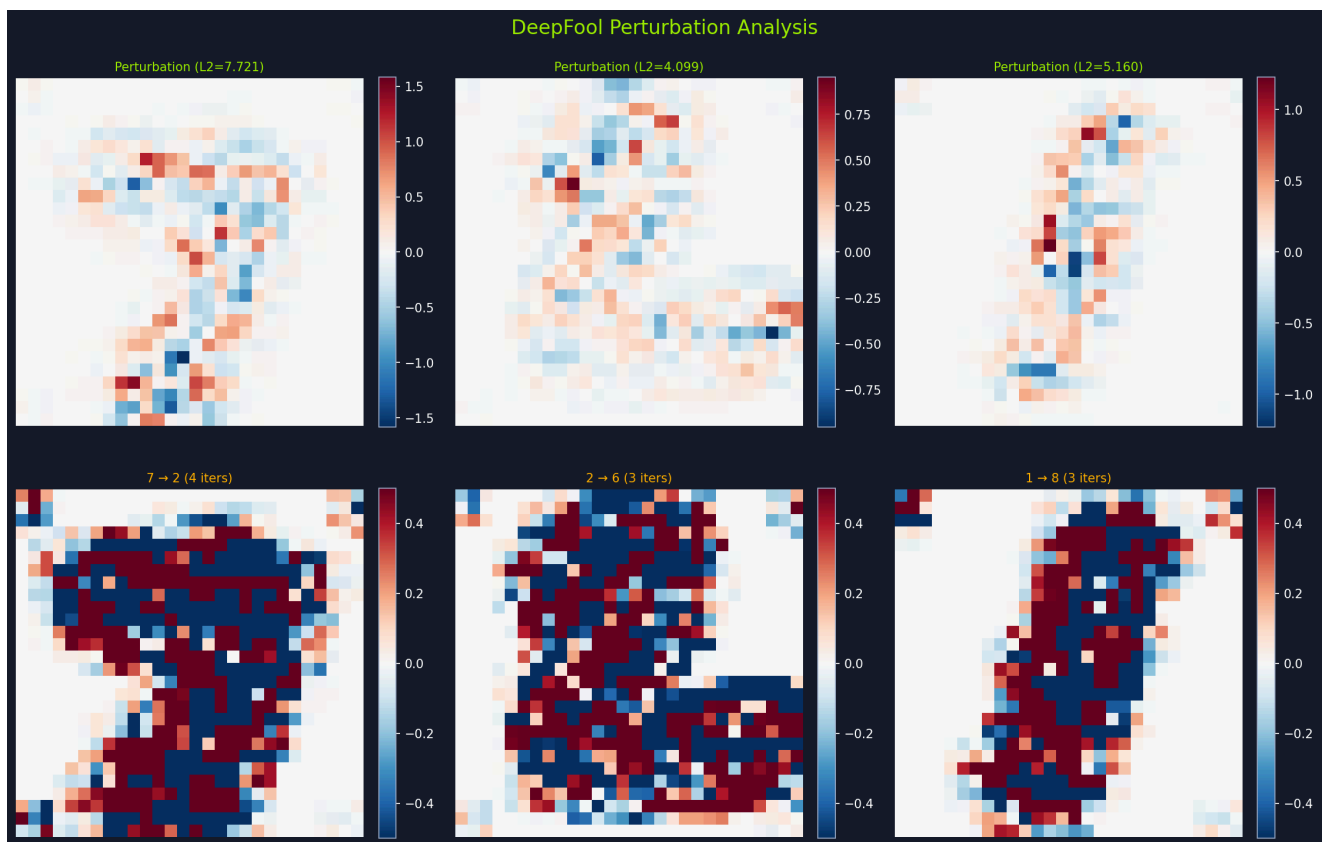
    cbar_bottom = plt.colorbar(im_bottom, ax=ax_bottom,
fraction=0.046, pad=0.04)
    cbar_bottom.outline.set_edgecolor(HACKER_GREY)
    cbar_bottom.ax.tick_params(colors=WHITE)

    plt.suptitle('DeepFool Perturbation Analysis', color=HTB_GREEN,
fontsize=16, y=0.98)
    plt.tight_layout()
    plt.savefig(os.path.join(save_dir, 'deepfool_perturbations.png'),
                facecolor=NODE_BLACK, dpi=150, bbox_inches='tight')
    plt.close()

    print(f"Perturbation analysis saved to
{save_dir}/deepfool_perturbations.png")

# Generate the perturbation analysis visualization
visualize_perturbation_analysis(results, save_dir='output')

```

The RdBu_r colormap (reversed Red-Blue) creates intuitive diverging visualization: red shows positive perturbations (brightening pixels), blue shows negative perturbations (darkening), and white indicates no change. The symmetric value range `vmin=-vmax, vmax=vmax` centers white at zero, ensuring neutral pixels remain uncolored. The amplified difference calculation $(adv_img - orig_img) * 10$ magnifies subtle modifications 10x: a 0.03 pixel change becomes 0.3, visually detectable after colormap application. Without amplification, changes of 0.01-0.05 appear uniform grey.

The heatmaps show concentrated modifications along digit boundaries and distinctive features. The visualization shows three successful attacks, with the first being our '7' flipped to '2' attack (L2L_2=7.72).

For this attack, perturbations concentrate along the digit stroke, with modifications targeting the areas where '7' structure must transform into '2' shape. Background regions remain white (unchanged), confirming DeepFool targets semantically meaningful areas where small changes maximally impact classification, not uniform noise scattering. The second and third panels show '2'→'6' (L2L_2=4.10) and '1'→'8' (L2L_2=5.16) transitions, each displaying concentrated modifications at class-discriminative boundaries.

Statistical Distribution Analysis

Spatial heatmaps show local targeting, but population-level statistics quantify global patterns. What's the typical perturbation magnitude across samples? How many iterations does DeepFool usually require? Which digit classes prove most vulnerable? We need three statistical visualizations:

L2L_2

norm distribution (showing perturbation magnitude clustering), iteration count distribution (measuring convergence efficiency), and per-class success rates (identifying vulnerable digits). These metrics enable comparing DeepFool's behavior across models, datasets, or attack configurations.

```
print("\nGenerating attack metrics visualization...")

# Setup three-panel figure
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
fig.patch.set_facecolor(NODE_BLACK)

for ax in axes:
    ax.set_facecolor(NODE_BLACK)
    for spine in ax.spines.values():
        spine.set_edgecolor(HACKER_GREY)
    ax.tick_params(colors=WHITE)
    ax.grid(True, alpha=0.3, color=HACKER_GREY, linestyle='--')

# Panel 1: L2 Norm Distribution
l2_norms = [r['l2_norm'] for r in results]
axes[0].hist(l2_norms, bins=15, color=HTB_GREEN, alpha=0.7,
             edgecolor=HACKER_GREY)
axes[0].set_xlabel('L2 Norm', color=WHITE)
axes[0].set_ylabel('Frequency', color=WHITE)
axes[0].set_title('Perturbation Magnitude Distribution', color=HTB_GREEN)
print(f"L2 norm range: [{min(l2_norms):.4f}, {max(l2_norms):.4f}]")

# Panel 2: Iteration Count Distribution
iterations = [r['iterations'] for r in results]
axes[1].hist(iterations, bins=range(1, max(iterations)+2),
             color=AZURE, alpha=0.7, edgecolor=HACKER_GREY)
axes[1].set_xlabel('Iterations', color=WHITE)
axes[1].set_ylabel('Frequency', color=WHITE)
axes[1].set_title('Iterations Required', color=HTB_GREEN)
print(f"Iteration range: [{min(iterations)}, {max(iterations)}]")
```

```

# Panel 3: Per-Class Success Rates
class_success = {}
for r in results:
    orig = r['original_label']
    if orig not in class_success:
        class_success[orig] = {'total': 0, 'success': 0}
    class_success[orig]['total'] += 1
    if r['success']:
        class_success[orig]['success'] += 1

classes = sorted(class_success.keys())
success_rates = [
    class_success[c]['success'] / class_success[c]['total'] * 100
    if class_success[c]['total'] > 0 else 0
    for c in classes
]

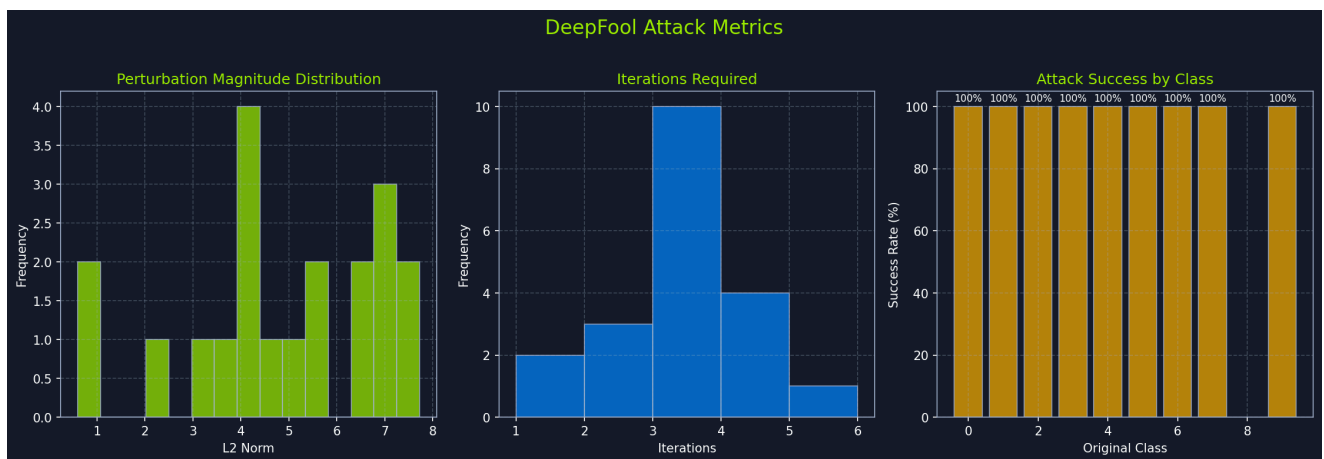
bars = axes[2].bar(classes, success_rates, color=NUGGET_YELLOW,
                  alpha=0.7, edgecolor=HACKER_GREY)
axes[2].set_xlabel('Original Class', color=WHITE)
axes[2].set_ylabel('Success Rate (%)', color=WHITE)
axes[2].set_title('Attack Success by Class', color=HTB_GREEN)
axes[2].set_ylim(0, 105)

# Add percentage labels on bars
for bar, rate in zip(bars, success_rates):
    height = bar.get_height()
    ax_x = bar.get_x() + bar.get_width() / 2.0
    axes[2].text(ax_x, height + 1, f'{rate:.0f}%',
                 ha='center', va='bottom', color=WHITE, fontsize=8)

# Save visualization
plt.suptitle('DeepFool Attack Metrics', color=HTB_GREEN, fontsize=16,
y=1.02)
plt.tight_layout()
plt.savefig('output/deepfool_metrics.png',
          facecolor=NODE_BLACK, dpi=150, bbox_inches='tight')
plt.close()

print("Metrics visualization saved to output/deepfool_metrics.png")

```



The

L2L₂

norm histogram with 15 bins shows distribution shape across the range [0.60, 7.72]. Most perturbations cluster around the 4-5 average, with the minimum at 0.60 representing the easiest boundary crossing (digit '3→5', structurally similar) and maximum at 7.72 representing the most challenging transformation (digit '7→2', requiring substantial structural changes). The `alpha=0.7` transparency creates softer visual appearance while maintaining distinct bar boundaries via `edgecolor`. The iteration histogram uses integer bins `range(1, max(iterations)+2)` ensuring each count gets its own bar. The `+2` compensates for range's exclusive endpoint. The distribution shows iterations from 1 to 5, with most samples converging in 2-4 iterations (averaging exactly 3.0), demonstrating efficient convergence across diverse digit pairs.

The per-class success computation uses a two-counter dictionary: `total` tracks samples per class, `success` counts classification flips. The percentage formula `success / total * 100` converts to success rate, with zero-division protection for classes with no samples. The bar chart text labels use `bar.get_x() + bar.get_width() / 2.0` for horizontal centering and `height + 1` for vertical positioning 1% above bar tops. The `ylim(0, 105)` provides 5% headroom preventing label clipping. Per-class rates indicate which digits (e.g., '1' vs '8') prove more vulnerable to minimal perturbations based on decision boundary geometry.

Aggregate Attack Summary

Visualizations show patterns, but summary statistics provide quantitative benchmarks for comparing DeepFool across different models or datasets. We need consolidated metrics answering: what's the overall success rate, what's the average perturbation magnitude, what's typical convergence speed, and which class transitions occur most frequently? These aggregate statistics enable reproducible comparisons and identify systematic misclassification patterns (e.g., '7→9' transitions might occur more often than '0→1' if those digit pairs have closer decision boundaries).

```

def print_summary_statistics(results):
    """
    Print summary statistics for attack results.

    Computes and displays success rate, perturbation statistics, iteration
    statistics, and common class transitions.

    Args:
        results (list): Attack results
    """
    print("\n" + "="*60)
    print("Attack Summary Statistics")
    print("="*60)

    successful_attacks = [r for r in results if r['success']]

    if successful_attacks:
        avg_l2 = np.mean([r['l2_norm'] for r in successful_attacks])
        avg_iterations = np.mean([r['iterations'] for r in
successful_attacks])
        min_l2 = min([r['l2_norm'] for r in successful_attacks])
        max_l2 = max([r['l2_norm'] for r in successful_attacks])

        print(f"Success Rate: {len(successful_attacks)}/{len(results)} "
              f"({100*len(successful_attacks)/len(results):.1f}%)")
        print(f"Average L2 Norm: {avg_l2:.4f}")
        print(f"L2 Range: [{min_l2:.4f}, {max_l2:.4f}]")
        print(f"Average Iterations: {avg_iterations:.1f}")

        # Class transition analysis
        transitions = {}
        for r in successful_attacks:
            key = f"{r['original_label']}->{r['adversarial_label']}"
            transitions[key] = transitions.get(key, 0) + 1

        print(f"\nMost Common Misclassifications:")
        for trans, count in sorted(transitions.items(), key=lambda x:
x[1], reverse=True)[:5]:
            print(f"  {trans}: {count} times")
        else:
            print("No successful attacks generated")

    print("="*60)

```

```
# Generate summary
print_summary_statistics(results)
```

Expected output:

```
=====
Attack Summary Statistics
=====
Success Rate: 20/20 (100.0%)
Average L2 Norm: 4.8661
L2 Range: [0.5960, 7.7214]
Average Iterations: 3.0

Most Common Misclassifications:
  9→4: 3 times
  7→2: 1 times
  2→6: 1 times
  1→8: 1 times
  0→6: 1 times
=====
```

The transition frequency analysis uses dictionary accumulation: `transitions[key] = transitions.get(key, 0) + 1` increments counts for each original→adversarial pairing. The sorting `sorted(transitions.items(), key=lambda x: x[1], reverse=True)[:5]` ranks by frequency (second tuple element `x[1]`) in descending order, selecting top 5. The most common transition '9→4' appears 3 times, indicating that these digit pairs have a particularly accessible decision boundary in the learned feature space. This makes geometric sense: both '9' and '4' have similar upper loops and vertical strokes, requiring smaller perturbations to bridge the gap compared to very different shapes like '0→1'.

DeepFool Challenge

Objective

Your task is to craft a targeted adversarial example using a DeepFool-style iterative attack. Unlike untargeted attacks that simply cause any misclassification, this challenge requires you to fool the classifier into predicting a specific target class while minimizing the perturbation under an

ℓ_2 norm

distance constraint.

The constraint is

$\|x_{adv} - x_{\text{adv}}\|_2 \leq \text{threshold}$ and $x_{\text{adv}} \cdot x_{\text{adv}_2} \leq \text{threshold}$,

where

x

is the baseline image and

x_{adv} is your crafted adversarial example. Both conditions must hold: the

predicted class must equal the specified target label (not just any misclassification), and the

ℓ_2 norm

distance must be at most the threshold.

Quick Start

All API endpoints expect images in $[0, 1]$ pixel space, not normalized tensors.

Start the instance from the platform. Check readiness:

```
export BASE_URL="http://instance_ip:port"
curl -s "$BASE_URL/health"
```

API

The API is minimal and deterministic. Each response is JSON. Images are base64-encoded PNG files of shape 28×28 and single channel in $[0, 1]$ after decoding.

GET /health

Returns service status and configuration parameters.

```
curl -s "$BASE_URL/health" | jq
```

Example response:

```
{
  "status": "ok",
  "l2_threshold": 0.75,
  "index": 95,
  "target": 6
}
```

```
}
```

The `l2_threshold` defines the maximum L_2 distance allowed. The `index` identifies the MNIST test sample. The `target` specifies the required target class for the adversarial example.

GET /challenge

Returns the fixed sample and the constraint that must be satisfied.

```
curl -s "$BASE_URL/challenge" | jq
```

Example response shape:

```
{
  "sample_index": 95,
  "label": 4,
  "target": 6,
  "l2_threshold": 0.75,
  "image_b64": "<base64 PNG>"
}
```

The `sample_index` identifies the specific MNIST test sample used for this challenge. The `label` is the ground-truth class of the baseline image. The `target` is the required predicted class for a successful adversarial example (targeted attack). The `l2_threshold` defines the maximum ell_2 distance allowed in $[0,1]$ pixel space. The `image_b64` field contains the baseline image as a base64-encoded PNG.

POST /predict

Returns the model's prediction for an arbitrary image. Use this endpoint to validate the pipeline, encoding, and intermediate candidates. It does not return the flag.

```
curl -s -X POST "$BASE_URL/predict" \
  -H 'content-type: application/json' \
  -d '{"image_b64": "<base64 PNG of 28x28 grayscale>"}' | jq
```


Example response shape:

```
{
  "pred": 8,
  "confidence": 0.73
}
```

The `pred` field contains the predicted class (integer in `[0,9]`). The `confidence` field provides the softmax probability for the predicted class (float in `[0,1]`).

GET /weights

Returns the pretrained model `state_dict` as a binary file. Load it on CPU and use the provided classifier architecture to compute input gradients locally.

```
curl -s -o deepfool_weights.pth "$BASE_URL/weights"
```

The downloaded file is a PyTorch state dictionary compatible with the `SimpleClassifier` architecture detailed below. Load with `torch.load(path, map_location=torch.device('cpu'))`.

POST /submit

Validates that the candidate is a proper `28x28` single-channel image in `[0,1]`, that the ℓ_2 distance to the baseline image is at most `l2_threshold`, and that the classifier's predicted class matches the target label. Returns the flag on success.

```
curl -s -X POST "$BASE_URL/submit" \
  -H 'content-type: application/json' \
  -d '{"image_b64": "<base64 PNG of 28x28 grayscale>"}' | jq
```

Example success response shape:

```
{
  "ok": true,
  "pred": 6,
  "target": 6,
}
```

```
"l2": 0.68,  
"flag": "HTB{...}"  
}
```

On validation failure, the endpoint returns HTTP 400 with a descriptive error message (e.g., "L2 too large: 0.82 > 0.75" or "Wrong target: predicted 8, need 6").

Minimal Python Scaffolds

The following scaffolds provide API helpers, model architecture, and integration testing. They do not implement the attack.

API Helpers and Challenge Fetching

```
import os, io, base64, numpy as np, requests, torch, torch.nn as nn  
from PIL import Image
```

```
BASE_URL = os.getenv("BASE_URL", "http://127.0.0.1:8000")  
MNIST_MEAN, MNIST_STD = 0.1307, 0.3081
```

```
def x01_from_b64_png(b64: str) -> np.ndarray:  
    """Convert base64 PNG to [0,1] numpy array.  
  
    Args:  
        b64: Base64 encoded PNG string  
  
    Returns:  
        np.ndarray: Image as (28, 28) array in [0,1] range  
    """  
    raw = base64.b64decode(b64)  
    img = Image.open(io.BytesIO(raw)).convert("L")  
    if img.size != (28, 28):  
        raise ValueError("Expected 28x28 PNG")  
    x = np.asarray(img, dtype=np.float32) / 255.0  
    return np.clip(x, 0.0, 1.0)
```

```
def b64_png_from_x01(x2d: np.ndarray) -> str:  
    """Convert [0,1] array to base64 PNG.  
  
    Args:  
        x2d: Image array in [0,1] range
```

```

Returns:
    str: Base64 encoded PNG string
    """
x255 = np.clip((x2d * 255.0).round(), 0, 255).astype(np.uint8)
img = Image.fromarray(x255, mode="L")
buf = io.BytesIO()
img.save(buf, format="PNG", optimize=True)
return base64.b64encode(buf.getvalue()).decode("ascii")

def l2(a: np.ndarray, b: np.ndarray) -> float:
    """Compute L2 distance between two arrays.

    Args:
        a, b: Arrays to compare

    Returns:
        float: Euclidean distance
    """
    return float(np.linalg.norm((a - b).ravel(), ord=2))

# Fetch challenge and check clean prediction
ch = requests.get(f"{BASE_URL}/challenge", timeout=10).json()
x = x01_from_b64_png(ch["image_b64"]) # (28, 28)
lab = int(ch["label"]) # baseline label
tgt = int(ch["target"]) # target label
thr = float(ch["l2_threshold"]) # numeric threshold
res = requests.post(f"{BASE_URL}/predict", json={"image_b64":
b64_png_from_x01(x)}, timeout=10).json()
print({"baseline_label": lab, "target": tgt, "server_pred": res["pred"],
"l2_threshold": thr})

```

Model Architecture and Loading

The server uses this architecture. You need to replicate it locally to compute gradients.

```

class SimpleClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, 1)
        self.conv2 = nn.Conv2d(32, 64, 3, 1)
        self.dropout1 = nn.Dropout(0.25)
        self.dropout2 = nn.Dropout(0.5)

```

```

self.fc1 = nn.Linear(9216, 128)
self.fc2 = nn.Linear(128, 10)

def forward(self, x01: torch.Tensor) -> torch.Tensor:
    """Forward pass with internal normalization.

    Args:
        x01: Input tensor in [0,1] with shape (N, 1, 28, 28)

    Returns:
        Log-probabilities with shape (N, 10)
    """
    x = (x01 - MNIST_MEAN) / MNIST_STD
    x = torch.relu(self.conv1(x))
    x = torch.relu(self.conv2(x))
    x = torch.max_pool2d(x, 2)
    x = self.dropout1(x)
    x = torch.flatten(x, 1)
    x = torch.relu(self.fc1(x))
    x = self.dropout2(x)
    x = self.fc2(x)
    return torch.log_softmax(x, dim=1)

# Download and load weights
wt = requests.get(f"{BASE_URL}/weights", timeout=10).content
open("deepfool_weights.pth", "wb").write(wt)

model = SimpleClassifier().eval()
state = torch.load("deepfool_weights.pth",
map_location=torch.device("cpu"))
model.load_state_dict(state)

# Verify model works locally
x_tensor = torch.from_numpy(x[None, None, ...]).float()
logits = model(x_tensor)
local_pred = int(torch.argmax(logits, dim=1).item())
print(f"Local prediction: {local_pred}, should match server:
{res['pred']}")

```

Testing Server Validation

Verify server-side checks by submitting the clean image and observing expected failures.

```
bad = requests.post(f"{BASE_URL}/submit", json={"image_b64":
b64_png_from_x01(x)}, timeout=10)
print(bad.status_code, bad.text) # expected: 400 with "Wrong target:
predicted X, need 6"
```

Enable step-by-step solutions for all questions



Questions

Answer the question(s) below

to complete this Section and earn cubes!

Click here to spawn the target system!

Target(s): Fetching status...

Life Left: 0 minute(s)

+ 5 After successfully completing the challenge, what is the flag you receive?

+10 Streak pts

Submit

Skills Assessment 1

Your task is to craft a targeted adversarial example using FGSM.

Unlike the simple MNIST challenges, this assessment uses CIFAR-10 with color images and a more sophisticated classifier. You must transform a dog image into one the classifier predicts as a cat, while staying within a strict

L_∞

perturbation budget.

The constraint is

$\|x_{adv} - x\|_\infty \leq \epsilon$

where

$\epsilon = 8/255$

in pixel space. All three conditions must hold: the predicted class

equals the target (cat, class 3), the maximum absolute pixel difference

is at most

ϵ

and all pixel values remain in $[0,1]^{32 \times 32 \times 3}$.

Quick Start

Start your instance and check readiness:

```
export BASE_URL="http://instance_ip:port"
curl -s "$BASE_URL/health"
```

API

The API is small. Each response is JSON. Images are base64 PNG files of shape $32 \times 32 \times 3$ (RGB) in $[0,1]$ after decoding.

GET /health

Service health check.

```
curl -s "$BASE_URL/health"
```

GET /challenge

Returns the fixed sample and constraint.

```
curl -s "$BASE_URL/challenge" | jq
```

Example response shape:

```
{
  "challenge": "I-FGSM Targeted Attack on CIFAR-10",
  "sample_index": 42,
  "original_class": 5,
  "original_class_name": "dog",
  "target_class": 3,
  "target_class_name": "cat",
  "epsilon": 0.03137254901960784,
  "epsilon_description": "0.031373 (8/255)",
  "max_iterations_hint": 100,
  "normalization": {
```

```
    "mean": [0.4914, 0.4822, 0.4465],
    "std": [0.247, 0.2435, 0.2616]
  },
  "image": "<base64 PNG>",
  "instructions": "Implement targeted I-FGSM..."
}
```

GET /model/weights

Download the model weights file (`cifar10_model_best.pth` , ~6.3MB). Required for whitebox gradient computation.

```
curl -s "$BASE_URL/model/weights" -o cifar10_model_best.pth
```

POST /predict

Returns the model's prediction for an arbitrary image. Use this to validate your pipeline and intermediate candidates. Does not return the flag.

```
curl -s -X POST "$BASE_URL/predict" \
  -H 'content-type: application/json' \
  -d '{"image": "<base64 PNG of 32x32 RGB>"}' | jq
```

Example response shape:

```
{
  "predicted_class": 3,
  "predicted_class_name": "cat",
  "confidence": 0.85,
  "all_probabilities": {
    "airplane": 0.01,
    "automobile": 0.02,
    ...
  }
}
```

POST /submit

Validates that your candidate is a proper $32 \times 32 \times 3$ RGB image in $[0,1]$, that the L^∞ distance to the baseline image is at most ϵ , and that the classifier predicts the target class. Returns the flag on success.

```
curl -s -X POST "$BASE_URL/submit" \
  -H 'content-type: application/json' \
  -d '{"image": "<base64 PNG of 32x32 RGB>"}' | jq
```

Example success response shape:

```
{
  "success": true,
  "validation": {
    "l_inf_norm": 0.031373,
    "l_inf_satisfied": true,
    "valid_range": true,
    "adversarial_class": "cat",
    "target_achieved": true
  },
  "flag": "HTB{...}"
}
```

Model Architecture

The target model is a simple CNN trained on CIFAR-10. Save this code as `model.py`:

```
import torch
import torch.nn as nn

class CIFAR10CNN(nn.Module):
    def __init__(self, num_classes: int = 10):
        super(CIFAR10CNN, self).__init__()

        # First convolutional block
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
```



```

self.relu1 = nn.ReLU()
self.pool1 = nn.MaxPool2d(2, 2)

# Second convolutional block
self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
self.bn2 = nn.BatchNorm2d(64)
self.relu2 = nn.ReLU()
self.pool2 = nn.MaxPool2d(2, 2)

# Fully connected layers
self.fc1 = nn.Linear(64 * 8 * 8, 128)
self.relu3 = nn.ReLU()
self.dropout = nn.Dropout(0.5)
self.fc2 = nn.Linear(128, num_classes)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    x = self.pool1(self.relu1(self.bn1(self.conv1(x))))
    x = self.pool2(self.relu2(self.bn2(self.conv2(x))))
    x = x.view(x.size(0), -1) # Flatten
    x = self.dropout(self.relu3(self.fc1(x)))
    x = self.fc2(x)
    return x

def load_model(model_path: str, device: str = "cuda") -> CIFAR10CNN:
    model = CIFAR10CNN(num_classes=10)

    # Load checkpoint
    checkpoint = torch.load(model_path, map_location=device)

    # Handle both direct state_dict and checkpoint dict formats
    if isinstance(checkpoint, dict) and "model_state_dict" in checkpoint:
        model.load_state_dict(checkpoint["model_state_dict"])
    else:
        model.load_state_dict(checkpoint)

    model = model.to(device)
    model.eval()

    return model

# CIFAR-10 class names
CIFAR10_CLASSES = [
    "airplane",

```

```

    "automobile",
    "bird",
    "cat",
    "deer",
    "dog",
    "frog",
    "horse",
    "ship",
    "truck",
]

# Normalization parameters (computed from CIFAR-10 training set)
NORMALIZATION_MEAN = [0.4914, 0.4822, 0.4465]
NORMALIZATION_STD = [0.247, 0.2435, 0.2616]

```

Minimal Python Scaffolds

The following helpers fetch the challenge, convert between base64 PNG and `[0,1]` tensors, and call the API.

```

import os, io, base64, requests
import torch
import torch.nn.functional as F
import torchvision.transforms as transforms
from PIL import Image
import numpy as np

BASE_URL = os.getenv("BASE_URL", "http://127.0.0.1:8000")

def tensor_from_b64_png(b64: str) -> torch.Tensor:
    raw = base64.b64decode(b64)
    img = Image.open(io.BytesIO(raw))
    if img.size != (32, 32):
        raise ValueError("Expected 32x32 PNG")
    tensor = transforms.ToTensor()(img) # converts to (3, 32, 32) in
[0,1]
    return tensor

def b64_png_from_tensor(tensor: torch.Tensor) -> str:
    img_array = (tensor.permute(1, 2, 0).numpy() * 255).astype(np.uint8)
    img = Image.fromarray(img_array)
    buf = io.BytesIO()

```

```

img.save(buf, format="PNG")
buf.seek(0)
return base64.b64encode(buf.getvalue()).decode("ascii")

def linf(a: torch.Tensor, b: torch.Tensor) -> float:
    return float(torch.abs(a - b).max())

# Download model weights
weights_path = "cifar10_model_best.pth"
if not os.path.exists(weights_path):
    print("Downloading model weights...")
    resp = requests.get(f"{BASE_URL}/model/weights")
    with open(weights_path, "wb") as f:
        f.write(resp.content)
    print(f"Saved to {weights_path}")

# Load model (assumes model.py from architecture section above)
from model import load_model, NORMALIZATION_MEAN, NORMALIZATION_STD

device = "cuda" if torch.cuda.is_available() else "cpu"
model = load_model(weights_path, device=device)
print(f"Model loaded on {device}")

# Fetch challenge
ch = requests.get(f"{BASE_URL}/challenge", timeout=10).json()
x = tensor_from_b64_png(ch["image"]) # (3, 32, 32)
orig_class = int(ch["original_class"]) # 5 (dog)
target_class = int(ch["target_class"]) # 3 (cat)
epsilon = float(ch["epsilon"]) # 8/255
mean = torch.tensor(ch["normalization"]["mean"]).view(3, 1, 1)
std = torch.tensor(ch["normalization"]["std"]).view(3, 1, 1)

# Verify clean prediction
x_norm = (x - mean) / std
with torch.no_grad():
    pred = model(x_norm.unsqueeze(0).to(device)).argmax(dim=1).item()
print(
    f"Original: class {orig_class}, Target: class {target_class}, Clean  

    pred: {pred}, Epsilon: {epsilon:.6f}"
)

```

To verify server-side checks, submit the clean image and observe expected failures:

```

bad = requests.post(
    f"{BASE_URL}/submit",
    json={"image": b64_png_from_tensor(x)},
    timeout=10
)
print(bad.status_code, bad.text) # expected: success=false, "Target not
achieved"

```

Enable step-by-step solutions for all questions



Questions

Answer the question(s) below
to complete this Section and earn cubes!

Click here to spawn the target
system!

Target(s): Fetching status...

Life Left: 0 minute(s)

+ 5 After successfully completing the assessment task, what is the flag you receive?

+10 Streak pts

Submit

Skills Assessment 2

Your task is to implement the DeepFool algorithm to find a minimal L_2 perturbation that causes misclassification on CIFAR-10. This is an untargeted attack where any misclassification succeeds.

The constraint is

$$\|x_{adv} - x\|_2 \leq \text{threshold} \quad \text{or} \quad \frac{\|x_{adv} - x\|_2}{\|x\|_2} \leq \text{threshold}$$

where the

L_2 norm

is measured in normalized space. All three conditions must hold:

the predicted class differs from the original (any class except horse),

the

L_2 norm

distance in normalized space is at most the threshold (3.5), and all pixel values remain in $[0,1]^{32 \times 32 \times 3}$.

Quick Start

Start your instance and check readiness:

```
export BASE_URL="http://instance_ip:port"
curl -s "$BASE_URL/health"
```

API

The API is small. Each response is JSON. Images are base64 PNG files of shape $32 \times 32 \times 3$ (RGB) in $[0,1]$ after decoding.

GET /health

Service health check.

```
curl -s "$BASE_URL/health"
```

GET /challenge

Returns the fixed sample and constraint.

```
curl -s "$BASE_URL/challenge" | jq
```

Example response shape:

<https://academy.hackthebox.com/path/preview/ai-red-teamer>

```
{
  "challenge": "DeepFool Minimal Perturbation on CIFAR-10",
  "sample_index": 137,
  "original_class": 7,
  "original_class_name": "horse",
  "l2_threshold": 3.5,
  "num_classes_hint": 10,
  "overshoot_hint": 0.02,
  "max_iterations_hint": 50,
```

```
"normalization": {  
  "mean": [0.4914, 0.4822, 0.4465],  
  "std": [0.247, 0.2435, 0.2616]  
},  
"image": "<base64 PNG>",  
"instructions": "Find minimal L2 perturbation..."  
}
```

GET /model/weights

Download the model weights file (`cifar10_model_best.pth` , ~6.3MB). Required for whitebox gradient computation.

```
curl -s "$BASE_URL/model/weights" -o cifar10_model_best.pth
```

POST /predict

Returns the model's prediction for an arbitrary image. Use this to validate your pipeline and intermediate candidates. Does not return the flag.

```
curl -s -X POST "$BASE_URL/predict" \  
-H 'content-type: application/json' \  
-d '{"image": "<base64 PNG of 32x32 RGB>"}' | jq
```

Example response shape:

```
{  
  "predicted_class": 9,  
  "predicted_class_name": "truck",  
  "confidence": 0.62,  
  "all_probabilities": {  
    "airplane": 0.01,  
    "automobile": 0.02,  
    ...  
  }  
}
```

POST /submit

Validates that your candidate is a proper `32×32×3` RGB image in `[0,1]`, that the ℓ_2 distance to the baseline image in normalized space is at most `l2_threshold`, and that the classifier misclassifies it. Returns the flag on success.

```
curl -s -X POST "$BASE_URL/submit" \
  -H 'content-type: application/json' \
  -d '{"image": "<base64 PNG of 32x32 RGB>"}' | jq
```

Example success response shape:

```
{
  "success": true,
  "validation": {
    "l2_norm": 0.9632,
    "l2_threshold": 3.5,
    "l2_satisfied": true,
    "valid_range": true,
    "original_class": "horse",
    "adversarial_class": "truck",
    "misclassification": true
  },
  "flag": "HTB{...}"
}
```

Model Architecture

The target model is a simple CNN trained on CIFAR-10. Save this code as `model.py`:

```
import torch
import torch.nn as nn

class CIFAR10CNN(nn.Module):
    def __init__(self, num_classes: int = 10):
        super(CIFAR10CNN, self).__init__()

        # First convolutional block
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
```

```

self.bn1 = nn.BatchNorm2d(32)
self.relu1 = nn.ReLU()
self.pool1 = nn.MaxPool2d(2, 2)

# Second convolutional block
self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
self.bn2 = nn.BatchNorm2d(64)
self.relu2 = nn.ReLU()
self.pool2 = nn.MaxPool2d(2, 2)

# Fully connected layers
self.fc1 = nn.Linear(64 * 8 * 8, 128)
self.relu3 = nn.ReLU()
self.dropout = nn.Dropout(0.5)
self.fc2 = nn.Linear(128, num_classes)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    x = self.pool1(self.relu1(self.bn1(self.conv1(x))))
    x = self.pool2(self.relu2(self.bn2(self.conv2(x))))
    x = x.view(x.size(0), -1) # Flatten
    x = self.dropout(self.relu3(self.fc1(x)))
    x = self.fc2(x)
    return x

def load_model(model_path: str, device: str = 'cuda') -> CIFAR10CNN:
    model = CIFAR10CNN(num_classes=10)

    # Load checkpoint
    checkpoint = torch.load(model_path, map_location=device)

    # Handle both direct state_dict and checkpoint dict formats
    if isinstance(checkpoint, dict) and 'model_state_dict' in checkpoint:
        model.load_state_dict(checkpoint['model_state_dict'])
    else:
        model.load_state_dict(checkpoint)

    model = model.to(device)
    model.eval()

    return model

# CIFAR-10 class names
CIFAR10_CLASSES = [

```



```

    'airplane', 'automobile', 'bird', 'cat', 'deer',
    'dog', 'frog', 'horse', 'ship', 'truck'
]

# Normalization parameters (computed from CIFAR-10 training set)
NORMALIZATION_MEAN = [0.4914, 0.4822, 0.4465]
NORMALIZATION_STD = [0.247, 0.2435, 0.2616]

```

Minimal Python Scaffolds

The following helpers fetch the challenge, convert between base64 PNG and `[0,1]` tensors, and call the API.

```

import os, io, base64, requests
import torch
import torch.nn.functional as F
import torchvision.transforms as transforms
from PIL import Image
import numpy as np

BASE_URL = os.getenv("BASE_URL", "http://127.0.0.1:8001")

def tensor_from_b64_png(b64: str) -> torch.Tensor:
    raw = base64.b64decode(b64)
    img = Image.open(io.BytesIO(raw))
    if img.size != (32, 32):
        raise ValueError("Expected 32x32 PNG")
    tensor = transforms.ToTensor()(img) # converts to (3, 32, 32) in
[0,1]
    return tensor

def b64_png_from_tensor(tensor: torch.Tensor) -> str:
    img_array = (tensor.permute(1, 2, 0).numpy() * 255).astype(np.uint8)
    img = Image.fromarray(img_array)
    buf = io.BytesIO()
    img.save(buf, format="PNG")
    buf.seek(0)
    return base64.b64encode(buf.getvalue()).decode("ascii")

def l2_normalized_space(a: torch.Tensor, b: torch.Tensor, mean, std) ->
float:
    mean_t = torch.tensor(mean).view(3, 1, 1)

```

```

std_t = torch.tensor(std).view(3, 1, 1)
a_norm = (a - mean_t) / std_t
b_norm = (b - mean_t) / std_t
return float(torch.norm(a_norm - b_norm))

# Download model weights
weights_path = "cifar10_model_best.pth"
if not os.path.exists(weights_path):
    print("Downloading model weights...")
    resp = requests.get(f"{BASE_URL}/model/weights")
    with open(weights_path, "wb") as f:
        f.write(resp.content)
    print(f"Saved to {weights_path}")

# Load model (assumes model.py from architecture section above)
from model import load_model, NORMALIZATION_MEAN, NORMALIZATION_STD

device = "cuda" if torch.cuda.is_available() else "cpu"
model = load_model(weights_path, device=device)
print(f"Model loaded on {device}")

# Fetch challenge
ch = requests.get(f"{BASE_URL}/challenge", timeout=10).json()
x = tensor_from_b64_png(ch["image"]) # (3, 32, 32)
orig_class = int(ch["original_class"]) # 7 (horse)
l2_threshold = float(ch["l2_threshold"]) # 3.5
mean = ch["normalization"]["mean"]
std = ch["normalization"]["std"]
mean_t = torch.tensor(mean).view(3, 1, 1)
std_t = torch.tensor(std).view(3, 1, 1)

# Verify clean prediction
x_norm = (x - mean_t) / std_t
with torch.no_grad():
    pred = model(x_norm.unsqueeze(0).to(device)).argmax(dim=1).item()
print(f"Original: class {orig_class}, Clean pred: {pred}, L2 threshold: {l2_threshold}")

```

To verify server-side checks, submit the clean image and observe expected failures:

```

bad = requests.post(
    f"{BASE_URL}/submit",

```

```
    json={"image": b64_png_from_tensor(x)},  
    timeout=10  
)  
print(bad.status_code, bad.text) # expected: success=false,  
"Misclassification not achieved"
```

Enable step-by-step solutions for all questions



Questions

Answer the question(s) below
to complete this Section and earn cubes!

Click here to spawn the target
system!

Target(s): Fetching status...

Life Left: 0 minute(s)

+ 5 After successfully completing the assessment task, what is the flag you receive?

+10 Streak pts

Submit